

Uniformes Brenda

Generado por Doxygen 1.12.0

1 Índice de clases	1
1.1 Lista de clases	1
2 Índice de archivos	3
2.1 Lista de archivos	3
3 Documentación de clases	5
3.1 Referencia de la clase <code>AlertasStockResult</code>	5
3.1.1 Descripción detallada	5
3.1.2 Documentación de constructores y destructores	5
3.1.2.1 <code>AlertasStockResult()</code>	5
3.1.3 Documentación de datos miembro	6
3.1.3.1 <code>agotados</code>	6
3.1.3.2 <code>criticos</code> [1/2]	6
3.1.3.3 <code>criticos</code> [2/2]	6
3.1.3.4 <code>hayAlertas</code>	6
3.1.3.5 <code>todas</code>	6
3.1.3.6 <code>totalAgotados</code>	6
3.1.3.7 <code>totalCriticos</code>	6
3.2 Referencia de la clase <code>BackupInfo</code>	6
3.2.1 Descripción detallada	7
3.2.2 Documentación de constructores y destructores	7
3.2.2.1 <code>BackupInfo()</code>	7
3.2.3 Documentación de datos miembro	7
3.2.3.1 <code>fechaCreacion</code>	7
3.2.3.2 <code>nombre</code>	7
3.2.3.3 <code>ruta</code>	8
3.2.3.4 <code>tamanoBytes</code>	8
3.2.3.5 <code>tamanoLegible</code>	8
3.3 Referencia de la clase <code>BackupRepository</code>	8
3.3.1 Descripción detallada	8
3.3.2 Documentación de funciones miembro	9
3.3.2.1 <code>_obtenerDirectorioBackups()</code>	9
3.3.2.2 <code>crearBackup()</code>	9
3.4 Referencia de la clase <code>BackupService</code>	9
3.4.1 Descripción detallada	10
3.4.2 Documentación de funciones miembro	10
3.4.2.1 <code>_directoriotBackups()</code>	10
3.4.2.2 <code>_limpiarBackupsAntiguos()</code>	10
3.4.2.3 <code>compartirBackup()</code>	10
3.4.2.4 <code>crearBackup()</code>	10
3.4.2.5 <code>crearYCompartirBackup()</code>	11
3.4.2.6 <code>eliminarBackup()</code>	11

3.4.2.7 listarBackups()	11
3.4.3 Documentación de datos miembro	11
3.4.3.1 _backupRepository	11
3.4.3.2 _maxBackups	11
3.5 Referencia de la clase ChangeNotifier	12
3.5.1 Descripción detallada	17
3.5.2 Documentación de funciones miembro	18
3.5.2.1 _inicializarDatos()	18
3.5.2.2 agregarEscuela()	19
3.5.2.3 agregarManual()	19
3.5.2.4 agregarQr()	19
3.5.2.5 agregarUnidades()	19
3.5.2.6 cambiarEscuela()	19
3.5.2.7 cambiarMonth()	20
3.5.2.8 cambiarYear()	20
3.5.2.9 cargarAlertas()	20
3.5.2.10 cargarCatalogos()	20
3.5.2.11 cargarDetalles() [1/2]	20
3.5.2.12 cargarDetalles() [2/2]	20
3.5.2.13 cargarInventario() [1/2]	21
3.5.2.14 cargarInventario() [2/2]	21
3.5.2.15 cargarPedidos()	21
3.5.2.16 cargarPendientes()	21
3.5.2.17 cargarReportes()	21
3.5.2.18 cargarVentas()	21
3.5.2.19 completarPedido()	22
3.5.2.20 confirmar()	22
3.5.2.21 desregistrarUnidad()	22
3.5.2.22 eliminarDetallePedido()	22
3.5.2.23 eliminarItem()	22
3.5.2.24 eliminarUnidad()	23
3.5.2.25 firstWhere()	23
3.5.2.26 imprimirQrExistente()	23
3.5.2.27 inicializar()	23
3.5.2.28 InventarioProvider()	23
3.5.2.29 limpiar() [1/3]	23
3.5.2.30 limpiar() [2/3]	24
3.5.2.31 limpiar() [3/3]	24
3.5.2.32 limpiarError()	24
3.5.2.33 limpiarMensaje()	24
3.5.2.34 login()	24
3.5.2.35 logout()	24

3.5.2.36 navigateToInventory()	25
3.5.2.37 navigateToNewOrder()	25
3.5.2.38 navigateToOrders()	25
3.5.2.39 pedidoCompleto()	25
3.5.2.40 PrintProvider()	25
3.5.2.41 procesarQr()	25
3.5.2.42 QrProvider()	26
3.5.2.43 recargarEscuelas()	26
3.5.2.44 refrescar()	26
3.5.2.45 refrescarAlertas()	26
3.5.2.46 RegistrarInventarioProvider()	26
3.5.2.47 registrarQrPedido()	26
3.5.2.48 return() [1/2]	27
3.5.2.49 return() [2/2]	27
3.5.2.50 seleccionarPrenda()	27
3.5.2.51 setUsername()	27
3.5.2.52 solicitarPermisosBluetooth()	27
3.5.3 Documentación de datos miembro	27
3.5.3.1 _agotados	27
3.5.3.2 _cargando [1/2]	27
3.5.3.3 _cargando [2/2]	28
3.5.3.4 _criticos [1/2]	28
3.5.3.5 _criticos [2/2]	28
3.5.3.6 _detalles	28
3.5.3.7 _error	28
3.5.3.8 _escuelaRepo	28
3.5.3.9 _escuelaRepository	28
3.5.3.10 _escuelas	29
3.5.3.11 _escuelaSeleccionada	29
3.5.3.12 _idPrendaSeleccionada	29
3.5.3.13 _inventario	29
3.5.3.14 _inventarioRepo	29
3.5.3.15 _inventarioRepository	29
3.5.3.16 _items	29
3.5.3.17 _itemsInventario	30
3.5.3.18 _loginUseCase	30
3.5.3.19 _pedidos	30
3.5.3.20 _pendientes	30
3.5.3.21 _prendaRepo	30
3.5.3.22 _prendas	30
3.5.3.23 _procesando	30
3.5.3.24 _repository [1/2]	31

3.5.3.25 _repository [2/2]	31
3.5.3.26 _unidades	31
3.5.3.27 _useCase [1/4]	31
3.5.3.28 _useCase [2/4]	31
3.5.3.29 _useCase [3/4]	31
3.5.3.30 _useCase [4/4]	31
3.5.3.31 _userName	32
3.5.3.32 _usuarioActual	32
3.5.3.33 _usuarioRepo	32
3.5.3.34 _ventas	32
3.5.3.35 agotados	32
3.5.3.36 alertasProvider	32
3.5.3.37 cantidadVentas	33
3.5.3.38 cargando [1/3]	33
3.5.3.39 cargando [2/3]	33
3.5.3.40 cargando [3/3]	33
3.5.3.41 criticos	33
3.5.3.42 detalles	33
3.5.3.43 error [1/2]	33
3.5.3.44 error [2/2]	34
3.5.3.45 escuelaRepository	34
3.5.3.46 escuelas [1/2]	34
3.5.3.47 escuelas [2/2]	34
3.5.3.48 escuelaSeleccionada	34
3.5.3.49 escuelasMasVentas	34
3.5.3.50 estaAutenticado	34
3.5.3.51 hayAlertas	35
3.5.3.52 idEscuelaSeleccionada	35
3.5.3.53 idPrendaSeleccionada	35
3.5.3.54 inventario	35
3.5.3.55 items	35
3.5.3.56 itemsInventario	35
3.5.3.57 loading	35
3.5.3.58 maxCantidadPrenda	35
3.5.3.59 maxTotalEscuela	36
3.5.3.60 mensaje	36
3.5.3.61 meses [1/2]	36
3.5.3.62 meses [2/2]	36
3.5.3.63 month	36
3.5.3.64 nombreEscuelaSeleccionada	37
3.5.3.65 nombreMesSeleccionado	37
3.5.3.66 pedidos	37

3.5.3.67 pendientes	37
3.5.3.68 prendaRepository	37
3.5.3.69 prendas [1/2]	37
3.5.3.70 prendas [2/2]	37
3.5.3.71 prendasMasVendidas	38
3.5.3.72 procesando	38
3.5.3.73 tallaRepository	38
3.5.3.74 tallas	38
3.5.3.75 todas	38
3.5.3.76 total	38
3.5.3.77 totalAlertas	38
3.5.3.78 totalPrendas	39
3.5.3.79 totalVendido	39
3.5.3.80 unidades	39
3.5.3.81 useCase [1/2]	39
3.5.3.82 useCase [2/2]	39
3.5.3.83 userName	39
3.5.3.84 usuarioActual	39
3.5.3.85 ventas	39
3.5.3.86 year	40
3.6 Referencia de la clase CustomPainter	40
3.6.1 Descripción detallada	40
3.6.2 Documentación de funciones miembro	40
3.6.2.1 paint()	40
3.6.2.2 shouldRepaint()	41
3.7 Referencia de la clase DatabaseHelper	41
3.7.1 Descripción detallada	41
3.7.2 Documentación de funciones miembro	42
3.7.2.1 _initDB()	42
3.7.2.2 _internal()	42
3.7.2.3 _onCreate()	42
3.7.3 Documentación de datos miembro	43
3.7.3.1 _database [1/2]	43
3.7.3.2 _database [2/2]	43
3.7.3.3 async	43
3.7.3.4 instance	43
3.8 Referencia de la clase EscuelaRepository	43
3.8.1 Descripción detallada	44
3.8.2 Documentación de funciones miembro	44
3.8.2.1 actualizar()	44
3.8.2.2 eliminar()	44
3.8.2.3 insertar()	44

3.8.2.4 obtenerPorId()	45
3.8.2.5 obtenerTodas()	45
3.8.3 Documentación de datos miembro	45
3.8.3.1 async	45
3.9 Referencia de la clase InventarioRepository	45
3.9.1 Descripción detallada	46
3.9.2 Documentación de funciones miembro	46
3.9.2.1 actualizar()	46
3.9.2.2 contarStock()	46
3.9.2.3 eliminar()	46
3.9.2.4 insertar()	47
3.9.2.5 obtenerInventarioCompleto()	47
3.9.2.6 obtenerInventarioFiltrado()	47
3.9.2.7 obtenerPorCombinacion()	47
3.9.2.8 obtenerPorEscuela()	47
3.9.2.9 obtenerPorId()	48
3.9.2.10 obtenerStockAgotado()	48
3.9.2.11 obtenerStockCritico()	48
3.9.2.12 obtenerTodos()	48
3.9.3 Documentación de datos miembro	48
3.9.3.1 async	48
3.10 Referencia de la clase LoginUseCase	48
3.10.1 Descripción detallada	49
3.10.2 Documentación de funciones miembro	49
3.10.2.1 _hashPassword()	49
3.10.2.2 execute()	49
3.11 Referencia de la clase NuevaOrdenUseCase	49
3.11.1 Descripción detallada	50
3.11.2 Documentación de funciones miembro	50
3.11.2.1 _calcularTotal()	50
3.11.2.2 _confirmarPedido()	51
3.11.2.3 _confirmarVenta()	51
3.11.2.4 _resolverUnidades()	51
3.11.2.5 agregarManual()	51
3.11.2.6 agregarPorQr()	52
3.11.2.7 confirmarMixto()	52
3.11.3 Documentación de datos miembro	52
3.11.3.1 _inventarioRepository	52
3.11.3.2 _ordenRepository	52
3.11.3.3 _pedidoRepository	52
3.11.3.4 _qrUseCase	52
3.11.3.5 _unidadRepository	52

3.11.3.6 _ventaRepository	53
3.12 Referencia de la clase OrdenRepository	53
3.12.1 Descripción detallada	53
3.12.2 Documentación de funciones miembro	53
3.12.2.1 insertar()	53
3.12.2.2 obtenerPorId()	53
3.12.2.3 obtenerTodas()	54
3.12.3 Documentación de datos miembro	54
3.12.3.1 async	54
3.13 Referencia de la clase PedidoPendienteUseCase	54
3.13.1 Descripción detallada	55
3.13.2 Documentación de funciones miembro	55
3.13.2.1 completarPedido()	55
3.13.2.2 desregistrarUnidad()	55
3.13.2.3 eliminarDetallePedido()	55
3.13.2.4 obtenerDetalles()	56
3.13.2.5 obtenerDetallesConInfo()	56
3.13.2.6 obtenerPedidosPendientes()	56
3.13.2.7 pedidoCompleto()	56
3.13.2.8 registrarQrPedido()	56
3.13.2.9 registrarUnidad()	57
3.13.3 Documentación de datos miembro	57
3.13.3.1 _pedidoRepository	57
3.13.3.2 _qrUseCase	57
3.13.3.3 _unidadRepository	57
3.13.3.4 _ventaRepository	57
3.14 Referencia de la clase PedidoRepository	57
3.14.1 Descripción detallada	58
3.14.2 Documentación de funciones miembro	58
3.14.2.1 actualizarEstado()	58
3.14.2.2 contarDetalles()	59
3.14.2.3 desregistrarUnidad()	59
3.14.2.4 eliminar()	59
3.14.2.5 eliminarDetallePedido()	59
3.14.2.6 eliminarPedidoCompleto()	59
3.14.2.7 insertarPedidoYDetalles()	59
3.14.2.8 obtenerDetalles()	60
3.14.2.9 obtenerDetallesConInfo()	60
3.14.2.10 obtenerPorId()	60
3.14.2.11 obtenerRegistroActivoPorUnidad()	60
3.14.2.12 obtenerTodos()	60
3.14.2.13 pedidoCompleto()	60

3.14.2.14 registrarUnidad()	61
3.14.3 Documentación de datos miembro	61
3.14.3.1 async	61
3.15 Referencia de la clase PrendaRepository	61
3.15.1 Descripción detallada	61
3.15.2 Documentación de funciones miembro	62
3.15.2.1 actualizar()	62
3.15.2.2 eliminar()	62
3.15.2.3 insertar()	62
3.15.2.4 obtenerPorId()	62
3.15.2.5 obtenerTodas()	62
3.15.3 Documentación de datos miembro	63
3.15.3.1 async	63
3.16 Referencia de la clase PrintService	63
3.16.1 Descripción detallada	63
3.16.2 Documentación de funciones miembro	63
3.16.2.1 conectar()	63
3.16.2.2 desconectar()	63
3.16.2.3 imprimirQR()	63
3.17 Referencia de la clase PrintUseCase	64
3.17.1 Descripción detallada	64
3.17.2 Documentación de constructores y destructores	64
3.17.2.1 PrintUseCase()	64
3.17.3 Documentación de funciones miembro	64
3.17.3.1 ejecutar()	64
3.17.3.2 imprimirQrExistente()	65
3.17.4 Documentación de datos miembro	65
3.17.4.1 printer	65
3.17.4.2 repo	65
3.18 Referencia de la clase QrUseCase	65
3.18.1 Descripción detallada	66
3.18.2 Documentación de constructores y destructores	66
3.18.2.1 QrUseCase()	66
3.18.3 Documentación de funciones miembro	66
3.18.3.1 existeQr()	66
3.18.3.2 obtenerUnidad()	66
3.18.4 Documentación de datos miembro	66
3.18.4.1 unidadRepository	66
3.19 Referencia de la clase RegistrarInventarioUseCase	67
3.19.1 Descripción detallada	67
3.19.2 Documentación de funciones miembro	67
3.19.2.1 ejecutar()	67

3.19.3 Documentación de datos miembro	68
3.19.3.1 _escuelaRepository	68
3.19.3.2 _inventarioRepository	68
3.19.3.3 _prendaRepository	68
3.19.3.4 _tallaRepository	68
3.20 Referencia de la clase RegistroTallaResult	68
3.20.1 Descripción detallada	68
3.20.2 Documentación de constructores y destructores	69
3.20.2.1 RegistroTallaResult()	69
3.20.3 Documentación de datos miembro	69
3.20.3.1 exitoso	69
3.20.3.2 idTalla	69
3.20.3.3 mensajeError	69
3.20.3.4 nombreTalla	69
3.21 Referencia de la clase ReporteRepository	69
3.21.1 Descripción detallada	70
3.21.2 Documentación de funciones miembro	70
3.21.2.1 obtenerCantidadVentas()	70
3.21.2.2 obtenerEscuelasMasVentas()	70
3.21.2.3 obtenerPrendasMasVendidas()	70
3.21.2.4 obtenerTotalVendido()	71
3.21.3 Documentación de datos miembro	71
3.21.3.1 async	71
3.22 Referencia de la clase StatefulWidget	71
3.22.1 Descripción detallada	72
3.22.2 Documentación de funciones miembro	73
3.22.2.1 AdministrarCantidadScreen()	73
3.22.2.2 AdministrarPrendaScreen()	74
3.22.2.3 AgregarPedidoScreen()	74
3.22.2.4 AgregarPrendasPedidoScreen()	74
3.22.2.5 createState() [1/12]	74
3.22.2.6 createState() [2/12]	74
3.22.2.7 createState() [3/12]	74
3.22.2.8 createState() [4/12]	74
3.22.2.9 createState() [5/12]	74
3.22.2.10 createState() [6/12]	74
3.22.2.11 createState() [7/12]	74
3.22.2.12 createState() [8/12]	75
3.22.2.13 createState() [9/12]	75
3.22.2.14 createState() [10/12]	75
3.22.2.15 createState() [11/12]	75
3.22.2.16 createState() [12/12]	75

3.22.2.17	ExitScreen()	75
3.22.2.18	Function()	75
3.22.2.19	LoginScreen()	75
3.22.2.20	PedidosPendientesScreen()	75
3.22.2.21	QRScannerScreen()	76
3.22.2.22	RegistrarPrendaScreen()	76
3.22.2.23	RevisarPedidoScreen()	76
3.22.2.24	RevisarVentaScreen()	76
3.22.2.25	VentasScreen()	76
3.22.3	Documentación de datos miembro	76
3.22.3.1	cantidad	76
3.22.3.2	idInventario	76
3.22.3.3	idPedido	77
3.22.3.4	idVenta	77
3.22.3.5	mostrarMensaje	77
3.22.3.6	nombreEscuela	77
3.22.3.7	nombrePrenda	77
3.22.3.8	registrarInventarioUseCase	77
3.22.3.9	talla	77
3.23	Referencia de la clase StatelessWidget	78
3.23.1	Descripción detallada	79
3.23.2	Documentación de funciones miembro	80
3.23.2.1	_Campo()	80
3.23.2.2	_FiltrosSection()	80
3.23.2.3	_HomeButton()	81
3.23.2.4	_RankingCard()	81
3.23.2.5	_StatCard()	81
3.23.2.6	AlertasPopup()	81
3.23.2.7	BottomNavBar()	81
3.23.2.8	build() [1/12]	81
3.23.2.9	build() [2/12]	81
3.23.2.10	build() [3/12]	82
3.23.2.11	build() [4/12]	82
3.23.2.12	build() [5/12]	82
3.23.2.13	build() [6/12]	82
3.23.2.14	build() [7/12]	82
3.23.2.15	build() [8/12]	82
3.23.2.16	build() [9/12]	82
3.23.2.17	build() [10/12]	83
3.23.2.18	build() [11/12]	83
3.23.2.19	build() [12/12]	83
3.23.2.20	HomeScreen()	83

3.23.2.21 InventarioScreen()	83
3.23.2.22 MyApp()	84
3.23.2.23 QrPendientesScreen()	84
3.23.2.24 ReportesScreen()	84
3.23.3 Documentación de datos miembro	84
3.23.3.1 _inputDecoration	84
3.23.3.2 backgroundColor	84
3.23.3.3 color	84
3.23.3.4 controller	85
3.23.3.5 esDinero	85
3.23.3.6 esPassword	85
3.23.3.7 hint	85
3.23.3.8 icon	85
3.23.3.9 iconColor	85
3.23.3.10 icono	85
3.23.3.11 items	85
3.23.3.12 label	85
3.23.3.13 onPressed	86
3.23.3.14 onTogglePassword	86
3.23.3.15 provider	86
3.23.3.16 textColor	86
3.23.3.17 valor	86
3.23.3.18 verPassword	86
3.24 Referencia de la clase TallaRepository	86
3.24.1 Descripción detallada	87
3.24.2 Documentación de funciones miembro	87
3.24.2.1 actualizar()	87
3.24.2.2 eliminar()	87
3.24.2.3 insertar()	87
3.24.2.4 obtenerPorId()	88
3.24.2.5 obtenerTodas()	88
3.24.3 Documentación de datos miembro	88
3.24.3.1 async	88
3.25 Referencia de la clase UnidadRepository	88
3.25.1 Descripción detallada	89
3.25.2 Documentación de funciones miembro	89
3.25.2.1 desactivar()	89
3.25.2.2 eliminar()	89
3.25.2.3 insertarUnidades()	89
3.25.2.4 marcarPendienteImpresion()	89
3.25.2.5 obtenerPendientesImpresion()	90
3.25.2.6 obtenerPorId()	90

3.25.2.7 obtenerPorInventario()	90
3.25.2.8 pertenece()	90
3.25.2.9 quitarPendienteImpresion()	90
3.25.2.10 reactivar()	90
3.25.3 Documentación de datos miembro	91
3.25.3.1 async	91
3.26 Referencia de la clase UsuarioRepository	91
3.26.1 Descripción detallada	91
3.26.2 Documentación de funciones miembro	92
3.26.2.1 actualizar()	92
3.26.2.2 desactivar()	92
3.26.2.3 eliminar()	92
3.26.2.4 insertar()	92
3.26.2.5 obtenerActivos()	92
3.26.2.6 obtenerPorId()	93
3.26.2.7 obtenerPorUsuario()	93
3.26.2.8 obtenerTodos()	93
3.26.3 Documentación de datos miembro	93
3.26.3.1 async	93
3.27 Referencia de la clase ValidacionInventario	93
3.27.1 Descripción detallada	94
3.27.2 Documentación de funciones miembro	94
3.27.2.1 validarCamposObligatorios()	94
3.27.2.2 validarCantidadUnidades()	94
3.27.2.3 validarEliminacion()	94
3.27.2.4 validarPrecio()	95
3.28 Referencia de la clase VentaRepository	95
3.28.1 Descripción detallada	95
3.28.2 Documentación de funciones miembro	96
3.28.2.1 actualizarEstadoCancelado()	96
3.28.2.2 contarDetallesVenta()	96
3.28.2.3 eliminar()	96
3.28.2.4 eliminarDetalleVenta()	96
3.28.2.5 insertarDetallesVenta()	96
3.28.2.6 insertarVentaYDetalles()	97
3.28.2.7 obtenerDetallesPorVenta()	97
3.28.2.8 obtenerPorId()	97
3.28.2.9 obtenerPorUsuario()	97
3.28.2.10 obtenerTodas()	97
3.28.3 Documentación de datos miembro	98
3.28.3.1 async	98
3.29 Referencia de la clase VentasUseCase	98

3.29.1 Descripción detallada	98
3.29.2 Documentación de funciones miembro	98
3.29.2.1 obtenerDetallesVenta()	98
3.29.2.2 obtenerVenta()	99
3.29.2.3 obtenerVentas()	99
3.29.3 Documentación de datos miembro	99
3.29.3.1 _ventaRepository	99
4 Documentación de archivos	101
4.1 Referencia del archivo lib/business/providers/auth_provider.dart	101
4.2 Referencia del archivo lib/business/providers/inventario_provider.dart	101
4.3 Referencia del archivo lib/business/services/backup_service.dart	101
4.3.1 Documentación de funciones	102
4.3.1.1 BackupResult()	102
4.3.2 Documentación de variables	102
4.3.2.1 backup	102
4.3.2.2 exitoso	102
4.3.2.3 mensaje	102
4.4 Referencia del archivo lib/business/services/print_service.dart	102
4.4.1 Documentación de funciones	103
4.4.1.1 _conectar()	103
4.4.1.2 _desconectar()	103
4.4.1.3 conectar()	104
4.4.1.4 desconectar()	104
4.4.1.5 imprimirQR()	104
4.4.2 Documentación de variables	104
4.4.2.1 _char	104
4.4.2.2 _device	104
4.4.2.3 serviceUuid	104
4.5 Referencia del archivo lib/business/usecases/alertas_stock_usecase.dart	105
4.5.1 Documentación de enumeraciones	105
4.5.1.1 TipoAlerta	105
4.5.2 Documentación de funciones	106
4.5.2.1 AlertaStock()	106
4.5.2.2 obtenerAgotados()	106
4.5.2.3 obtenerAlertas()	106
4.5.2.4 obtenerCriticos()	106
4.5.2.5 obtenerResumen()	106
4.5.2.6 verificarItem()	107
4.5.3 Documentación de variables	107
4.5.3.1 _inventarioRepository	107
4.5.3.2 escuela	107

4.5.3.3 idInventario	107
4.5.3.4 mensaje	107
4.5.3.5 precio	108
4.5.3.6 prenda	108
4.5.3.7 stock	108
4.5.3.8 talla	108
4.5.3.9 tipo	108
4.6 Referencia del archivo lib/business/usecases/login_usecase.dart	108
4.7 Referencia del archivo lib/business/usecases/nueva_order_usecase.dart	108
4.7.1 Documentación de enumeraciones	109
4.7.1.1 TipoConfirmacion	109
4.7.1.2 TipoItemOrden	109
4.7.2 Documentación de funciones	110
4.7.2.1 ItemOrden()	110
4.7.2.2 NuevaOrdenResult()	110
4.7.3 Documentación de variables	110
4.7.3.1 idInventario	110
4.7.3.2 idPedido	110
4.7.3.3 idUnidad	110
4.7.3.4 idVenta	111
4.7.3.5 precioUnitario	111
4.7.3.6 tieneUnidadResuelta	111
4.7.3.7 tipo	111
4.7.3.8 total	111
4.7.3.9 totalPrendas	111
4.8 Referencia del archivo lib/business/usecases/pedidos_pendientes_usecase.dart	111
4.9 Referencia del archivo lib/business/usecases/print_usecase.dart	112
4.10 Referencia del archivo lib/business/usecases/qr_usecase.dart	112
4.11 Referencia del archivo lib/business/usecases/regar_inventario_usecase.dart	112
4.11.1 Documentación de funciones	113
4.11.1.1 RegistrarInventarioResult()	113
4.11.2 Documentación de variables	113
4.11.2.1 algunoExitoso	113
4.11.2.2 resultados	113
4.11.2.3 todosExitosos	113
4.11.2.4 totalExitosos	113
4.11.2.5 totalFallidos	114
4.12 Referencia del archivo lib/business/usecases/restar_unidades_usecase.dart	114
4.12.1 Documentación de enumeraciones	114
4.12.1.1 ResultadoRestarUnidad	114
4.12.2 Documentación de funciones	115
4.12.2.1 ejecutar()	115

4.12.2.2 RestarUnidadesUseCase()	115
4.12.3 Documentación de variables	115
4.12.3.1 unidadRepository	115
4.13 Referencia del archivo lib/business/usecases/ventas_usecase.dart	115
4.14 Referencia del archivo lib/business/validators/validacion_inventario.dart	115
4.15 Referencia del archivo lib/data/database/database_helper.dart	116
4.16 Referencia del archivo lib/data/repositories/backup_repository.dart	116
4.17 Referencia del archivo lib/data/repositories/escuela_repository.dart	116
4.18 Referencia del archivo lib/data/repositories/inventario_repository.dart	116
4.19 Referencia del archivo lib/data/repositories/orden_repository.dart	117
4.20 Referencia del archivo lib/data/repositories/pedido_repository.dart	117
4.21 Referencia del archivo lib/data/repositories/prenda_repository.dart	117
4.22 Referencia del archivo lib/data/repositories/reporte_repository.dart	117
4.23 Referencia del archivo lib/data/repositories/talla_repository.dart	118
4.24 Referencia del archivo lib/data/repositories/unidad_repository.dart	118
4.25 Referencia del archivo lib/data/repositories/usuario_repository.dart	118
4.26 Referencia del archivo lib/data/repositories/venta_repository.dart	118
4.27 Referencia del archivo lib/main.dart	119
4.27.1 Documentación de funciones	119
4.27.1.1 main()	119
4.28 Referencia del archivo lib/models/models.dart	119
4.28.1 Documentación de enumeraciones	120
4.28.1.1 RolUsuario	120
4.28.2 Documentación de funciones	120
4.28.2.1 toDb()	120
4.28.3 Documentación de variables	120
4.28.3.1 admin	120
4.28.3.2 cancelada	120
4.28.3.3 cancelado	120
4.28.3.4 completada	120
4.28.3.5 completado	120
4.28.3.6 pendiente	121
4.28.3.7 vendedor	121
4.29 Referencia del archivo lib/presentation/providers/alertas_provider.dart	121
4.30 Referencia del archivo lib/presentation/providers/home_provider.dart	121
4.31 Referencia del archivo lib/presentation/providers/nueva_orden_provider.dart	121
4.32 Referencia del archivo lib/presentation/providers/pedidos_pendientes_provider.dart	122
4.33 Referencia del archivo lib/presentation/providers/print_provider.dart	122
4.34 Referencia del archivo lib/presentation/providers/qr_pendientes_provider.dart	122
4.35 Referencia del archivo lib/presentation/providers/qr_provider.dart	122
4.36 Referencia del archivo lib/presentation/providers/registrar_inventario_provider.dart	123
4.37 Referencia del archivo lib/presentation/providers/reportes_provider.dart	123

4.38 Referencia del archivo lib/presentation/providers/ventas_provider.dart	123
4.39 Referencia del archivo lib/presentation/screens/administrar_cantidad_screen.dart	123
4.39.1 Documentación de funciones	124
4.39.1.1 _confirmarAgregar()	124
4.39.1.2 _confirmarRestar()	124
4.39.1.3 build()	125
4.39.1.4 initState()	125
4.39.2 Documentación de variables	125
4.39.2.1 _controller	125
4.39.2.2 _restarUseCase	125
4.39.2.3 _unidadRepo	125
4.40 Referencia del archivo lib/presentation/screens/administrar_prenda_screen.dart	126
4.40.1 Documentación de funciones	126
4.40.1.1 _actualizarCantidadDesdeProvider()	126
4.40.1.2 _cargarDatos()	127
4.40.1.3 _dialogoEliminar()	127
4.40.1.4 _dialogoGuardar()	127
4.40.1.5 _eliminar()	127
4.40.1.6 _guardarCambios()	127
4.40.1.7 build()	127
4.40.1.8 initState()	128
4.40.2 Documentación de variables	128
4.40.2.1 _inventarioRepo	128
4.40.2.2 _precioController	128
4.40.2.3 cantidadActual	128
4.40.2.4 cargando	128
4.40.2.5 inventario	128
4.41 Referencia del archivo lib/presentation/screens/agregar_prendas_pedido_screen.dart	129
4.41.1 Documentación de funciones	129
4.41.1.1 _buildDropdown()	129
4.41.1.2 build()	130
4.41.2 Documentación de variables	130
4.41.2.1 _escuelas	130
4.41.2.2 _escuelaSeleccionada	130
4.41.2.3 _prendas	130
4.41.2.4 _prendasDisponibles	130
4.41.2.5 _prendaSeleccionada	130
4.42 Referencia del archivo lib/presentation/screens/bottom_nav_bar.dart	131
4.43 Referencia del archivo lib/presentation/screens/exito_screen.dart	131
4.43.1 Documentación de funciones	131
4.43.1.1 build()	131
4.43.1.2 createState()	131

4.43.1.3	ExitScreen()	132
4.43.1.4	initState()	132
4.44	Referencia del archivo lib/presentation/screens/home_screen.dart	132
4.44.1	Documentación de funciones	132
4.44.1.1	build()	132
4.44.1.2	HomeScreen()	133
4.44.1.3	HomeView()	133
4.45	Referencia del archivo lib/presentation/screens/inventario_screen.dart	133
4.45.1	Documentación de funciones	133
4.45.1.1	_ListaInventario()	133
4.45.1.2	build()	134
4.45.2	Documentación de variables	134
4.45.2.1	items	134
4.45.2.2	String	134
4.46	Referencia del archivo lib/presentation/screens/login_screen.dart	134
4.46.1	Documentación de funciones	135
4.46.1.1	_handleLogin()	135
4.46.1.2	build()	135
4.46.1.3	dispose()	135
4.46.1.4	initState()	135
4.46.2	Documentación de variables	135
4.46.2.1	_animController	135
4.46.2.2	_fadeAnim	136
4.46.2.3	_passwordController	136
4.46.2.4	_slideAnim	136
4.46.2.5	_usuarioController	136
4.46.2.6	_verPassword	136
4.47	Referencia del archivo lib/presentation/screens/nueva_orden_screen.dart	136
4.47.1	Documentación de funciones	137
4.47.1.1	_agruparItem()	137
4.47.1.2	_buildDropdown()	137
4.47.1.3	_buildInputDecoration()	138
4.47.1.4	dispose()	138
4.47.1.5	initState()	138
4.47.2	Documentación de variables	138
4.47.2.1	_clienteController	138
4.47.2.2	_escuelaSeleccionada	138
4.47.2.3	_prendaSeleccionada	138
4.47.2.4	_tallaSeleccionada	138
4.48	Referencia del archivo lib/presentation/screens/pedidos_pendientes_screen.dart	139
4.48.1	Documentación de funciones	139
4.48.1.1	_buildPedidoCard()	139

4.48.1.2 build()	139
4.48.1.3 createState()	139
4.48.1.4 initState()	140
4.48.1.5 PedidosPendientesScreen()	140
4.49 Referencia del archivo lib/presentation/screens/qr_pendientes_screen.dart	140
4.49.1 Documentación de funciones	140
4.49.1.1 _QrPendientesView()	140
4.49.1.2 build()	141
4.49.1.3 QrPendientesScreen()	141
4.50 Referencia del archivo lib/presentation/screens/qr_screen.dart	141
4.50.1 Documentación de enumeraciones	142
4.50.1.1 ResultadoScan	142
4.50.2 Documentación de funciones	142
4.50.2.1 _handleScan()	142
4.50.2.2 build()	143
4.50.2.3 dispose()	143
4.50.2.4 initState()	143
4.50.2.5 ScanFeedback()	143
4.50.3 Documentación de variables	143
4.50.3.1 _colorMensaje	143
4.50.3.2 _mensaje	143
4.50.3.3 _procesando	143
4.50.3.4 controller	144
4.50.3.5 mensaje	144
4.50.3.6 resultado	144
4.51 Referencia del archivo lib/presentation/screens/regarstrar_prenda_screen.dart	144
4.51.1 Documentación de funciones	145
4.51.1.1 _guardarCambios()	145
4.51.1.2 _inputDecoration()	145
4.51.1.3 _mostrarDialogoEscuela()	145
4.51.1.4 build()	146
4.51.1.5 dispose()	146
4.51.1.6 initState()	146
4.51.2 Documentación de variables	146
4.51.2.1 _cargando	146
4.51.2.2 _formKey	146
4.51.2.3 _idEscuela	146
4.51.2.4 _idPrendaSeleccionado	146
4.51.2.5 _idTallaSeleccionada	146
4.51.2.6 _precioController	147
4.52 Referencia del archivo lib/presentation/screens/reportes_screen.dart	147
4.52.1 Documentación de funciones	147

4.52.1.1 _EmptyState()	147
4.52.1.2 _RankingItem()	148
4.52.1.3 _ReportesView()	148
4.52.1.4 _SectionTitle()	148
4.52.1.5 build()	148
4.52.1.6 ReportesScreen()	148
4.52.2 Documentación de variables	148
4.52.2.1 mensaje	148
4.52.2.2 monto	148
4.52.2.3 nombre	149
4.52.2.4 proporcion	149
4.52.2.5 subtítulo	149
4.52.2.6 título	149
4.53 Referencia del archivo lib/presentation/screens/revisar_pedido_screen.dart	149
4.53.1 Documentación de funciones	150
4.53.1.1 build()	150
4.53.1.2 createState()	150
4.53.1.3 initState()	150
4.53.1.4 RevisarPedidoScreen()	150
4.53.2 Documentación de variables	150
4.53.2.1 idPedido	150
4.54 Referencia del archivo lib/presentation/screens/revisar_venta_screen.dart	150
4.54.1 Documentación de funciones	151
4.54.1.1 _devolverPrenda()	151
4.54.1.2 build()	151
4.54.1.3 initState()	151
4.54.2 Documentación de variables	152
4.54.2.1 _unidadRepository	152
4.54.2.2 _ventaRepository	152
4.55 Referencia del archivo lib/presentation/screens/ventas_screen.dart	152
4.55.1 Documentación de funciones	153
4.55.1.1 _nombreMes()	153
4.55.1.2 build()	153
4.55.1.3 initState()	153
4.55.2 Documentación de variables	153
4.55.2.1 _anioSeleccionado	153
4.55.2.2 _mesSeleccionado	153
4.56 Referencia del archivo lib/presentation/widgets/alertas_popup.dart	153
4.56.1 Documentación de funciones	154
4.56.1.1 _ItemAlerta()	154
4.56.1.2 build()	154
4.56.2 Documentación de variables	154

4.56.2.1 color	154
4.56.2.2 stock	154
4.56.2.3 subtitulo	154
4.56.2.4 titulo	154

Capítulo 1

Índice de clases

1.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

AlertasStockResult	
Agrupar el resultado de la consulta de alertas de stock, separando los items agotados de los que están en nivel crítico	5
BackupInfo	
Modelo que representa la información de un archivo de backup generado	6
BackupRepository	
Repositorio encargado de gestionar la creación física de archivos de backup de la base de datos SQLite en el almacenamiento local del dispositivo	8
BackupService	
Servicio que gestiona las operaciones de backup de la base de datos	9
ChangeNotifier	
Proveedor de estado para la autenticación de usuarios	12
CustomPainter	
Overlay visual del scanner que dibuja:	40
DatabaseHelper	
Helper que gestiona el ciclo de vida de la base de datos SQLite local	41
EscuelaRepository	
Repositorio que gestiona las operaciones de acceso a datos para la tabla de escuelas	43
InventarioRepository	
Repositorio que gestiona las operaciones de acceso a datos para la tabla de inventario y sus consultas relacionadas con stock	45
LoginUseCase	
Caso de uso para el inicio de sesión	48
NuevaOrdenUseCase	
Caso de uso que gestiona la creación de órdenes mixtas, combinando items de venta directa (con unidad física) y pedidos (sin unidad asignada)	49
OrdenRepository	
Repositorio que gestiona las operaciones de acceso a datos para la tabla de órdenes	53
PedidoPendienteUseCase	
Caso de uso que gestiona el ciclo de vida de los pedidos en estado pendiente	54
PedidoRepository	
Repositorio encargado de gestionar las operaciones relacionadas con pedidos y detalles de pedido en la base de datos	57
PrendaRepository	
Repositorio encargado de gestionar las operaciones CRUD relacionadas con la entidad Prenda	61

PrintService	
Contrato base para servicios de impresión	63
PrintUseCase	
Caso de uso que gestiona la impresión de etiquetas QR para unidades de inventario	64
QrUseCase	
Caso de uso que gestiona la interpretación y validación de códigos QR para la identificación de unidades físicas de inventario	65
RegistrarInventarioUseCase	
Caso de uso que gestiona el registro de items de inventario para una combinación de prenda, escuela y una o más tallas	67
RegistroTallaResult	
Resultado del intento de registro de inventario para una talla específica	68
ReporteRepository	
Repositorio encargado de generar consultas estadísticas y métricas utilizadas en reportes	69
StatefulWidget	
Pantalla para modificar la cantidad de unidades de un inventario específico	71
StatelessWidget	
Widget raíz de la aplicación	78
TallaRepository	
Repositorio encargado de gestionar las operaciones CRUD relacionadas con la entidad Talla	86
UnidadRepository	
Repositorio encargado de gestionar las operaciones relacionadas con unidades de inventario	88
UsuarioRepository	
Repositorio encargado de gestionar las operaciones CRUD relacionadas con usuarios del sistema	91
ValidacionInventario	
Clase utilitaria que centraliza las validaciones de negocio aplicables a los registros de inventario	93
VentaRepository	
Repositorio encargado de gestionar operaciones relacionadas con ventas y detalles de venta	95
VentasUseCase	
Caso de uso que expone las operaciones de consulta sobre el historial de ventas	98

Capítulo 2

Índice de archivos

2.1. Lista de archivos

Lista de todos los archivos con breves descripciones:

lib/main.dart	119
lib/business/providers/auth_provider.dart	101
lib/business/providers/inventario_provider.dart	101
lib/business/services/backup_service.dart	101
lib/business/services/print_service.dart	102
lib/business/usecases/alertas_stock_usecase.dart	105
lib/business/usecases/login_usecase.dart	108
lib/business/usecases/nueva_order_usecase.dart	108
lib/business/usecases/pedidos_pendientes_usecase.dart	111
lib/business/usecases/print_usecase.dart	112
lib/business/usecases/qr_usecase.dart	112
lib/business/usecases/registro_inventario_usecase.dart	112
lib/business/usecases/restar_unidades_usecase.dart	114
lib/business/usecases/ventas_usecase.dart	115
lib/business/validators/validacion_inventario.dart	115
lib/data/database/database_helper.dart	116
lib/data/repositories/backup_repository.dart	116
lib/data/repositories/escuela_repository.dart	116
lib/data/repositories/inventario_repository.dart	116
lib/data/repositories/order_repository.dart	117
lib/data/repositories/pedido_repository.dart	117
lib/data/repositories/prenda_repository.dart	117
lib/data/repositories/reporte_repository.dart	117
lib/data/repositories/talla_repository.dart	118
lib/data/repositories/unidad_repository.dart	118
lib/data/repositories/usuario_repository.dart	118
lib/data/repositories/venta_repository.dart	118
lib/models/models.dart	119
lib/presentation/providers/alertas_provider.dart	121
lib/presentation/providers/home_provider.dart	121
lib/presentation/providers/nueva_order_provider.dart	121
lib/presentation/providers/pedidos_pendientes_provider.dart	122
lib/presentation/providers/print_provider.dart	122
lib/presentation/providers/qr_pendientes_provider.dart	122
lib/presentation/providers/qr_provider.dart	122

lib/presentation/providers/registrar_inventario_provider.dart	123
lib/presentation/providers/reportes_provider.dart	123
lib/presentation/providers/ventas_provider.dart	123
lib/presentation/screens/administrar_cantidad_screen.dart	123
lib/presentation/screens/administrar_prenda_screen.dart	126
lib/presentation/screens/agregar_prendas_pedido_screen.dart	129
lib/presentation/screens/bottom_nav_bar.dart	131
lib/presentation/screens/exito_screen.dart	131
lib/presentation/screens/home_screen.dart	132
lib/presentation/screens/inventario_screen.dart	133
lib/presentation/screens/login_screen.dart	134
lib/presentation/screens/nueva_order_screen.dart	136
lib/presentation/screens/pedidos_pendientes_screen.dart	139
lib/presentation/screens/qr_pendientes_screen.dart	140
lib/presentation/screens/qr_screen.dart	141
lib/presentation/screens/registrar_prenda_screen.dart	144
lib/presentation/screens/reportes_screen.dart	147
lib/presentation/screens/revisar_pedido_screen.dart	149
lib/presentation/screens/revisar_venta_screen.dart	150
lib/presentation/screens/ventas_screen.dart	152
lib/presentation/widgets/alertas_popup.dart	153

Capítulo 3

Documentación de clases

3.1. Referencia de la clase `AlertasStockResult`

Agrupar el resultado de la consulta de alertas de stock, separando los items agotados de los que están en nivel crítico.

Métodos privados

- `const AlertasStockResult ({required this.agotados, required this.criticos})`

Atributos privados

- `final List< AlertaStock > agotados`
- `final List< AlertaStock > criticos`
- `List< AlertaStock > get todas => [...agotados`
Retorna la lista combinada de todas las alertas, con los agotados primero.
- `List< AlertaStock > get criticos`
- `bool get hayAlertas => agotados.isNotEmpty || criticos.isNotEmpty`
Retorna true si existe al menos una alerta activa entre agotados y críticos.
- `int get totalAgotados => agotados.length`
- `int get totalCriticos => criticos.length`

3.1.1. Descripción detallada

Agrupar el resultado de la consulta de alertas de stock, separando los items agotados de los que están en nivel crítico.

3.1.2. Documentación de constructores y destructores

3.1.2.1. `AlertasStockResult()`

```
const AlertasStockResult::AlertasStockResult (  
    {required this.agotados, required this.criticos} ) [private]
```

3.1.3. Documentación de datos miembro

3.1.3.1. agotados

```
final List<AlertaStock> AlertasStockResult::agotados [private]
```

3.1.3.2. criticos [1/2]

```
final List<AlertaStock> AlertasStockResult::criticos [private]
```

3.1.3.3. criticos [2/2]

```
List<AlertaStock> get AlertasStockResult::criticos [private]
```

3.1.3.4. hayAlertas

```
bool get AlertasStockResult::hayAlertas => agotados.isNotEmpty || criticos.isNotEmpty [private]
```

Retorna true si existe al menos una alerta activa entre agotados y críticos.

3.1.3.5. todas

```
List<AlertaStock> get AlertasStockResult::todas => [...agotados [private]
```

Retorna la lista combinada de todas las alertas, con los agotados primero.

3.1.3.6. totalAgotados

```
int get AlertasStockResult::totalAgotados => agotados.length [private]
```

3.1.3.7. totalCriticos

```
int get AlertasStockResult::totalCriticos => criticos.length [private]
```

La documentación de esta clase está generada del siguiente archivo:

- [lib/business/usecases/alertas_stock_usecase.dart](#)

3.2. Referencia de la clase BackupInfo

Modelo que representa la información de un archivo de backup generado.

Métodos privados

- const BackupInfo ({ required this.ruta, required this.nombre, required this.fechaCreacion, required this.↵ tamanoBytes, })

Atributos privados

- final String ruta
Ruta absoluta del archivo de backup en el sistema de archivos.
- final String nombre
Nombre del archivo de backup.
- final DateTime fechaCreacion
Fecha y hora en que fue creado el archivo de backup.
- final int tamanoBytes
Tamaño del archivo de backup en bytes.
- String get tamanoLegible
Retorna el tamaño del archivo en formato legible (B, KB o MB) según el rango en que se encuentre el valor en bytes.

3.2.1. Descripción detallada

Modelo que representa la información de un archivo de backup generado.

3.2.2. Documentación de constructores y destructores

3.2.2.1. BackupInfo()

```
const BackupInfo::BackupInfo (  
    { required this.ruta, required this.nombre, required this.fechaCreacion, required  
    this.tamanoBytes, } ) [private]
```

3.2.3. Documentación de datos miembro

3.2.3.1. fechaCreacion

```
final DateTime BackupInfo::fechaCreacion [private]
```

Fecha y hora en que fue creado el archivo de backup.

3.2.3.2. nombre

```
final String BackupInfo::nombre [private]
```

Nombre del archivo de backup.

3.2.3.3. ruta

```
final String BackupInfo::ruta [private]
```

Ruta absoluta del archivo de backup en el sistema de archivos.

3.2.3.4. tamanoBytes

```
final int BackupInfo::tamanoBytes [private]
```

Tamaño del archivo de backup en bytes.

3.2.3.5. tamanoLegible

```
String get BackupInfo::tamanoLegible [private]
```

Valor inicial:

```
{
  if (tamanoBytes < 1024) {
    return '$tamanoBytes B';
  }

  if (tamanoBytes < 1024 * 1024) {
    return '${(tamanoBytes / 1024).toStringAsFixed(1)} KB';
  }

  return '${(tamanoBytes / (1024 * 1024)).toStringAsFixed(1)} MB'
}
```

Retorna el tamaño del archivo en formato legible (B, KB o MB) según el rango en que se encuentre el valor en bytes.

La documentación de esta clase está generada del siguiente archivo:

- lib/business/services/[backup_service.dart](#)

3.3. Referencia de la clase BackupRepository

Repositorio encargado de gestionar la creación física de archivos de backup de la base de datos SQLite en el almacenamiento local del dispositivo.

Métodos privados

- Future< String > [crearBackup](#) () async
Genera una copia del archivo de base de datos activo en el directorio de backups.
- Future< Directory > [_obtenerDirectorioBackups](#) () async
Retorna el directorio donde se almacenan los backups dentro del directorio de documentos de la aplicación.

3.3.1. Descripción detallada

Repositorio encargado de gestionar la creación física de archivos de backup de la base de datos SQLite en el almacenamiento local del dispositivo.

3.3.2. Documentación de funciones miembro

3.3.2.1. _obtenerDirectorioBackups()

```
Future< Directory > BackupRepository::_obtenerDirectorioBackups () [inline], [private]
```

Retorna el directorio donde se almacenan los backups dentro del directorio de documentos de la aplicación.

Si el directorio no existe, lo crea de forma recursiva.

3.3.2.2. crearBackup()

```
Future< String > BackupRepository::crearBackup () [inline], [private]
```

Genera una copia del archivo de base de datos activo en el directorio de backups.

El nombre del archivo incluye la fecha y hora de creación con formato 'backup_uniformes_YYYY_MM_DD_HH_mm.db' para facilitar su identificación. Retorna la ruta absoluta del archivo de backup generado.

La documentación de esta clase está generada del siguiente archivo:

- lib/data/repositories/[backup_repository.dart](#)

3.4. Referencia de la clase BackupService

Servicio que gestiona las operaciones de backup de la base de datos.

Métodos privados

- [BackupService](#)({required [BackupRepository](#) backupRepository}) Future< [BackupResult](#) > [crearBackup](#) () async
Crea un nuevo archivo de backup de la base de datos.
- Future< [BackupResult](#) > [crearYCompartirBackup](#) () async
Crea un nuevo backup y lo comparte inmediatamente a través del sistema de compartición nativo del dispositivo.
- Future< List< [BackupInfo](#) > > [listarBackups](#) () async
Obtiene la lista de backups disponibles en el directorio de backups, ordenados de más reciente a más antiguo.
- Future< [BackupResult](#) > [compartirBackup](#) ([BackupInfo](#) backup) async
Comparte un archivo de backup existente mediante el sistema nativo del dispositivo.
- Future< [BackupResult](#) > [eliminarBackup](#) ([BackupInfo](#) backup) async
Elimina el archivo de backup del sistema de archivos.
- Future< void > [_limpiarBackupsAntiguos](#) () async
Elimina los backups más antiguos cuando se supera el límite de [_maxBackups].
- Future< Directory > [_directorioBackups](#) () async
Retorna el directorio donde se almacenan los backups.

Atributos privados

- final [BackupRepository](#) [_backupRepository](#)
Repositorio que ejecuta la creación física del archivo de backup.

Atributos estáticos privados

- static const int `_maxBackups` = 5

Número máximo de backups que se conservan en el directorio.

3.4.1. Descripción detallada

Servicio que gestiona las operaciones de backup de la base de datos.

Permite crear, listar, compartir y eliminar backups, además de mantener un límite máximo de archivos almacenados.

3.4.2. Documentación de funciones miembro

3.4.2.1. `_directorioBackups()`

```
Future< Directory > BackupService::_directorioBackups () [inline], [private]
```

Retorna el directorio donde se almacenan los backups.

Si el directorio no existe, lo crea de forma recursiva.

3.4.2.2. `_limpiarBackupsAntiguos()`

```
Future< void > BackupService::_limpiarBackupsAntiguos () [inline], [private]
```

Elimina los backups más antiguos cuando se supera el límite de `_maxBackups`.

La lista ya viene ordenada de más reciente a más antiguo, por lo que se eliminan los elementos al final de la lista.

3.4.2.3. `compartirBackup()`

```
Future< BackupResult > BackupService::compartirBackup (  
    BackupInfo backup) [inline], [private]
```

Comparte un archivo de backup existente mediante el sistema nativo del dispositivo.

Verifica que el archivo exista en el sistema de archivos antes de intentar compartirlo.

3.4.2.4. `crearBackup()`

```
BackupService({required BackupRepository backupRepository}) Future< BackupResult > BackupService::crearBackup () [inline], [private]
```

Crea un nuevo archivo de backup de la base de datos.

Una vez creado, ejecuta la limpieza de backups antiguos para mantener el límite definido por `_maxBackups`. Retorna un `[BackupResult]` con el resultado de la operación.

3.4.2.5. crearYCompartirBackup()

```
Future< BackupResult > BackupService::crearYCompartirBackup () [inline], [private]
```

Crea un nuevo backup y lo comparte inmediatamente a través del sistema de compartición nativo del dispositivo.

Si la creación falla, retorna el resultado sin intentar compartir. Si el backup se crea pero no se puede compartir, retorna el error manteniendo la referencia al backup generado.

3.4.2.6. eliminarBackup()

```
Future< BackupResult > BackupService::eliminarBackup (  
    BackupInfo backup) [inline], [private]
```

Elimina el archivo de backup del sistema de archivos.

Si el archivo no existe, la operación se considera exitosa para mantener consistencia con el estado esperado.

3.4.2.7. listarBackups()

```
Future< List< BackupInfo > > BackupService::listarBackups () [inline], [private]
```

Obtiene la lista de backups disponibles en el directorio de backups, ordenados de más reciente a más antiguo.

Filtra únicamente archivos con el prefijo 'backup_uniformes_' y extensión '.db'. Retorna una lista vacía si el directorio no existe o si ocurre un error. Los backups se ordenan descendientemente por fecha de creación para que el más reciente aparezca primero en la lista.

3.4.3. Documentación de datos miembro

3.4.3.1. _backupRepository

```
final BackupRepository BackupService::_backupRepository [private]
```

Repositorio que ejecuta la creación física del archivo de backup.

3.4.3.2. _maxBackups

```
const int BackupService::_maxBackups = 5 [static], [private]
```

Número máximo de backups que se conservan en el directorio.

Los backups más antiguos se eliminan automáticamente al superar este límite.

La documentación de esta clase está generada del siguiente archivo:

- [lib/business/services/backup_service.dart](#)

3.5. Referencia de la clase ChangeNotifier

Proveedor de estado para la autenticación de usuarios.

Métodos privados

- `Future< bool > login (String usuario, String password) async`
Ejecuta el flujo de autenticación con las credenciales proporcionadas.
- `void logout ()`
Cierra la sesión del usuario actual limpiando el estado de autenticación.
- `void limpiarError ()`
Limpia el mensaje de error actual y notifica a los listeners.
- `InventarioProvider ()`
Constructor que dispara la carga inicial de catálogos al instanciar el proveedor.
- `Future< void > _inicializarDatos () async`
Carga los catálogos de escuelas y prendas necesarios para el funcionamiento del módulo.
- `Future< void > cargarInventario (int idEscuela) async`
Carga el inventario filtrado por la escuela indicada y, opcionalmente, por la prenda actualmente seleccionada.
- `Future< void > seleccionarPrenda (int? idPrenda) async`
Actualiza el filtro de prenda seleccionada y recarga el inventario si hay una escuela activa.
- `Future< void > recargarEscuelas () async`
Recarga el catálogo de escuelas desde el repositorio y, si hay una escuela seleccionada, recarga también el inventario activo para reflejar cualquier cambio reciente en los datos.
- `Future< void > cargarAlertas () async`
Ejecuta [AlertasStockUseCase.obtenerAlertas] y actualiza el estado local.
- `Future< void > refrescar () async`
Actualiza las alertas desde el caso de uso sin activar el indicador de carga.
- `void limpiar ()`
Reinicia las listas de alertas a vacío y notifica a los listeners.
- `void setUsername (String name)`
Actualiza el nombre del usuario y notifica a los listeners.
- `void navigateToInventory (BuildContext context)`
Navega a la pantalla de inventario.
- `void navigateToOrders (BuildContext context)`
Navega a la pantalla de órdenes.
- `void navigateToNewOrder (BuildContext context)`
Navega a la pantalla de nueva orden.
- `Future< void > cargarInventario () async`
Obtiene el inventario completo desde [InventarioRepository] y notifica a los listeners.
- `Future< void > agregarQr (String qr) async`
Agrega un ítem a la orden mediante escaneo de QR, correspondiente a una venta.
- `Future< void > agregarManual ({required int idInventario}) async`
Agrega un ítem a la orden mediante selección manual de inventario, correspondiente a un pedido.
- `void eliminarItem (ItemOrden item)`
Elimina un [ItemOrden] de la lista de ítems de la orden actual.
- `void limpiar ()`
Vacía la lista de ítems de la orden sin persistir cambios.
- `Future< void > confirmar ({ required int idUsuario, required String? nombreCliente, }) async`
Confirma y persiste la orden actual ejecutando [NuevaOrdenUseCase.confirmarMixto].
- `Future< void > cargarPedidos () async`

- Carga la lista de pedidos pendientes desde [PedidoPendienteUseCase].*

 - Future< void > **cargarDetalles** (int idPedido) async

Carga los detalles del pedido identificado por [idPedido].
- Future< Map< String, dynamic >? > **registrarQrPedido** ({ required int idPedido, required int idDetallePedido, required int idInventarioEsperado, required String qr, bool forzarMovimiento=false, }) async

Registra un QR escaneado contra un detalle de pedido específico.
- Future< void > **desregistrarUnidad** ({ required int idPedido, required int idDetallePedido, }) async

Desregistra la unidad asociada a un detalle de pedido, revirtiendo su registro.
- Future< bool > **eliminarDetallePedido** ({ required int idPedido, required int idDetallePedido, }) async

Elimina un detalle de pedido y recarga el estado del provider.
- Future< bool > **pedidoCompleto** (int idPedido) async

Verifica si todos los detalles del pedido identificado por [idPedido] han sido registrados con QR.
- Future< void > **completarPedido** (int idPedido) async

Marca el pedido como completado y recarga la lista de pedidos pendientes.
- **PrintProvider** (this.useCase, this.alertasProvider)

Crea una instancia de [PrintProvider] con el [useCase] y [alertasProvider] requeridos.
- Future< bool > **solicitarPermisosBluetooth** () async

Solicita los permisos de Bluetooth y ubicación requeridos para la impresión.
- Future< void > **agregarUnidades** (int idInventario, int cantidad) async

Agrega unidades al inventario e imprime los QRs correspondientes.
- Future< void > **imprimirQrExistente** (int idUnidad) async

Reimprime el QR de una unidad ya existente sin crear un nuevo registro.
- void **limpiarMensaje** ()

Reinicia [mensaje] a vacío y notifica a los listeners.
- Future< void > **cargarPendientes** () async

Obtiene desde [UnidadRepository] las unidades con QR pendiente de impresión.
- **QrProvider** (this.useCase)

Crea una instancia de [QrProvider] con el [useCase] requerido.
- Future< void > **procesarQr** (String qr) async

Procesa un código QR escaneado y agrega la unidad correspondiente a la lista.
- void **eliminarUnidad** (int unidadId)

Elimina la unidad identificada por [unidadId] de la lista de unidades procesadas.
- void **limpiar** ()

Reinicia el estado completo del provider, limpiando unidades y errores.
- **RegistrarInventarioProvider** ({ required this.prendaRepository, required this.tallaRepository, required this.↵ escuelaRepository, })

Crea una instancia de [RegistrarInventarioProvider] con los repositorios requeridos.
- Future< void > **cargarCatalogos** () async

Carga los catálogos de prendas, tallas y escuelas desde sus repositorios.
- Future< Escuela? > **agregarEscuela** (String nombre) async

Inserta una nueva escuela en el repositorio y actualiza la lista local.
- Future< void > **inicializar** () async

Inicializa el provider cargando el catálogo de escuelas y los reportes iniciales.
- Future< void > **cargarReportes** () async

Carga todos los indicadores de reporte aplicando los filtros activos.
- Future< void > **cambiarYear** (int nuevo) async

Actualiza el filtro de año y recarga los reportes.
- Future< void > **cambiarMonth** (int? nuevo) async

Actualiza el filtro de mes y recarga los reportes.
- Future< void > **cambiarEscuela** (int? idEscuela) async

Actualiza el filtro de escuela y recarga los reportes.

- `return escuelas firstWhere ((e)=> e.idEscuela==idEscuelaSeleccionada, orElse:())=> Escuela(idEscuela:0, nombre:'Todas las escuelas'),) .nombre`
- `return (escuelasMasVentas.first['total'] as num).toDouble()`
- `return (prendasMasVendidas.first['cantidad'] as num).toDouble()`
- `Future< void > cargarVentas () async`
Carga el historial completo de ventas desde [VentasUseCase].
- `Future< void > cargarDetalles (int idVenta) async`
Carga los detalles de la venta identificada por [idVenta].
- `Future< void > refrescarAlertas () async`
Refresca las alertas de stock a través de [AlertasProvider].

Atributos privados

- `final _usuarioRepo = UsuarioRepository()`
Repositorio encargado de obtener los datos de usuarios.
- `final _loginUseCase = LoginUseCase()`
Caso de uso que ejecuta la lógica de autenticación.
- `bool _cargando = false`
Indica si hay una operación asíncrona en curso.
- `String _error`
Mensaje de error producido durante el último intento de autenticación. Null si no hay error.
- `Usuario _usuarioActual`
Usuario autenticado actualmente. Null si no hay sesión activa.
- `bool get cargando => _cargando`
Expone el estado de carga a los widgets consumidores.
- `String get error => _error`
Expone el mensaje de error a los widgets consumidores.
- `Usuario get usuarioActual => _usuarioActual`
- `bool get estaAutenticado => _usuarioActual != null`
Retorna true si existe un usuario autenticado en sesión.
- `final _inventarioRepo = InventarioRepository()`
Repositorio encargado de obtener y filtrar los registros de inventario.
- `final _escuelaRepo = EscuelaRepository()`
Repositorio encargado de obtener el catálogo de escuelas.
- `final _prendaRepo = PrendaRepository()`
Repositorio encargado de obtener el catálogo de prendas.
- `List< Escuela > _escuelas = []`
Lista de escuelas disponibles cargadas al inicializar el proveedor.
- `List< Prenda > _prendas = []`
Lista de prendas disponibles cargadas al inicializar el proveedor.
- `Escuela _escuelaSeleccionada`
Escuela seleccionada actualmente para filtrar el inventario.
- `int _idPrendaSeleccionada`
Identificador de la prenda seleccionada como filtro adicional. Null si no se aplica filtro por prenda.
- `List< Map< String, dynamic > > _itemsInventario = []`
Lista de items del inventario resultante del filtrado aplicado.
- `List< Escuela > get escuelas => _escuelas`
Lista de escuelas disponibles para el filtro.
- `List< Prenda > get prendas => _prendas`
- `Escuela get escuelaSeleccionada => _escuelaSeleccionada`
- `int get idPrendaSeleccionada => _idPrendaSeleccionada`

- List< Map< String, dynamic > > get itemsInventario => _itemsInventario
- final AlertasStockUseCase _useCase
 - Caso de uso encargado de obtener las alertas de stock.*
- AlertasProvider({required AlertasStockUseCase useCase}) bool _cargando = false
 - Crea una instancia de [AlertasProvider] con el [useCase] requerido.*
- List< AlertaStock > _agotados = []
 - Lista de inventarios con stock en cero.*
- List< AlertaStock > _criticos = []
 - Lista de inventarios con stock por debajo del umbral crítico.*
- List< AlertaStock > get agotados => _agotados
 - Expone los registros de inventario agotados.*
- List< AlertaStock > get criticos => _criticos
 - Expone los registros de inventario en estado crítico.*
- bool get hayAlertas => _agotados.isNotEmpty || _criticos.isNotEmpty
 - Retorna true si existe al menos una alerta activa de cualquier tipo.*
- int get totalAlertas => _agotados.length + _criticos.length
 - Retorna el total de alertas activas sumando agotados y críticos.*
- List< AlertaStock > get todas => [..._agotados]
 - Retorna la lista unificada de alertas, con agotados antes que críticos.*
- List< AlertaStock > get _criticos
- String _userName = 'Usuario'
 - Nombre del usuario actualmente autenticado.*
- String get userName => _userName
 - Expone el nombre del usuario a los widgets consumidores.*
- final NuevaOrdenUseCase _useCase
 - Caso de uso que encapsula la lógica de negocio para crear órdenes.*
- final InventarioRepository _inventarioRepository
 - Repositorio para obtener el inventario disponible.*
- final AlertasProvider alertasProvider
 - Referencia al provider de alertas para refrescarlas al confirmar una orden.*
- NuevaOrdenProvider({ required InventarioRepository inventarioRepository, required UnidadRepository unidadRepository, required VentaRepository ventaRepository, required PedidoRepository pedidoRepository, required OrdenRepository ordenRepository, required QrUseCase qrUseCase, required this.alertasProvider, }) final List< ItemOrden > _items = []
 - Crea una instancia de [NuevaOrdenProvider] inyectando todos los repositorios y casos de uso necesarios para el flujo de nueva orden.*
- List< ItemOrden > get items => _items
 - Expone los items de la orden actual como lista de solo lectura.*
- List< Map< String, dynamic > > _inventario = []
 - Lista del inventario completo disponible para selección manual.*
- List< Map< String, dynamic > > get inventario => _inventario
 - Expone el inventario disponible a los widgets consumidores.*
- String error
 - Mensaje de error del último fallo ocurrido. Null si no hay error.*
- double get total
 - Calcula el total monetario de la orden sumando el precio unitario de cada item.*
- int get totalPrendas => _items.length
 - Retorna el número total de prendas agregadas a la orden.*
- final PedidoPendienteUseCase _useCase
 - Caso de uso que encapsula la lógica de negocio de pedidos pendientes.*

- PedidosPendientesProvider({ required [PedidoRepository](#) pedidoRepository, required [UnidadRepository](#) unidadRepository, required [VentaRepository](#) ventaRepository, required [QrUseCase](#) qrUseCase, }) List< Pedido > [_pedidos](#) = []
Crea una instancia de [PedidosPendientesProvider] inyectando los repositorios y el caso de uso de QR necesarios para el flujo de pedidos.
- List< Pedido > get [pedidos](#) => [_pedidos](#)
Expone la lista de pedidos pendientes a los widgets consumidores.
- List< Map< [String](#), dynamic > > [_detalles](#) = []
Lista de detalles del pedido actualmente seleccionado.
- List< Map< [String](#), dynamic > > get [detalles](#) => [_detalles](#)
Expone los detalles del pedido seleccionado a los widgets consumidores.
- final [PrintUseCase](#) useCase
Caso de uso que encapsula la lógica de impresión de QRs.
- bool [loading](#) = false
Indica si hay una operación de impresión en curso.
- [String](#) [mensaje](#) = ""
Mensaje de resultado de la última operación ejecutada.
- final [UnidadRepository](#) [_repository](#)
Repositorio de acceso a datos de unidades.
- QrPendientesProvider({required [UnidadRepository](#) repository}) List< Map< [String](#), dynamic > > [_pendientes](#) = []
Crea una instancia de [QrPendientesProvider] con el [repository] requerido.
- List< Map< [String](#), dynamic > > get [pendientes](#) => [_pendientes](#)
Expone la lista de unidades pendientes a los widgets consumidores.
- final [QrUseCase](#) useCase
Caso de uso encargado de resolver un código QR a una [Unidad].
- final List< Unidad > [_unidades](#) = []
Lista interna de unidades procesadas en la sesión actual.
- bool [_procesando](#) = false
Indica si hay un procesamiento de QR en curso.
- List< Unidad > get [unidades](#) => List.unmodifiable([_unidades](#))
Expone la lista de unidades como vista de solo lectura.
- bool get [procesando](#) => [_procesando](#)
Expone el estado de procesamiento a los widgets consumidores.
- final [PrendaRepository](#) [prendaRepository](#)
Repositorio de acceso a datos de prendas.
- final [TallaRepository](#) [tallaRepository](#)
Repositorio de acceso a datos de tallas.
- final [EscuelaRepository](#) [escuelaRepository](#)
Repositorio de acceso a datos de escuelas.
- List< Prenda > [prendas](#) = []
Lista de prendas disponibles en el catálogo.
- List< Talla > [tallas](#) = []
Lista de tallas disponibles en el catálogo.
- List< Escuela > [escuelas](#) = []
Lista de escuelas disponibles en el catálogo.
- bool [cargando](#) = false
Indica si hay una operación de carga en curso.
- final [ReporteRepository](#) [_repository](#)
Repositorio de acceso a datos de reportes de ventas.
- final [EscuelaRepository](#) [_escuelaRepository](#)

- Repositorio de acceso a datos de escuelas para poblar el filtro.*
- ReportesProvider({ required [ReporteRepository](#) repository, required [EscuelaRepository](#) escuelaRepository, }) bool [cargando](#) = false
 - Crea una instancia de [ReportesProvider] con los repositorios requeridos.*
- int [year](#) = DateTime.now().year
 - Año actualmente seleccionado como filtro. Por defecto el año en curso.*
- int [month](#)
 - Mes actualmente seleccionado como filtro. Null indica todos los meses.*
- int [idEscuelaSeleccionada](#)
 - ID de la escuela seleccionada como filtro. Null indica todas las escuelas.*
- double [totalVendido](#) = 0
 - Total monetario de ventas según los filtros activos.*
- int [cantidadVentas](#) = 0
 - Cantidad de ventas realizadas según los filtros activos.*
- List< Map< [String](#), dynamic > > [prendasMasVendidas](#) = []
 - Ranking de prendas más vendidas según los filtros activos.*
- List< Map< [String](#), dynamic > > [escuelasMasVentas](#) = []
 - Ranking de escuelas con más ventas.*
- [String](#) get [nombreMesSeleccionado](#)
 - Retorna el nombre del mes seleccionado, o 'Todo el año' si no hay mes activo.*
- const [meses](#)
- [return](#) [meses](#) [month!]
- [String](#) get [nombreEscuelaSeleccionada](#)
 - Retorna el nombre de la escuela seleccionada, o 'Todas las escuelas' si no hay filtro activo.*
- double get [maxTotalEscuela](#)
 - Retorna el total de ventas de la escuela con mayor venta para normalizar barras.*
- double get [maxCantidadPrenda](#)
 - Retorna la cantidad de la prenda más vendida para normalizar barras.*
- final [VentasUseCase](#) [_useCase](#)
 - Caso de uso que encapsula la lógica de negocio de ventas.*
- VentasProvider({ required [VentaRepository](#) ventaRepository, required this.alertasProvider, }) List< Venta > [_ventas](#) = []
 - Crea una instancia de [VentasProvider] inyectando el repositorio y el provider de alertas requeridos.*
- List< Venta > get [ventas](#) => [_ventas](#)
 - Expone la lista de ventas a los widgets consumidores.*

3.5.1. Descripción detallada

Proveedor de estado para la autenticación de usuarios.

Provider de estado para la gestión del historial de ventas.

Provider de estado para la pantalla de reportes de ventas.

Provider de estado para el flujo de registro de inventario.

Provider de estado para la gestión de unidades obtenidas mediante escaneo de QR.

Provider de estado para la gestión de unidades con QR pendiente de impresión.

Provider de estado para la gestión de impresión de QRs via Bluetooth.

Provider de estado para la gestión de pedidos pendientes.

Provider de estado para el flujo de creación de una nueva orden.

Provider de estado para la pantalla principal de la aplicación.

Provider de estado para la gestión de alertas de stock del inventario.

Proveedor de estado para el módulo de inventario.

Gestiona la carga y filtrado de inventario por escuela y prenda, así como el estado de selección activa del usuario. Extiende [\[ChangeNotifier\]](#) para notificar cambios a los widgets suscritos.

Consume [\[AlertasStockUseCase\]](#) para obtener los registros de inventario en estado agotado o crítico, y notifica a los widgets suscritos ante cualquier cambio de estado.

Gestiona el nombre del usuario autenticado y centraliza la navegación hacia las secciones principales del sistema.

Coordina la interacción entre [\[NuevaOrdenUseCase\]](#), [\[InventarioRepository\]](#) y [\[AlertasProvider\]](#) para gestionar los items de la orden, ya sea por escaneo de QR (venta) o por selección manual (pedido).

Coordina [\[PedidoPendienteUseCase\]](#) para cargar, registrar, desregistrar y completar pedidos, notificando a los widgets suscritos ante cada cambio.

Coordina [\[PrintUseCase\]](#) con la solicitud de permisos de Bluetooth y la actualización de [\[AlertasProvider\]](#) tras cada operación de impresión.

Consume [\[UnidadRepository\]](#) para obtener las unidades que han sido registradas en el sistema pero cuyos QRs aún no han sido impresos físicamente.

Consume [\[QrUseCase\]](#) para resolver los códigos QR escaneados a instancias de [\[Unidad\]](#), manteniendo una lista acumulada de unidades procesadas en la sesión.

Gestiona la carga de catálogos de prendas, tallas y escuelas, y expone la operación de agregar nuevas escuelas al sistema.

Gestiona los filtros de año, mes y escuela, y consume [\[ReporteRepository\]](#) para obtener los indicadores de ventas y los rankings de prendas y escuelas.

Consume [\[VentasUseCase\]](#) para obtener el listado de ventas y sus detalles, y coordina con [\[AlertasProvider\]](#) para refrescar alertas de stock ante operaciones que modifiquen el inventario.

3.5.2. Documentación de funciones miembro

3.5.2.1. `_inicializarDatos()`

```
Future< void > ChangeNotifier::_inicializarDatos () [inline], [private]
```

Carga los catálogos de escuelas y prendas necesarios para el funcionamiento del módulo.

En caso de error, inicializa las listas vacías para evitar estados inconsistentes.

3.5.2.2. agregarEscuela()

```
Future< Escuela?> ChangeNotifier::agregarEscuela (
    String nombre) [inline], [private]
```

Inserta una nueva escuela en el repositorio y actualiza la lista local.

Regla de negocio: el nombre no puede estar vacío o contener solo espacios. Retorna la instancia de [Escuela] insertada con su ID asignado, o null si el nombre no es válido.

3.5.2.3. agregarManual()

```
Future< void > ChangeNotifier::agregarManual (
    {required int idInventario} ) [inline], [private]
```

Agrega un ítem a la orden mediante selección manual de inventario, correspondiente a un pedido.

Delega la lógica de validación a [NuevaOrdenUseCase.agregarManual]. Lanza una excepción si el inventario no tiene stock disponible o ya fue agregado.

3.5.2.4. agregarQr()

```
Future< void > ChangeNotifier::agregarQr (
    String qr) [inline], [private]
```

Agrega un ítem a la orden mediante escaneo de QR, correspondiente a una venta.

Delega la validación y búsqueda del ítem a [NuevaOrdenUseCase.agregarPorQr]. Lanza una excepción si el QR es inválido, ya fue agregado, o no pertenece al inventario.

3.5.2.5. agregarUnidades()

```
Future< void > ChangeNotifier::agregarUnidades (
    int idInventario,
    int cantidad) [inline], [private]
```

Agrega unidades al inventario e imprime los QRs correspondientes.

Regla de negocio: no se ejecuta si ya hay una impresión en curso o si la cantidad es menor o igual a cero. Solicita permisos Bluetooth antes de proceder con la impresión. Al completarse exitosamente, refresca [AlertasProvider]. Propaga la excepción al caller en caso de error.

3.5.2.6. cambiarEscuela()

```
Future< void > ChangeNotifier::cambiarEscuela (
    int? idEscuela) [inline], [private]
```

Actualiza el filtro de escuela y recarga los reportes.

Null indica todas las escuelas.

3.5.2.7. `cambiarMonth()`

```
Future< void > ChangeNotifier::cambiarMonth (
    int? nuevo) [inline], [private]
```

Actualiza el filtro de mes y recarga los reportes.

Null indica todos los meses del año seleccionado.

3.5.2.8. `cambiarYear()`

```
Future< void > ChangeNotifier::cambiarYear (
    int nuevo) [inline], [private]
```

Actualiza el filtro de año y recarga los reportes.

3.5.2.9. `cargarAlertas()`

```
Future< void > ChangeNotifier::cargarAlertas () [inline], [private]
```

Ejecuta [AlertasStockUseCase.obtenerAlertas] y actualiza el estado local.

Activa el indicador de carga antes de la llamada y lo desactiva al terminar. En caso de error, reinicia ambas listas a vacío para evitar datos inconsistentes.

3.5.2.10. `cargarCatalogos()`

```
Future< void > ChangeNotifier::cargarCatalogos () [inline], [private]
```

Carga los catálogos de prendas, tallas y escuelas desde sus repositorios.

Activa el indicador de carga durante la operación y lo desactiva al finalizar.

3.5.2.11. `cargarDetalles()` [1/2]

```
Future< void > ChangeNotifier::cargarDetalles (
    int idPedido) [inline], [private]
```

Carga los detalles del pedido identificado por [idPedido].

Incluye información enriquecida de cada línea de detalle. Activa el indicador de carga durante la operación.

3.5.2.12. `cargarDetalles()` [2/2]

```
Future< void > ChangeNotifier::cargarDetalles (
    int idVenta) [inline], [private]
```

Carga los detalles de la venta identificada por [idVenta].

Activa el indicador de carga durante la operación y lo desactiva al finalizar. En caso de error, almacena el mensaje en [error].

3.5.2.13. cargarInventario() [1/2]

```
Future< void > ChangeNotifier::cargarInventario () [inline], [private]
```

Obtiene el inventario completo desde [\[InventarioRepository\]](#) y notifica a los listeners.

3.5.2.14. cargarInventario() [2/2]

```
Future< void > ChangeNotifier::cargarInventario (
    int idEscuela) [inline], [private]
```

Carga el inventario filtrado por la escuela indicada y, opcionalmente, por la prenda actualmente seleccionada.

Actualiza `_escuelaSeleccionada` buscando la coincidencia en el catálogo local.

3.5.2.15. cargarPedidos()

```
Future< void > ChangeNotifier::cargarPedidos () [inline], [private]
```

Carga la lista de pedidos pendientes desde [\[PedidoPendienteUseCase\]](#).

Activa el indicador de carga durante la operación y lo desactiva al finalizar. En caso de error, almacena el mensaje en `[error]`.

3.5.2.16. cargarPendientes()

```
Future< void > ChangeNotifier::cargarPendientes () [inline], [private]
```

Obtiene desde [\[UnidadRepository\]](#) las unidades con QR pendiente de impresión.

Activa el indicador de carga durante la operación y lo desactiva al finalizar, independientemente del resultado mediante el bloque `finally`.

3.5.2.17. cargarReportes()

```
Future< void > ChangeNotifier::cargarReportes () [inline], [private]
```

Carga todos los indicadores de reporte aplicando los filtros activos.

Regla de negocio: el ranking de escuelas solo se calcula cuando no hay una escuela específica seleccionada como filtro.

3.5.2.18. cargarVentas()

```
Future< void > ChangeNotifier::cargarVentas () [inline], [private]
```

Carga el historial completo de ventas desde [\[VentasUseCase\]](#).

Activa el indicador de carga durante la operación y lo desactiva al finalizar. En caso de error, almacena el mensaje en `[error]`.

3.5.2.19. completarPedido()

```
Future< void > ChangeNotifier::completarPedido (
    int idPedido) [inline], [private]
```

Marca el pedido como completado y recarga la lista de pedidos pendientes.

Limpia [_detalles] al finalizar ya que el pedido deja de estar pendiente. Propaga la excepción al caller en caso de error.

3.5.2.20. confirmar()

```
Future< void > ChangeNotifier::confirmar (
    { required int idUsuario, required String? nombreCliente, } ) [inline], [private]
```

Confirma y persiste la orden actual ejecutando [[NuevaOrdenUseCase.confirmarMixto](#)].

Activa el indicador de carga durante la operación. Al completarse exitosamente, limpia los items y refresca [AlertasProvider]. En caso de error, propaga la excepción al caller para manejo en la UI.

3.5.2.21. desregistrarUnidad()

```
Future< void > ChangeNotifier::desregistrarUnidad (
    { required int idPedido, required int idDetallePedido, } ) [inline], [private]
```

Desregistra la unidad asociada a un detalle de pedido, revirtiendo su registro.

Recarga los detalles del pedido al finalizar. Propaga la excepción al caller en caso de error.

3.5.2.22. eliminarDetallePedido()

```
Future< bool > ChangeNotifier::eliminarDetallePedido (
    { required int idPedido, required int idDetallePedido, } ) [inline], [private]
```

Elimina un detalle de pedido y recarga el estado del provider.

Retorna true si el pedido completo fue eliminado como consecuencia de eliminar su último detalle, en cuyo caso limpia [_detalles]. Retorna false si el pedido aún tiene otros detalles pendientes.

3.5.2.23. eliminarItem()

```
void ChangeNotifier::eliminarItem (
    ItemOrden item) [inline], [private]
```

Elimina un [ItemOrden] de la lista de items de la orden actual.

3.5.2.24. eliminarUnidad()

```
void ChangeNotifier::eliminarUnidad (
    int unidadId) [inline], [private]
```

Elimina la unidad identificada por [unidadId] de la lista de unidades procesadas.

3.5.2.25. firstWhere()

```
return escuelas ChangeNotifier::firstWhere (
    (e) ,
    e. idEscuela == idEscuelaSeleccionada,
    orElse: () ,
    Escuela(idEscuela:0, nombre:'Todas las escuelas') ) [private]
```

3.5.2.26. imprimirQrExistente()

```
Future< void > ChangeNotifier::imprimirQrExistente (
    int idUnidad) [inline], [private]
```

Reimprime el QR de una unidad ya existente sin crear un nuevo registro.

Lanza una excepción si ya hay una impresión en curso. Solicita permisos Bluetooth antes de proceder. No modifica el inventario ni actualiza las alertas de stock.

3.5.2.27. inicializar()

```
Future< void > ChangeNotifier::inicializar () [inline], [private]
```

Inicializa el provider cargando el catálogo de escuelas y los reportes iniciales.

3.5.2.28. InventarioProvider()

```
ChangeNotifier::InventarioProvider () [inline], [private]
```

Constructor que dispara la carga inicial de catálogos al instanciar el proveedor.

3.5.2.29. limpiar() [1/3]

```
void ChangeNotifier::limpiar () [inline], [private]
```

Reinicia las listas de alertas a vacío y notifica a los listeners.

Se utiliza al cerrar sesión o al salir de la sección de alertas.

3.5.2.30. limpiar() [2/3]

```
void ChangeNotifier::limpiar () [inline], [private]
```

Vacía la lista de items de la orden sin persistir cambios.

3.5.2.31. limpiar() [3/3]

```
void ChangeNotifier::limpiar () [inline], [private]
```

Reinicia el estado completo del provider, limpiando unidades y errores.

3.5.2.32. limpiarError()

```
void ChangeNotifier::limpiarError () [inline], [private]
```

Limpia el mensaje de error actual y notifica a los listeners.

3.5.2.33. limpiarMensaje()

```
void ChangeNotifier::limpiarMensaje () [inline], [private]
```

Reinicia [mensaje] a vacío y notifica a los listeners.

Se invoca al iniciar una pantalla para evitar mostrar mensajes residuales.

3.5.2.34. login()

```
Future< bool > ChangeNotifier::login (  
    String usuario,  
    String password) [inline], [private]
```

Ejecuta el flujo de autenticación con las credenciales proporcionadas.

Obtiene la lista de usuarios desde el repositorio y delega la validación al caso de uso [\[LoginUseCase\]](#). Retorna true si las credenciales son válidas, false en caso contrario. En caso de error durante la operación, se establece un mensaje genérico para evitar exponer detalles internos del sistema. Regla de negocio: no se distingue entre usuario inexistente y contraseña incorrecta para evitar enumeración de usuarios.

Cualquier excepción durante el proceso de autenticación se trata como credenciales inválidas, sin exponer el error real al cliente.

3.5.2.35. logout()

```
void ChangeNotifier::logout () [inline], [private]
```

Cierra la sesión del usuario actual limpiando el estado de autenticación.

3.5.2.36. navigateToInventory()

```
void ChangeNotifier::navigateToInventory (  
    BuildContext context) [inline], [private]
```

Navega a la pantalla de inventario.

3.5.2.37. navigateToNewOrder()

```
void ChangeNotifier::navigateToNewOrder (  
    BuildContext context) [inline], [private]
```

Navega a la pantalla de nueva orden.

Actualmente deshabilitado.

3.5.2.38. navigateToOrders()

```
void ChangeNotifier::navigateToOrders (  
    BuildContext context) [inline], [private]
```

Navega a la pantalla de órdenes.

Actualmente deshabilitado.

3.5.2.39. pedidoCompleto()

```
Future< bool > ChangeNotifier::pedidoCompleto (  
    int idPedido) [inline], [private]
```

Verifica si todos los detalles del pedido identificado por [idPedido] han sido registrados con QR.

Retorna true si el pedido está listo para ser completado.

3.5.2.40. PrintProvider()

```
ChangeNotifier::PrintProvider (  
    this. useCase,  
    this. alertasProvider) [private]
```

Crea una instancia de [PrintProvider] con el [useCase] y [alertasProvider] requeridos.

3.5.2.41. procesarQr()

```
Future< void > ChangeNotifier::procesarQr (  
    String qr) [inline], [private]
```

Procesa un código QR escaneado y agrega la unidad correspondiente a la lista.

Regla de negocio: no se procesa un nuevo QR si ya hay uno en curso. Regla de negocio: una unidad ya escaneada no puede agregarse de nuevo. Si el QR no corresponde a ninguna unidad registrada, establece un mensaje de error.

3.5.2.42. QrProvider()

```
ChangeNotifier::QrProvider (
    this. useCase) [private]
```

Crea una instancia de [QrProvider] con el [useCase] requerido.

3.5.2.43. recargarEscuelas()

```
Future< void > ChangeNotifier::recargarEscuelas () [inline], [private]
```

Recarga el catálogo de escuelas desde el repositorio y, si hay una escuela seleccionada, recarga también el inventario activo para reflejar cualquier cambio reciente en los datos.

3.5.2.44. refrescar()

```
Future< void > ChangeNotifier::refrescar () [inline], [private]
```

Actualiza las alertas desde el caso de uso sin activar el indicador de carga.

Diseñado para refrescos en segundo plano donde no se requiere feedback visual. Los errores se suprimen silenciosamente para no interrumpir la UI.

3.5.2.45. refrescarAlertas()

```
Future< void > ChangeNotifier::refrescarAlertas () [inline], [private]
```

Refresca las alertas de stock a través de [AlertasProvider].

Se invoca manualmente ante operaciones que modifiquen el inventario, como devoluciones, cancelaciones o ajustes de stock.

3.5.2.46. RegistrarInventarioProvider()

```
ChangeNotifier::RegistrarInventarioProvider (
    { required this.prendaRepository, required this.tallaRepository, required this.↵
    escuelaRepository, } ) [private]
```

Crea una instancia de [RegistrarInventarioProvider] con los repositorios requeridos.

3.5.2.47. registrarQrPedido()

```
Future< Map< String, dynamic >>> ChangeNotifier::registrarQrPedido (
    { required int idPedido, required int idDetallePedido, required int idInventario↵
    Esperado, required String qr, bool forzarMovimiento=false, } ) [inline], [private]
```

Registra un QR escaneado contra un detalle de pedido específico.

Retorna un mapa con información del conflicto si la unidad escaneada no corresponde al inventario esperado, o null si el registro fue exitoso. El parámetro [forzarMovimiento] permite ignorar conflictos de inventario. Recarga los detalles del pedido al finalizar la operación.

3.5.2.48. return() [1/2]

```
ChangeNotifier::return (
    escuelasMasVentas.first as num[ 'total']) [private]
```

3.5.2.49. return() [2/2]

```
ChangeNotifier::return (
    prendasMasVendidas.first as num[ 'cantidad']) [private]
```

3.5.2.50. seleccionarPrenda()

```
Future< void > ChangeNotifier::seleccionarPrenda (
    int? idPrenda) [inline], [private]
```

Actualiza el filtro de prenda seleccionada y recarga el inventario si hay una escuela activa.

Un valor null elimina el filtro por prenda.

3.5.2.51. setUsername()

```
void ChangeNotifier::setUserName (
    String name) [inline], [private]
```

Actualiza el nombre del usuario y notifica a los listeners.

3.5.2.52. solicitarPermisosBluetooth()

```
Future< bool > ChangeNotifier::solicitarPermisosBluetooth () [inline], [private]
```

Solicita los permisos de Bluetooth y ubicación requeridos para la impresión.

Retorna true únicamente si los tres permisos fueron concedidos: bluetoothScan, bluetoothConnect y location.

3.5.3. Documentación de datos miembro**3.5.3.1. _agotados**

```
List<AlertaStock> ChangeNotifier::_agotados = [] [private]
```

Lista de inventarios con stock en cero.

3.5.3.2. _cargando [1/2]

```
bool ChangeNotifier::_cargando = false [private]
```

Indica si hay una operación asíncrona en curso.

Indica si hay una operación de carga en curso.

3.5.3.3. `_cargando` [2/2]

```
AlertasProvider ({required AlertasStockUseCase useCase}) bool ChangeNotifier::_cargando =  
false [private]
```

Crea una instancia de [AlertasProvider] con el [useCase] requerido.

Indica si hay una operación de carga en curso.

3.5.3.4. `_criticos` [1/2]

```
List<AlertaStock> ChangeNotifier::_criticos = [] [private]
```

Lista de inventarios con stock por debajo del umbral crítico.

3.5.3.5. `_criticos` [2/2]

```
List<AlertaStock> get ChangeNotifier::_criticos [private]
```

3.5.3.6. `_detalles`

```
List< Map< String, dynamic > > ChangeNotifier::_detalles = [] [private]
```

Lista de detalles del pedido actualmente seleccionado.

Lista de detalles de la venta actualmente seleccionada.

3.5.3.7. `_error`

```
String ChangeNotifier::_error [private]
```

Mensaje de error producido durante el último intento de autenticación. Null si no hay error.

Mensaje del último error ocurrido. Null si no hay error activo.

3.5.3.8. `_escuelaRepo`

```
final ChangeNotifier::_escuelaRepo = EscuelaRepository() [private]
```

Repositorio encargado de obtener el catálogo de escuelas.

3.5.3.9. `_escuelaRepository`

```
final EscuelaRepository ChangeNotifier::_escuelaRepository [private]
```

Repositorio de acceso a datos de escuelas para poblar el filtro.

3.5.3.10. _escuelas

```
List<Escuela> ChangeNotifier::_escuelas = [] [private]
```

Lista de escuelas disponibles cargadas al inicializar el proveedor.

3.5.3.11. _escuelaSeleccionada

```
Escuela ChangeNotifier::_escuelaSeleccionada [private]
```

Escuela seleccionada actualmente para filtrar el inventario.

3.5.3.12. _idPrendaSeleccionada

```
int ChangeNotifier::_idPrendaSeleccionada [private]
```

Identificador de la prenda seleccionada como filtro adicional. Null si no se aplica filtro por prenda.

3.5.3.13. _inventario

```
List<Map<String, dynamic> > ChangeNotifier::_inventario = [] [private]
```

Lista del inventario completo disponible para selección manual.

3.5.3.14. _inventarioRepo

```
final ChangeNotifier::_inventarioRepo = InventarioRepository() [private]
```

Repositorio encargado de obtener y filtrar los registros de inventario.

3.5.3.15. _inventarioRepository

```
final InventarioRepository ChangeNotifier::_inventarioRepository [private]
```

Repositorio para obtener el inventario disponible.

3.5.3.16. _items

```
NuevaOrdenProvider ({ required InventarioRepository inventarioRepository, required UnidadRepository  
unidadRepository, required VentaRepository ventaRepository, required PedidoRepository pedido←  
Repository, required OrdenRepository ordenRepository, required QrUseCase qrUseCase, required  
this.alertasProvider, }) final List<ItemOrden> ChangeNotifier::_items = [] [private]
```

Crea una instancia de [NuevaOrdenProvider] inyectando todos los repositorios y casos de uso necesarios para el flujo de nueva orden.

Lista de items agregados a la orden actual.

3.5.3.17. `_itemsInventario`

```
List<Map<String, dynamic> > ChangeNotifier::_itemsInventario = [] [private]
```

Lista de items del inventario resultante del filtrado aplicado.

3.5.3.18. `_loginUseCase`

```
final ChangeNotifier::_loginUseCase = LoginUseCase() [private]
```

Caso de uso que ejecuta la lógica de autenticación.

3.5.3.19. `_pedidos`

```
PedidosPendientesProvider ({ required PedidoRepository pedidoRepository, required UnidadRepository  
unidadRepository, required VentaRepository ventaRepository, required QrUseCase qrUseCase, })  
List<Pedido> ChangeNotifier::_pedidos = [] [private]
```

Crea una instancia de [PedidosPendientesProvider] inyectando los repositorios y el caso de uso de QR necesarios para el flujo de pedidos.

Lista de pedidos pendientes cargados desde el repositorio.

3.5.3.20. `_pendientes`

```
QrPendientesProvider ({required UnidadRepository repository}) List<Map<String, dynamic> >  
ChangeNotifier::_pendientes = [] [private]
```

Crea una instancia de [QrPendientesProvider] con el [repository] requerido.

Lista de unidades con QR pendiente de impresión.

3.5.3.21. `_prendaRepo`

```
final ChangeNotifier::_prendaRepo = PrendaRepository() [private]
```

Repositorio encargado de obtener el catálogo de prendas.

3.5.3.22. `_prendas`

```
List<Prenda> ChangeNotifier::_prendas = [] [private]
```

Lista de prendas disponibles cargadas al inicializar el proveedor.

3.5.3.23. `_procesando`

```
bool ChangeNotifier::_procesando = false [private]
```

Indica si hay un procesamiento de QR en curso.

3.5.3.24. _repository [1/2]

```
final UnidadRepository ChangeNotifier::_repository [private]
```

Repositorio de acceso a datos de unidades.

3.5.3.25. _repository [2/2]

```
final ReporteRepository ChangeNotifier::_repository [private]
```

Repositorio de acceso a datos de reportes de ventas.

3.5.3.26. _unidades

```
final List<Unidad> ChangeNotifier::_unidades = [] [private]
```

Lista interna de unidades procesadas en la sesión actual.

3.5.3.27. _useCase [1/4]

```
final AlertasStockUseCase ChangeNotifier::_useCase [private]
```

Caso de uso encargado de obtener las alertas de stock.

3.5.3.28. _useCase [2/4]

```
final NuevaOrdenUseCase ChangeNotifier::_useCase [private]
```

Caso de uso que encapsula la lógica de negocio para crear órdenes.

3.5.3.29. _useCase [3/4]

```
final PedidoPendienteUseCase ChangeNotifier::_useCase [private]
```

Caso de uso que encapsula la lógica de negocio de pedidos pendientes.

3.5.3.30. _useCase [4/4]

```
final VentasUseCase ChangeNotifier::_useCase [private]
```

Caso de uso que encapsula la lógica de negocio de ventas.

3.5.3.31. `_userName`

```
String ChangeNotifier::_userName = 'Usuario' [private]
```

Nombre del usuario actualmente autenticado.

Valor por defecto: 'Usuario'.

3.5.3.32. `_usuarioActual`

```
Usuario ChangeNotifier::_usuarioActual [private]
```

Usuario autenticado actualmente. Null si no hay sesión activa.

3.5.3.33. `_usuarioRepo`

```
final ChangeNotifier::_usuarioRepo = UsuarioRepository() [private]
```

Repositorio encargado de obtener los datos de usuarios.

3.5.3.34. `_ventas`

```
VentasProvider ({ required VentaRepository ventaRepository, required this.alertasProvider, })  
List<Venta> ChangeNotifier::_ventas = [] [private]
```

Crea una instancia de [VentasProvider] inyectando el repositorio y el provider de alertas requeridos.

Lista de ventas cargadas desde el repositorio.

3.5.3.35. `agotados`

```
List<AlertaStock> get ChangeNotifier::agotados => _agotados [private]
```

Expone los registros de inventario agotados.

3.5.3.36. `alertasProvider`

```
final AlertasProvider ChangeNotifier::alertasProvider [private]
```

Referencia al provider de alertas para refrescarlas al confirmar una orden.

Referencia al provider de alertas para refrescarlas ante cambios de stock.

Referencia al provider de alertas para refrescarlas tras agregar unidades.

3.5.3.37. cantidadVentas

```
int ChangeNotifier::cantidadVentas = 0 [private]
```

Cantidad de ventas realizadas según los filtros activos.

3.5.3.38. cargando [1/3]

```
bool get ChangeNotifier::cargando => _cargando [private]
```

Expone el estado de carga a los widgets consumidores.

3.5.3.39. cargando [2/3]

```
bool ChangeNotifier::cargando = false [private]
```

Indica si hay una operación de carga en curso.

3.5.3.40. cargando [3/3]

```
ReportesProvider ({ required ReporteRepository repository, required EscuelaRepository escuelaRepository,  
}) bool ChangeNotifier::cargando = false [private]
```

Crea una instancia de [ReportesProvider] con los repositorios requeridos.

Indica si hay una operación de carga en curso.

3.5.3.41. criticos

```
List<AlertaStock> get ChangeNotifier::criticos => _criticos [private]
```

Expone los registros de inventario en estado crítico.

3.5.3.42. detalles

```
List< Map< String, dynamic > > get ChangeNotifier::detalles => _detalles [private]
```

Expone los detalles del pedido seleccionado a los widgets consumidores.

Expone los detalles de la venta seleccionada a los widgets consumidores.

3.5.3.43. error [1/2]

```
String ChangeNotifier::error => _error [private]
```

Expone el mensaje de error a los widgets consumidores.

Mensaje del último error ocurrido. Null si no hay error activo.

3.5.3.44. error [2/2]

```
String ChangeNotifier::error [private]
```

Mensaje de error del último fallo ocurrido. Null si no hay error.

Mensaje del último error ocurrido. Null si no hay error activo.

3.5.3.45. escuelaRepository

```
final EscuelaRepository ChangeNotifier::escuelaRepository [private]
```

Repositorio de acceso a datos de escuelas.

3.5.3.46. escuelas [1/2]

```
List< Escuela > ChangeNotifier::escuelas => _escuelas [private]
```

Lista de escuelas disponibles para el filtro.

3.5.3.47. escuelas [2/2]

```
List<Escuela> ChangeNotifier::escuelas = [] [private]
```

Lista de escuelas disponibles en el catálogo.

Lista de escuelas disponibles para el filtro.

3.5.3.48. escuelaSeleccionada

```
Escuela get ChangeNotifier::escuelaSeleccionada => _escuelaSeleccionada [private]
```

3.5.3.49. escuelasMasVentas

```
List<Map<String, dynamic> > ChangeNotifier::escuelasMasVentas = [] [private]
```

Ranking de escuelas con más ventas.

Se vacía cuando hay una escuela específica seleccionada como filtro.

3.5.3.50. estaAutenticado

```
bool get ChangeNotifier::estaAutenticado => _usuarioActual != null [private]
```

Retorna true si existe un usuario autenticado en sesión.

3.5.3.51. hayAlertas

```
bool get ChangeNotifier::hayAlertas => _agotados.isNotEmpty || _criticos.isNotEmpty [private]
```

Retorna true si existe al menos una alerta activa de cualquier tipo.

3.5.3.52. idEscuelaSeleccionada

```
int ChangeNotifier::idEscuelaSeleccionada [private]
```

ID de la escuela seleccionada como filtro. Null indica todas las escuelas.

3.5.3.53. idPrendaSeleccionada

```
int get ChangeNotifier::idPrendaSeleccionada => _idPrendaSeleccionada [private]
```

3.5.3.54. inventario

```
List<Map<String, dynamic> > get ChangeNotifier::inventario => _inventario [private]
```

Expone el inventario disponible a los widgets consumidores.

3.5.3.55. items

```
List<ItemOrden> get ChangeNotifier::items => _items [private]
```

Expone los items de la orden actual como lista de solo lectura.

3.5.3.56. itemsInventario

```
List<Map<String, dynamic> > get ChangeNotifier::itemsInventario => _itemsInventario [private]
```

3.5.3.57. loading

```
bool ChangeNotifier::loading = false [private]
```

Indica si hay una operación de impresión en curso.

3.5.3.58. maxCantidadPrenda

```
double get ChangeNotifier::maxCantidadPrenda [private]
```

Valor inicial:

```
{  
    if (prendasMasVendidas.isEmpty) return 1  
}
```

Retorna la cantidad de la prenda más vendida para normalizar barras.

Retorna 1 si la lista está vacía para evitar división por cero.

3.5.3.59. maxTotalEscuela

```
double get ChangeNotifier::maxTotalEscuela [private]
```

Valor inicial:

```
{  
    if (escuelasMasVentas.isEmpty) return 1
```

Retorna el total de ventas de la escuela con mayor venta para normalizar barras.

Retorna 1 si la lista está vacía para evitar división por cero.

3.5.3.60. mensaje

```
String ChangeNotifier::mensaje = "" [private]
```

Mensaje de resultado de la última operación ejecutada.

Vacío si no hay mensaje activo.

3.5.3.61. meses [1/2]

```
const ChangeNotifier::meses [private]
```

Valor inicial:

```
= [  
    "  
    'Enero',  
    'Febrero',  
    'Marzo',  
    'Abril',  
    'Mayo',  
    'Junio',  
    'Julio',  
    'Agosto',  
    'Septiembre',  
    'Octubre',  
    'Noviembre',  
    'Diciembre',  
]
```

3.5.3.62. meses [2/2]

```
return ChangeNotifier::meses[month!] [private]
```

3.5.3.63. month

```
int ChangeNotifier::month [private]
```

Mes actualmente seleccionado como filtro. Null indica todos los meses.

3.5.3.64. nombreEscuelaSeleccionada

```
String get ChangeNotifier::nombreEscuelaSeleccionada [private]
```

Valor inicial:

```
{  
  if (idEscuelaSeleccionada == null) return 'Todas las escuelas'
```

Retorna el nombre de la escuela seleccionada, o 'Todas las escuelas' si no hay filtro activo.

3.5.3.65. nombreMesSeleccionado

```
String get ChangeNotifier::nombreMesSeleccionado [private]
```

Valor inicial:

```
{  
  if (month == null) return 'Todo el año'
```

Retorna el nombre del mes seleccionado, o 'Todo el año' si no hay mes activo.

3.5.3.66. pedidos

```
List<Pedido> get ChangeNotifier::pedidos => _pedidos [private]
```

Expone la lista de pedidos pendientes a los widgets consumidores.

3.5.3.67. pendientes

```
List<Map<String, dynamic> > get ChangeNotifier::pendientes => _pendientes [private]
```

Expone la lista de unidades pendientes a los widgets consumidores.

3.5.3.68. prendaRepository

```
final PrendaRepository ChangeNotifier::prendaRepository [private]
```

Repositorio de acceso a datos de prendas.

3.5.3.69. prendas [1/2]

```
List<Prenda> get ChangeNotifier::prendas => _prendas [private]
```

3.5.3.70. prendas [2/2]

```
List<Prenda> ChangeNotifier::prendas = [] [private]
```

Lista de prendas disponibles en el catálogo.

3.5.3.71. prendasMasVendidas

```
List<Map<String, dynamic> > ChangeNotifier::prendasMasVendidas = [] [private]
```

Ranking de prendas más vendidas según los filtros activos.

3.5.3.72. procesando

```
bool get ChangeNotifier::procesando => _procesando [private]
```

Expone el estado de procesamiento a los widgets consumidores.

3.5.3.73. tallaRepository

```
final TallaRepository ChangeNotifier::tallaRepository [private]
```

Repositorio de acceso a datos de tallas.

3.5.3.74. tallas

```
List<Talla> ChangeNotifier::tallas = [] [private]
```

Lista de tallas disponibles en el catálogo.

3.5.3.75. todas

```
List<AlertaStock> get ChangeNotifier::todas => [..._agotados [private]
```

Retorna la lista unificada de alertas, con agotados antes que críticos.

3.5.3.76. total

```
double get ChangeNotifier::total [private]
```

Valor inicial:

```
{  
    return _items.fold(0, (sum, item) => sum + item.precioUnitario)
```

Calcula el total monetario de la orden sumando el precio unitario de cada item.

3.5.3.77. totalAlertas

```
int get ChangeNotifier::totalAlertas => _agotados.length + _criticos.length [private]
```

Retorna el total de alertas activas sumando agotados y críticos.

3.5.3.78. totalPrendas

```
int get ChangeNotifier::totalPrendas => _items.length [private]
```

Retorna el número total de prendas agregadas a la orden.

3.5.3.79. totalVendido

```
double ChangeNotifier::totalVendido = 0 [private]
```

Total monetario de ventas según los filtros activos.

3.5.3.80. unidades

```
List<Unidad> get ChangeNotifier::unidades => List.unmodifiable(_unidades) [private]
```

Expone la lista de unidades como vista de solo lectura.

3.5.3.81. useCase [1/2]

```
final PrintUseCase ChangeNotifier::useCase [private]
```

Caso de uso que encapsula la lógica de impresión de QRs.

3.5.3.82. useCase [2/2]

```
final QrUseCase ChangeNotifier::useCase [private]
```

Caso de uso encargado de resolver un código QR a una [Unidad].

3.5.3.83. userName

```
String get ChangeNotifier::userName => _userName [private]
```

Expone el nombre del usuario a los widgets consumidores.

3.5.3.84. usuarioActual

```
Usuario get ChangeNotifier::usuarioActual => _usuarioActual [private]
```

3.5.3.85. ventas

```
List<Venta> get ChangeNotifier::ventas => _ventas [private]
```

Expone la lista de ventas a los widgets consumidores.

3.5.3.86. year

```
int ChangeNotifier::year = DateTime.now().year [private]
```

Año actualmente seleccionado como filtro. Por defecto el año en curso.

La documentación de esta clase está generada de los siguientes archivos:

- [lib/business/providers/auth_provider.dart](#)
- [lib/business/providers/inventario_provider.dart](#)
- [lib/presentation/providers/alertas_provider.dart](#)
- [lib/presentation/providers/home_provider.dart](#)
- [lib/presentation/providers/nueva_order_provider.dart](#)
- [lib/presentation/providers/pedidos_pendientes_provider.dart](#)
- [lib/presentation/providers/print_provider.dart](#)
- [lib/presentation/providers/qr_pendientes_provider.dart](#)
- [lib/presentation/providers/qr_provider.dart](#)
- [lib/presentation/providers/registras_inventario_provider.dart](#)
- [lib/presentation/providers/reportes_provider.dart](#)
- [lib/presentation/providers/ventas_provider.dart](#)

3.6. Referencia de la clase CustomPainter

Overlay visual del scanner que dibuja:

Métodos privados

- override void [paint](#) (Canvas canvas, Size size,)
- override bool [shouldRepaint](#) (covariant [CustomPainter](#) oldDelegate,)

3.6.1. Descripción detallada

Overlay visual del scanner que dibuja:

- una máscara oscura sobre la pantalla,
- un recuadro central transparente donde se espera el QR,
- esquinas resaltadas en color para guiar al usuario.

3.6.2. Documentación de funciones miembro

3.6.2.1. paint()

```
override void CustomPainter::paint (  
    Canvas canvas,  
    Size size) [inline], [private]
```

3.6.2.2. shouldRepaint()

```
override bool CustomPainter::shouldRepaint (  
    covariant CustomPainter oldDelegate) [inline], [private]
```

La documentación de esta clase está generada del siguiente archivo:

- [lib/presentation/screens/qr_screen.dart](#)

3.7. Referencia de la clase DatabaseHelper

Helper que gestiona el ciclo de vida de la base de datos SQLite local.

Métodos privados

- [DatabaseHelper _internal](#) ()
- `Future< Database > _initDB () async`
Obtiene la ruta del sistema de archivos y abre la base de datos.
- `Future< void > _onCreate (Database db, int version,) async`
Crea el esquema completo de la base de datos en la primera ejecución.

Atributos privados

- `Future< Database > get database async`
Retorna la instancia de la base de datos, inicializándola si aún no existe.
- `return _database`

Atributos estáticos privados

- `static final DatabaseHelper instance`
Instancia única del helper accesible globalmente.
- `static Database _database`
Referencia a la base de datos abierta. Null hasta la primera inicialización.

3.7.1. Descripción detallada

Helper que gestiona el ciclo de vida de la base de datos SQLite local.

Implementa el patrón Singleton para garantizar una única instancia de la base de datos durante toda la ejecución de la aplicación.

3.7.2. Documentación de funciones miembro

3.7.2.1. `_initDB()`

```
Future< Database > DatabaseHelper::_initDB () [inline], [private]
```

Obtiene la ruta del sistema de archivos y abre la base de datos.

Habilita las llaves foráneas mediante PRAGMA en la configuración inicial. El manejador de actualizaciones está definido pero vacío, pendiente de implementación para futuras migraciones de esquema.

3.7.2.2. `_internal()`

```
DatabaseHelper DatabaseHelper::_internal () [private]
```

3.7.2.3. `_onCreate()`

```
Future< void > DatabaseHelper::_onCreate (
    Database db,
    int version) [inline], [private]
```

Crea el esquema completo de la base de datos en la primera ejecución.

Define todas las tablas, restricciones de llaves foráneas y unicidad, e inserta los datos iniciales requeridos para el funcionamiento del sistema. Tabla de usuarios del sistema. El campo [usuario] es único y [password_hash] almacena el hash SHA-256 con salt de la contraseña.

Tabla de escuelas a las que se asocian los registros de inventario.

Catálogo de tipos de prenda disponibles en el sistema.

Catálogo de tallas disponibles. El campo [talla] es único para evitar duplicados en el catálogo.

Tabla de inventario que representa cada combinación única de escuela, prenda y talla con su precio asignado. La restricción UNIQUE sobre (id_escuela, id_prenda, id_talla) garantiza que no existan registros duplicados por combinación.

Tabla de unidades físicas asociadas a un registro de inventario. Cada unidad representa una prenda física identificable por QR. [activo] indica si la unidad está disponible (1) o fue vendida/dada de baja (0). [pendiente_impresion] indica si la etiqueta QR aún no ha sido impresa.

Tabla de órdenes que agrupa ventas y pedidos generados en una misma transacción. Permite vincular una venta directa con su pedido asociado dentro de una orden mixta.

Tabla de ventas completadas. Puede estar vinculada a una orden de origen cuando proviene de una transacción mixta con pedido. [estado] por defecto es 'completada' al momento de la inserción.

Tabla de detalle de venta. Cada registro representa una unidad física incluida en la venta con su precio al momento de la transacción.

Tabla de pedidos pendientes de entrega. Puede vincularse a una venta de origen cuando se generó como parte de una orden mixta. [estado] refleja el ciclo de vida del pedido (pendiente/completado).

Tabla de detalle de pedido. Cada registro representa una prenda encargada identificada por su inventario. [id←_unidad_registrada] se asigna al momento de la entrega y [registrado] cambia a 1 cuando la unidad física es confirmada.

Inserción del usuario administrador inicial del sistema. La contraseña se hashea con SHA-256 usando el mismo salt definido en [LoginUseCase] para mantener consistencia.

Catálogo inicial de prendas disponibles en el sistema. Estos valores son fijos y se insertan una única vez al crear la base de datos.

Catálogo inicial de tallas disponibles en el sistema. Incluye tallas numéricas de niño y tallas estándar de adulto.

3.7.3. Documentación de datos miembro

3.7.3.1. `_database` [1/2]

```
Database DatabaseHelper::_database [static], [private]
```

Referencia a la base de datos abierta. Null hasta la primera inicialización.

3.7.3.2. `_database` [2/2]

```
return DatabaseHelper::_database [private]
```

3.7.3.3. `async`

```
Future<Database> get database DatabaseHelper::async [private]
```

Valor inicial:

```
{  
    if (_database != null) {  
        return _database!;  
    }  
  
    _database = await _initDB()  
}
```

Retorna la instancia de la base de datos, inicializándola si aún no existe.

3.7.3.4. `instance`

```
final DatabaseHelper DatabaseHelper::instance [static], [private]
```

Valor inicial:

```
=  
    DatabaseHelper._internal()
```

Instancia única del helper accesible globalmente.

La documentación de esta clase está generada del siguiente archivo:

- `lib/data/database/database_helper.dart`

3.8. Referencia de la clase EscuelaRepository

Repositorio que gestiona las operaciones de acceso a datos para la tabla de escuelas.

Métodos privados

- `Future< int > insertar (Escuela escuela) async`
Inserta una nueva escuela en la base de datos.
- `Future< Escuela?> obtenerPorId (int id) async`
Retorna la escuela con el identificador indicado.
- `Future< List< Escuela > > obtenerTodas () async`
Retorna todas las escuelas registradas, ordenadas alfabéticamente por nombre.
- `Future< int > actualizar (Escuela escuela) async`
Actualiza los datos de una escuela existente identificada por su id.
- `Future< int > eliminar (int id) async`
Elimina la escuela con el identificador indicado.

Atributos privados

- `Future< Database > get_db async => await DatabaseHelper.instance.database`

3.8.1. Descripción detallada

Repositorio que gestiona las operaciones de acceso a datos para la tabla de escuelas.

3.8.2. Documentación de funciones miembro

3.8.2.1. actualizar()

```
Future< int > EscuelaRepository::actualizar (  
    Escuela escuela) [inline], [private]
```

Actualiza los datos de una escuela existente identificada por su id.

3.8.2.2. eliminar()

```
Future< int > EscuelaRepository::eliminar (  
    int id) [inline], [private]
```

Elimina la escuela con el identificador indicado.

3.8.2.3. insertar()

```
Future< int > EscuelaRepository::insertar (  
    Escuela escuela) [inline], [private]
```

Inserta una nueva escuela en la base de datos.

Lanza una excepción si ya existe un registro con el mismo nombre (`ConflictAlgorithm.abort`).

3.8.2.4. obtenerPorId()

```
Future< Escuela?> EscuelaRepository::obtenerPorId (
    int id) [inline], [private]
```

Retorna la escuela con el identificador indicado.

Retorna null si no existe.

3.8.2.5. obtenerTodas()

```
Future< List< Escuela > > EscuelaRepository::obtenerTodas () [inline], [private]
```

Retorna todas las escuelas registradas, ordenadas alfabéticamente por nombre.

3.8.3. Documentación de datos miembro

3.8.3.1. async

```
Future<Database> get _db EscuelaRepository::async => await DatabaseHelper.instance.database
[private]
```

La documentación de esta clase está generada del siguiente archivo:

- lib/data/repositories/[escuela_repository.dart](#)

3.9. Referencia de la clase InventarioRepository

Repository que gestiona las operaciones de acceso a datos para la tabla de inventario y sus consultas relacionadas con stock.

Métodos privados

- Future< int > [insertar](#) (Inventario [inventario](#)) [async](#)
Inserta un nuevo registro de inventario.
- Future< Inventario?> [obtenerPorId](#) (int id) [async](#)
Retorna el registro de inventario con el identificador indicado.
- Future< List< Inventario > > [obtenerTodos](#) () [async](#)
Retorna todos los registros de inventario sin filtro aplicado.
- Future< List< Inventario > > [obtenerPorEscuela](#) (int idEscuela) [async](#)
Retorna todos los registros de inventario asociados a una escuela específica.
- Future< Inventario?> [obtenerPorCombinacion](#) ({ required int idEscuela, required int idPrenda, required int idTalla, }) [async](#)
Busca un registro de inventario por la combinación exacta de escuela, prenda y talla.
- Future< int > [actualizar](#) (Inventario [inventario](#)) [async](#)
Actualiza el registro de inventario identificado por su id.
- Future< int > [eliminar](#) (int id) [async](#)

Elimina el registro de inventario con el identificador indicado.

- `Future< int > contarStock (int idInventario) async`

Retorna el conteo de unidades activas asociadas a un registro de inventario.

- `Future< List< Map< String, dynamic > > > obtenerInventarioFiltrado ({ required int idEscuela, int? id←Prenda, }) async`

Retorna el inventario de una escuela enriquecido con nombre de prenda, talla, precio y stock actual.

- `Future< List< Map< String, dynamic > > > obtenerInventarioCompleto () async`

Retorna el inventario completo de todas las escuelas con nombre de escuela, prenda, talla, precio y stock actual calculado por subconsulta.

- `Future< List< Map< String, dynamic > > > obtenerStockAgotado () async`

Retorna los registros de inventario cuyo stock activo es igual a cero.

- `Future< List< Map< String, dynamic > > > obtenerStockCritico () async`

Retorna los registros de inventario con stock activo entre 1 y 3 unidades (inclusive).

Atributos privados

- `Future< Database > get_db async => await DatabaseHelper.instance.database`

3.9.1. Descripción detallada

Repositorio que gestiona las operaciones de acceso a datos para la tabla de inventario y sus consultas relacionadas con stock.

3.9.2. Documentación de funciones miembro

3.9.2.1. actualizar()

```
Future< int > InventarioRepository::actualizar (
    Inventario inventario) [inline], [private]
```

Actualiza el registro de inventario identificado por su id.

3.9.2.2. contarStock()

```
Future< int > InventarioRepository::contarStock (
    int idInventario) [inline], [private]
```

Retorna el conteo de unidades activas asociadas a un registro de inventario.

Utilizado para determinar el stock disponible en tiempo real.

3.9.2.3. eliminar()

```
Future< int > InventarioRepository::eliminar (
    int id) [inline], [private]
```

Elimina el registro de inventario con el identificador indicado.

3.9.2.4. insertar()

```
Future< int > InventarioRepository::insertar (
    Inventario inventario) [inline], [private]
```

Inserta un nuevo registro de inventario.

Si ya existe la misma combinación de escuela, prenda y talla, la operación se ignora sin lanzar error.

3.9.2.5. obtenerInventarioCompleto()

```
Future< List< Map< String, dynamic > > > InventarioRepository::obtenerInventarioCompleto ()
[inline], [private]
```

Retorna el inventario completo de todas las escuelas con nombre de escuela, prenda, talla, precio y stock actual calculado por subconsulta.

Ordenado por escuela, prenda y talla de forma ascendente.

3.9.2.6. obtenerInventarioFiltrado()

```
Future< List< Map< String, dynamic > > > InventarioRepository::obtenerInventarioFiltrado (
    { required int idEscuela, int? idPrenda, } ) [inline], [private]
```

Retorna el inventario de una escuela enriquecido con nombre de prenda, talla, precio y stock actual.

Permite filtrar opcionalmente por prenda. Los resultados se ordenan por nombre de prenda y talla de forma ascendente.

3.9.2.7. obtenerPorCombinacion()

```
Future< Inventario?> InventarioRepository::obtenerPorCombinacion (
    { required int idEscuela, required int idPrenda, required int idTalla, } ) [inline],
[private]
```

Busca un registro de inventario por la combinación exacta de escuela, prenda y talla.

Retorna null si no existe ningún registro con esa combinación.

3.9.2.8. obtenerPorEscuela()

```
Future< List< Inventario > > InventarioRepository::obtenerPorEscuela (
    int idEscuela) [inline], [private]
```

Retorna todos los registros de inventario asociados a una escuela específica.

3.9.2.9. obtenerPorId()

```
Future< Inventario?> InventarioRepository::obtenerPorId (
    int id) [inline], [private]
```

Retorna el registro de inventario con el identificador indicado.

Retorna null si no existe.

3.9.2.10. obtenerStockAgotado()

```
Future< List< Map< String, dynamic > > > InventarioRepository::obtenerStockAgotado () [inline],
[private]
```

Retorna los registros de inventario cuyo stock activo es igual a cero.

Utilizado por [AlertasStockUseCase] para generar alertas de agotado.

3.9.2.11. obtenerStockCritico()

```
Future< List< Map< String, dynamic > > > InventarioRepository::obtenerStockCritico () [inline],
[private]
```

Retorna los registros de inventario con stock activo entre 1 y 3 unidades (inclusive).

Utilizado por [AlertasStockUseCase] para generar alertas de stock crítico. Los resultados se ordenan por stock ascendente para priorizar los más críticos.

3.9.2.12. obtenerTodos()

```
Future< List< Inventario > > InventarioRepository::obtenerTodos () [inline], [private]
```

Retorna todos los registros de inventario sin filtro aplicado.

3.9.3. Documentación de datos miembro

3.9.3.1. async

```
Future<Database> get _db InventarioRepository::async => await DatabaseHelper.instance.↵
database [private]
```

La documentación de esta clase está generada del siguiente archivo:

- [lib/data/repositories/inventario_repository.dart](#)

3.10. Referencia de la clase LoginUseCase

Caso de uso para el inicio de sesión.

Métodos privados

- `Future< Usuario?> execute` (`List< Usuario > usuarios`, `String username`, `String password`,) `async`
Ejecuta la lógica de autenticación contra la lista de usuarios proporcionada.
- `String _hashPassword` (`String password`)
Genera un hash SHA-256 de la contraseña concatenada con un salt estático.

3.10.1. Descripción detallada

Caso de uso para el inicio de sesión.

Se encarga de validar credenciales y retornar el usuario autenticado.

3.10.2. Documentación de funciones miembro

3.10.2.1. _hashPassword()

```
String LoginUseCase::_hashPassword (
    String password) [inline], [private]
```

Genera un hash SHA-256 de la contraseña concatenada con un salt estático.

El salt reduce la efectividad de ataques por tablas rainbow sobre los hashes almacenados.

3.10.2.2. execute()

```
Future< Usuario?> LoginUseCase::execute (
    List< Usuario > usuarios,
    String username,
    String password) [inline], [private]
```

Ejecuta la lógica de autenticación contra la lista de usuarios proporcionada.

Valida que los campos no estén vacíos, busca el usuario por nombre, verifica que esté activo y compara el hash de la contraseña ingresada con el hash almacenado en el modelo. Lanza una excepción genérica en cualquier caso de fallo para evitar revelar si el error es por usuario inexistente o contraseña incorrecta. Cualquier excepción durante la búsqueda o validación se unifica en un error genérico de credenciales para evitar enumeración de usuarios.

La documentación de esta clase está generada del siguiente archivo:

- `lib/business/usecases/login_usecase.dart`

3.11. Referencia de la clase NuevaOrdenUseCase

Caso de uso que gestiona la creación de órdenes mixtas, combinando items de venta directa (con unidad física) y pedidos (sin unidad asignada).

Métodos privados

- `NuevaOrdenUseCase` ({ required `InventarioRepository` inventarioRepository, required `UnidadRepository` unidadRepository, required `VentaRepository` ventaRepository, required `PedidoRepository` pedidoRepository, required `OrdenRepository` ordenRepository, required `QrUseCase` qrUseCase, }) `Future< ItemOrden > agregarPorQr` ({ required `String` qr, required `List< ItemOrden >` itemsActuales, }) `async`
Agrega un ítem a la orden mediante escaneo de código QR.
- `Future< ItemOrden > agregarManual` ({ required `int` idInventario, required `List< ItemOrden >` itemsActuales, }) `async`
Agrega un ítem a la orden de forma manual mediante el identificador de inventario.
- `Future< void > confirmarMixto` ({ required `List< ItemOrden >` items, required `int` idUsuario, required `String?` nombreCliente, }) `async`
Confirma una orden mixta que puede contener ítems de venta y de pedido.
- `Future< NuevaOrdenResult > _confirmarVenta` ({ required `List< ItemOrden >` items, required `int` idUsuario, required `String?` nombreCliente, required `int` idOrdenOrigen, }) `async`
Procesa los ítems de tipo venta dentro de una orden.
- `Future< NuevaOrdenResult > _confirmarPedido` ({ required `List< ItemOrden >` items, required `int` idUsuario, required `String?` nombreCliente, required `int` idOrdenOrigen, required `int?` idVentaOrigen, }) `async`
Procesa los ítems de tipo pedido dentro de una orden.
- `Future< List< ItemOrden > > _resolverUnidades` (`List< ItemOrden >` items) `async`
Asigna una unidad física a cada ítem que aún no tenga una.
- `double _calcularTotal` (`List< ItemOrden >` items)
Calcula el total de la orden sumando el precio unitario de cada ítem.

Atributos privados

- `final InventarioRepository _inventarioRepository`
- `final UnidadRepository _unidadRepository`
- `final VentaRepository _ventaRepository`
- `final PedidoRepository _pedidoRepository`
- `final OrdenRepository _ordenRepository`
- `final QrUseCase _qrUseCase`

3.11.1. Descripción detallada

Caso de uso que gestiona la creación de órdenes mixtas, combinando ítems de venta directa (con unidad física) y pedidos (sin unidad asignada).

Coordina la validación, resolución de unidades, persistencia de ventas, pedidos y la orden raíz que los agrupa.

3.11.2. Documentación de funciones miembro

3.11.2.1. _calcularTotal()

```
double NuevaOrdenUseCase::_calcularTotal (
    List< ItemOrden > items) [inline], [private]
```

Calcula el total de la orden sumando el precio unitario de cada ítem.

3.11.2.2. _confirmarPedido()

```
Future< NuevaOrdenResult > NuevaOrdenUseCase::_confirmarPedido (
    { required List< ItemOrden > items, required int idUsuario, required String?
nombreCliente, required int idOrdenOrigen, required int? idVentaOrigen, } ) [inline], [private]
```

Procesa los items de tipo pedido dentro de una orden.

Crea el pedido en estado pendiente con sus detalles sin unidad física asignada. Puede vincularse a una venta previa de la misma orden mediante [idVentaOrigen]. La unidad física se asigna posteriormente al registrar la entrega del pedido.

3.11.2.3. _confirmarVenta()

```
Future< NuevaOrdenResult > NuevaOrdenUseCase::_confirmarVenta (
    { required List< ItemOrden > items, required int idUsuario, required String?
nombreCliente, required int idOrdenOrigen, } ) [inline], [private]
```

Procesa los items de tipo venta dentro de una orden.

Resuelve las unidades físicas de los items que no las tengan asignadas, verifica que cada unidad siga disponible, persiste la venta con sus detalles y desactiva las unidades vendidas para retirarlas del stock disponible. Verificación de disponibilidad previa a la escritura: garantiza que ninguna unidad haya sido vendida entre el momento en que se agregó a la orden y la confirmación.

Desactivación de unidades tras la persistencia exitosa de la venta. Una unidad desactivada no puede ser vendida ni asignada a nuevas órdenes.

3.11.2.4. _resolverUnidades()

```
Future< List< ItemOrden > > NuevaOrdenUseCase::_resolverUnidades (
    List< ItemOrden > items) [inline], [private]
```

Asigna una unidad física a cada item que aún no tenga una.

Para items sin unidad resuelta, consulta las unidades activas del inventario y selecciona la primera que no haya sido asignada ya en esta misma operación. Lanza una excepción si no hay unidades disponibles para algún item. Conjunto de IDs de unidades ya asignadas, para evitar asignar la misma unidad a más de un item dentro del mismo proceso de resolución.

3.11.2.5. agregarManual()

```
Future< ItemOrden > NuevaOrdenUseCase::agregarManual (
    { required int idInventario, required List< ItemOrden > itemsActuales, } ) [inline],
[private]
```

Agrega un item a la orden de forma manual mediante el identificador de inventario.

No requiere unidad física; el item se tratará como pedido. Retorna un [ItemOrden] de tipo [TipoItemOrden.pedido] sin unidad asignada.

3.11.2.6. agregarPorQr()

```
NuevaOrdenUseCase({ required InventarioRepository inventarioRepository, required UnidadRepository
unidadRepository, required VentaRepository ventaRepository, required PedidoRepository pedido←
Repository, required OrdenRepository ordenRepository, required QrUseCase qrUseCase, }) Future<
ItemOrden > NuevaOrdenUseCase::agregarPorQr (
    { required String qr, required List< ItemOrden > itemsActuales, } ) [inline],
[private]
```

Agrega un ítem a la orden mediante escaneo de código QR.

Valida que la unidad exista, esté activa y no haya sido agregada previamente. Retorna un [ItemOrden] de tipo [TipoItemOrden.venta] con la unidad física asignada. Regla de negocio: no se permite agregar la misma unidad física más de una vez en la orden.

3.11.2.7. confirmarMixto()

```
Future< void > NuevaOrdenUseCase::confirmarMixto (
    { required List< ItemOrden > items, required int idUsuario, required String?
nombreCliente, } ) [inline], [private]
```

Confirma una orden mixta que puede contener ítems de venta y de pedido.

Crea primero el registro de orden raíz y luego procesa cada grupo por separado. Si existen ítems de venta, se procesan primero y su identificador se vincula al pedido resultante para mantener la trazabilidad entre ambas transacciones.

3.11.3. Documentación de datos miembro

3.11.3.1. _inventarioRepository

```
final InventarioRepository NuevaOrdenUseCase::_inventarioRepository [private]
```

3.11.3.2. _ordenRepository

```
final OrdenRepository NuevaOrdenUseCase::_ordenRepository [private]
```

3.11.3.3. _pedidoRepository

```
final PedidoRepository NuevaOrdenUseCase::_pedidoRepository [private]
```

3.11.3.4. _qrUseCase

```
final QrUseCase NuevaOrdenUseCase::_qrUseCase [private]
```

3.11.3.5. _unidadRepository

```
final UnidadRepository NuevaOrdenUseCase::_unidadRepository [private]
```

3.11.3.6. _ventaRepository

```
final VentaRepository NuevaOrdenUseCase::_ventaRepository [private]
```

La documentación de esta clase está generada del siguiente archivo:

- lib/business/usecases/nueva_orden_usecase.dart

3.12. Referencia de la clase OrdenRepository

Repositorio que gestiona las operaciones de acceso a datos para la tabla de órdenes.

Métodos privados

- Future< int > **insertar** (Orden orden) **async**
Inserta una nueva orden en la base de datos y retorna su identificador generado.
- Future< Orden?> **obtenerPorId** (int id) **async**
Retorna la orden con el identificador indicado.
- Future< List< Orden > > **obtenerTodas** () **async**
Retorna todas las órdenes registradas, ordenadas por fecha descendente.

Atributos privados

- Future< Database > get_db **async** => await DatabaseHelper.instance.database

3.12.1. Descripción detallada

Repositorio que gestiona las operaciones de acceso a datos para la tabla de órdenes.

3.12.2. Documentación de funciones miembro

3.12.2.1. insertar()

```
Future< int > OrdenRepository::insertar (  
    Orden orden) [inline], [private]
```

Inserta una nueva orden en la base de datos y retorna su identificador generado.

3.12.2.2. obtenerPorId()

```
Future< Orden?> OrdenRepository::obtenerPorId (  
    int id) [inline], [private]
```

Retorna la orden con el identificador indicado.

Retorna null si no existe ninguna orden con ese id.

3.12.2.3. obtenerTodas()

```
Future< List< Orden > > OrdenRepository::obtenerTodas () [inline], [private]
```

Retorna todas las órdenes registradas, ordenadas por fecha descendente.

3.12.3. Documentación de datos miembro

3.12.3.1. async

```
Future<Database> get _db OrdenRepository::async => await DatabaseHelper.instance.database [private]
```

La documentación de esta clase está generada del siguiente archivo:

- lib/data/repositories/[orden_repository.dart](#)

3.13. Referencia de la clase PedidoPendienteUseCase

Caso de uso que gestiona el ciclo de vida de los pedidos en estado pendiente.

Métodos privados

- [PedidoPendienteUseCase](#)({ required [PedidoRepository](#) pedidoRepository, required [UnidadRepository](#) unidadRepository, required [VentaRepository](#) ventaRepository, required [QrUseCase](#) qrUseCase, }) Future< List< Pedido > > [obtenerPedidosPendientes](#) () async
Retorna todos los pedidos cuyo estado sea [EstadoPedido.pendiente].
- Future< List< Map< [String](#), dynamic > > > [obtenerDetallesConInfo](#) (int idPedido,) async
Retorna los detalles del pedido enriquecidos con información de inventario, prenda, escuela y talla para su presentación en pantalla.
- Future< List< DetallePedido > > [obtenerDetalles](#) (int idPedido) async
Retorna los detalles del pedido como modelos [DetallePedido].
- Future< void > [registrarUnidad](#) ({ required int idDetallePedido, required int idUnidad, }) async
Asocia una unidad física a un detalle de pedido específico.
- Future< Map< [String](#), dynamic >? > [registrarQrPedido](#) ({ required int idPedido, required int idDetallePedido, required int idInventarioEsperado, required [String](#) qr, bool forzarMovimiento=false, }) async
Registra una unidad física en un detalle de pedido mediante escaneo QR.
- Future< void > [desregistrarUnidad](#) (int idDetallePedido) async
Elimina la asociación de una unidad física a un detalle de pedido, dejando el detalle sin unidad registrada.
- Future< bool > [pedidoCompleto](#) (int idPedido) async
Retorna true si todos los detalles del pedido tienen una unidad física registrada.
- Future< void > [completarPedido](#) (int idPedido) async
Completa un pedido pendiente convirtiéndolo en venta.
- Future< bool > [eliminarDetallePedido](#) ({ required int idPedido, required int idDetallePedido, }) async
Elimina un detalle del pedido y, si era el último detalle restante, elimina también el pedido completo.

Atributos privados

- final [PedidoRepository](#) _pedidoRepository
- final [UnidadRepository](#) _unidadRepository
- final [VentaRepository](#) _ventaRepository
- final [QrUseCase](#) _qrUseCase

3.13.1. Descripción detallada

Caso de uso que gestiona el ciclo de vida de los pedidos en estado pendiente.

Cubre la consulta, el registro de unidades físicas por QR, la validación de completitud y la conversión del pedido a venta al completarse.

3.13.2. Documentación de funciones miembro

3.13.2.1. completarPedido()

```
Future< void > PedidoPendienteUseCase::completarPedido (
    int idPedido) [inline], [private]
```

Completa un pedido pendiente convirtiéndolo en venta.

Verifica que el pedido exista y que todos sus detalles tengan unidad registrada. Desactiva cada unidad entregada para retirarla del stock disponible. Si el pedido proviene de una orden con venta existente, agrega los detalles a esa venta. Si no tiene venta de origen, crea una nueva venta completada. Finalmente actualiza el estado del pedido a [EstadoPedido.completado]. Si el pedido no tiene una venta de origen, se genera una nueva venta completada con los detalles de las unidades entregadas.

Si ya existe una venta de origen, los detalles se agregan a ella para mantener la trazabilidad de la orden completa.

3.13.2.2. desregistrarUnidad()

```
Future< void > PedidoPendienteUseCase::desregistrarUnidad (
    int idDetallePedido) [inline], [private]
```

Elimina la asociación de una unidad física a un detalle de pedido, dejando el detalle sin unidad registrada.

3.13.2.3. eliminarDetallePedido()

```
Future< bool > PedidoPendienteUseCase::eliminarDetallePedido (
    { required int idPedido, required int idDetallePedido, } ) [inline], [private]
```

Elimina un detalle del pedido y, si era el último detalle restante, elimina también el pedido completo.

Retorna true si el pedido fue eliminado por quedar sin detalles, false en caso contrario. Regla de negocio: un pedido sin detalles no tiene razón de existir y se elimina automáticamente.

3.13.2.4. obtenerDetalles()

```
Future< List< DetallePedido > > PedidoPendienteUseCase::obtenerDetalles (
    int idPedido) [inline], [private]
```

Retorna los detalles del pedido como modelos [DetallePedido].

3.13.2.5. obtenerDetallesConInfo()

```
Future< List< Map< String, dynamic > > > PedidoPendienteUseCase::obtenerDetallesConInfo (
    int idPedido) [inline], [private]
```

Retorna los detalles del pedido enriquecidos con información de inventario, prenda, escuela y talla para su presentación en pantalla.

3.13.2.6. obtenerPedidosPendientes()

```
PedidoPendienteUseCase({ required PedidoRepository pedidoRepository, required UnidadRepository
unidadRepository, required VentaRepository ventaRepository, required QrUseCase qrUseCase,
}) Future< List< Pedido > > PedidoPendienteUseCase::obtenerPedidosPendientes () [inline],
[private]
```

Retorna todos los pedidos cuyo estado sea [EstadoPedido.pendiente].

3.13.2.7. pedidoCompleto()

```
Future< bool > PedidoPendienteUseCase::pedidoCompleto (
    int idPedido) [inline], [private]
```

Retorna true si todos los detalles del pedido tienen una unidad física registrada.

3.13.2.8. registrarQrPedido()

```
Future< Map< String, dynamic >?> PedidoPendienteUseCase::registrarQrPedido (
    { required int idPedido, required int idDetallePedido, required int idInventario←
Esperado, required String qr, bool forzarMovimiento=false, } ) [inline], [private]
```

Registra una unidad física en un detalle de pedido mediante escaneo QR.

Valida que la unidad exista, esté activa y corresponda al inventario esperado. Si la unidad ya está asignada a otro detalle del mismo u otro pedido, retorna un mapa con los datos del conflicto para que la capa de presentación solicite confirmación al usuario antes de forzar el movimiento. Si [forzarMovimiento] es true, desregistra la asignación anterior y registra la unidad en el detalle indicado sin solicitar confirmación adicional. Retorna null si la operación se completó sin conflictos. Regla de negocio: la unidad escaneada debe pertenecer al mismo inventario que el detalle del pedido que se intenta registrar.

Si la unidad ya está asignada al mismo detalle, no se realiza ninguna acción.

Si la unidad está asignada a un detalle diferente y no se fuerza el movimiento, se retorna la información del conflicto para que el usuario decida.

3.13.2.9. registrarUnidad()

```
Future< void > PedidoPendienteUseCase::registrarUnidad (
    { required int idDetallePedido, required int idUnidad, } ) [inline], [private]
```

Asocia una unidad física a un detalle de pedido específico.

3.13.3. Documentación de datos miembro

3.13.3.1. _pedidoRepository

```
final PedidoRepository PedidoPendienteUseCase::_pedidoRepository [private]
```

3.13.3.2. _qrUseCase

```
final QrUseCase PedidoPendienteUseCase::_qrUseCase [private]
```

3.13.3.3. _unidadRepository

```
final UnidadRepository PedidoPendienteUseCase::_unidadRepository [private]
```

3.13.3.4. _ventaRepository

```
final VentaRepository PedidoPendienteUseCase::_ventaRepository [private]
```

La documentación de esta clase está generada del siguiente archivo:

- [lib/business/usecases/pedidos_pendientes_usecase.dart](#)

3.14. Referencia de la clase PedidoRepository

Repositorio encargado de gestionar las operaciones relacionadas con pedidos y detalles de pedido en la base de datos.

Métodos privados

- `Future< int > insertarPedidoYDetalles ({ required Pedido pedido, required List< DetallePedido > detalles, }) async`
Inserta un pedido junto con todos sus detalles asociados dentro de una transacción atómica.
- `Future< List< Pedido > > obtenerTodos () async`
Obtiene todos los pedidos registrados ordenados por fecha descendente.
- `Future< Pedido? > obtenerPorId (int idPedido) async`
Obtiene un pedido específico mediante su identificador.
- `Future< List< DetallePedido > > obtenerDetalles (int idPedido) async`
Obtiene todos los detalles asociados a un pedido.
- `Future< List< Map< String, dynamic > > > obtenerDetallesConInfo (int idPedido,) async`
Obtiene los detalles de un pedido junto con información descriptiva proveniente de tablas relacionadas.
- `Future< int > actualizarEstado ({ required int idPedido, required EstadoPedido estado, }) async`
Actualiza el estado actual de un pedido.
- `Future< int > registrarUnidad ({ required int idDetallePedido, required int idUnidad, }) async`
Registra una unidad física dentro de un detalle de pedido.
- `Future< int > desregistrarUnidad (int idDetallePedido) async`
Elimina el registro de una unidad asociada a un detalle de pedido.
- `Future< Map< String, dynamic >? > obtenerRegistroActivoPorUnidad (int idUnidad,) async`
Obtiene el registro activo asociado a una unidad específica.
- `Future< bool > pedidoCompleto (int idPedido) async`
Verifica si todos los detalles de un pedido han sido registrados.
- `Future< int > eliminarDetallePedido (int idDetallePedido) async`
Elimina un detalle específico de pedido.
- `Future< int > contarDetalles (int idPedido) async`
Obtiene la cantidad total de detalles asociados a un pedido.
- `Future< void > eliminarPedidoCompleto (int idPedido) async`
Elimina un pedido y todos sus detalles asociados dentro de una transacción.
- `Future< int > eliminar (int idPedido) async`
Elimina un pedido mediante su identificador.

Atributos privados

- `Future< Database > get\_db async => await DatabaseHelper.instance.database`
Obtiene una instancia activa de la base de datos.

3.14.1. Descripción detallada

Repositorio encargado de gestionar las operaciones relacionadas con pedidos y detalles de pedido en la base de datos.

3.14.2. Documentación de funciones miembro

3.14.2.1. actualizarEstado()

```
Future< int > PedidoRepository::actualizarEstado (
    { required int idPedido, required EstadoPedido estado, } ) [inline], [private]
```

Actualiza el estado actual de un pedido.

3.14.2.2. contarDetalles()

```
Future< int > PedidoRepository::contarDetalles (
    int idPedido) [inline], [private]
```

Obtiene la cantidad total de detalles asociados a un pedido.

3.14.2.3. desregistrarUnidad()

```
Future< int > PedidoRepository::desregistrarUnidad (
    int idDetallePedido) [inline], [private]
```

Elimina el registro de una unidad asociada a un detalle de pedido.

La operación restablece el estado de registro a pendiente.

3.14.2.4. eliminar()

```
Future< int > PedidoRepository::eliminar (
    int idPedido) [inline], [private]
```

Elimina un pedido mediante su identificador.

3.14.2.5. eliminarDetallePedido()

```
Future< int > PedidoRepository::eliminarDetallePedido (
    int idDetallePedido) [inline], [private]
```

Elimina un detalle específico de pedido.

3.14.2.6. eliminarPedidoCompleto()

```
Future< void > PedidoRepository::eliminarPedidoCompleto (
    int idPedido) [inline], [private]
```

Elimina un pedido y todos sus detalles asociados dentro de una transacción.

La eliminación respeta la integridad lógica entre tablas relacionadas.

3.14.2.7. insertarPedidoYDetalles()

```
Future< int > PedidoRepository::insertarPedidoYDetalles (
    { required Pedido pedido, required List< DetallePedido > detalles, } ) [inline],
[private]
```

Inserta un pedido junto con todos sus detalles asociados dentro de una transacción atómica.

La operación garantiza consistencia entre la tabla de pedidos y la tabla de detalle_pedido.

3.14.2.8. obtenerDetalles()

```
Future< List< DetallePedido > > PedidoRepository::obtenerDetalles (
    int idPedido) [inline], [private]
```

Obtiene todos los detalles asociados a un pedido.

3.14.2.9. obtenerDetallesConInfo()

```
Future< List< Map< String, dynamic > > > PedidoRepository::obtenerDetallesConInfo (
    int idPedido) [inline], [private]
```

Obtiene los detalles de un pedido junto con información descriptiva proveniente de tablas relacionadas.

La consulta integra información de escuela, prenda y talla asociadas al inventario.

3.14.2.10. obtenerPorId()

```
Future< Pedido?> PedidoRepository::obtenerPorId (
    int idPedido) [inline], [private]
```

Obtiene un pedido específico mediante su identificador.

Retorna null si no existe un registro asociado.

3.14.2.11. obtenerRegistroActivoPorUnidad()

```
Future< Map< String, dynamic >?> PedidoRepository::obtenerRegistroActivoPorUnidad (
    int idUnidad) [inline], [private]
```

Obtiene el registro activo asociado a una unidad específica.

Se considera activo cuando el detalle se encuentra marcado como registrado.

3.14.2.12. obtenerTodos()

```
Future< List< Pedido > > PedidoRepository::obtenerTodos () [inline], [private]
```

Obtiene todos los pedidos registrados ordenados por fecha descendente.

3.14.2.13. pedidoCompleto()

```
Future< bool > PedidoRepository::pedidoCompleto (
    int idPedido) [inline], [private]
```

Verifica si todos los detalles de un pedido han sido registrados.

Un pedido se considera completo cuando no existen registros pendientes.

3.14.2.14. registrarUnidad()

```
Future< int > PedidoRepository::registrarUnidad (
    { required int idDetallePedido, required int idUnidad, } ) [inline], [private]
```

Registra una unidad física dentro de un detalle de pedido.

La operación marca el detalle como registrado y asocia la unidad correspondiente.

3.14.3. Documentación de datos miembro**3.14.3.1. async**

```
Future<Database> get _db PedidoRepository::async => await DatabaseHelper.instance.database
[private]
```

Obtiene una instancia activa de la base de datos.

La documentación de esta clase está generada del siguiente archivo:

- [lib/data/repositories/pedido_repository.dart](#)

3.15. Referencia de la clase PrendaRepository

Repositorio encargado de gestionar las operaciones CRUD relacionadas con la entidad Prenda.

Métodos privados

- `Future< int > insertar (Prenda prenda) async`
Inserta una nueva prenda en la base de datos.
- `Future< Prenda?> obtenerPorId (int id) async`
Obtiene una prenda mediante su identificador.
- `Future< List< Prenda > > obtenerTodas () async`
Obtiene todas las prendas registradas ordenadas alfabéticamente por nombre.
- `Future< int > actualizar (Prenda prenda) async`
Actualiza la información de una prenda existente.
- `Future< int > eliminar (int id) async`
Elimina una prenda mediante su identificador.

Atributos privados

- `Future< Database > get _db async => await DatabaseHelper.instance.database`
Obtiene una instancia activa de la base de datos.

3.15.1. Descripción detallada

Repositorio encargado de gestionar las operaciones CRUD relacionadas con la entidad Prenda.

3.15.2. Documentación de funciones miembro

3.15.2.1. actualizar()

```
Future< int > PrendaRepository::actualizar (  
    Prenda prenda) [inline], [private]
```

Actualiza la información de una prenda existente.

La actualización se realiza utilizando el identificador primario de la entidad.

3.15.2.2. eliminar()

```
Future< int > PrendaRepository::eliminar (  
    int id) [inline], [private]
```

Elimina una prenda mediante su identificador.

3.15.2.3. insertar()

```
Future< int > PrendaRepository::insertar (  
    Prenda prenda) [inline], [private]
```

Inserta una nueva prenda en la base de datos.

El identificador primario es eliminado del mapa antes de la inserción para permitir el autoincremento.

La operación utiliza conflicto tipo abort para evitar inserciones duplicadas o inválidas.

3.15.2.4. obtenerPorId()

```
Future< Prenda?> PrendaRepository::obtenerPorId (  
    int id) [inline], [private]
```

Obtiene una prenda mediante su identificador.

Retorna null cuando no existe un registro asociado.

3.15.2.5. obtenerTodas()

```
Future< List< Prenda > > PrendaRepository::obtenerTodas () [inline], [private]
```

Obtiene todas las prendas registradas ordenadas alfabéticamente por nombre.

3.15.3. Documentación de datos miembro

3.15.3.1. async

```
Future<Database> get _db PrendaRepository::async => await DatabaseHelper.instance.database [private]
```

Obtiene una instancia activa de la base de datos.

La documentación de esta clase está generada del siguiente archivo:

- lib/data/repositories/[prenda_repository.dart](#)

3.16. Referencia de la clase PrintService

Contrato base para servicios de impresión.

Métodos privados

- Future< void > [conectar](#) ()
- Future< void > [desconectar](#) ()
- Future< void > [imprimirQR](#) (String id)

3.16.1. Descripción detallada

Contrato base para servicios de impresión.

Define las operaciones mínimas requeridas para gestionar la conexión con una impresora y ejecutar impresiones.

3.16.2. Documentación de funciones miembro

3.16.2.1. conectar()

```
Future< void > PrintService::conectar () [private]
```

3.16.2.2. desconectar()

```
Future< void > PrintService::desconectar () [private]
```

3.16.2.3. imprimirQR()

```
Future< void > PrintService::imprimirQR (  
    String id) [private]
```

La documentación de esta clase está generada del siguiente archivo:

- lib/business/services/[print_service.dart](#)

3.17. Referencia de la clase PrintUseCase

Caso de uso que gestiona la impresión de etiquetas QR para unidades de inventario.

Métodos privados

- `PrintUseCase` ({required this.printer, required this.repo})
- `Future< String > ejecutar` (int `idInventario`, int cantidad) async
Inserta e imprime la cantidad indicada de unidades para el inventario dado.
- `Future< void > imprimirQrExistente` (int `idUnidad`) async
Reimprime la etiqueta QR de una unidad existente sin crear un nuevo registro.

Atributos privados

- final `PrintService printer`
Servicio de impresión utilizado para conectar y enviar comandos a la impresora.
- final `UnidadRepository repo`
Repositorio de unidades para insertar y eliminar registros durante el proceso.

3.17.1. Descripción detallada

Caso de uso que gestiona la impresión de etiquetas QR para unidades de inventario.

Coordina la creación de unidades en el repositorio, el envío a la impresora y el rollback de unidades que no pudieron imprimirse.

3.17.2. Documentación de constructores y destructores

3.17.2.1. PrintUseCase()

```
PrintUseCase::PrintUseCase (
    {required this.printer, required this.repo} ) [private]
```

3.17.3. Documentación de funciones miembro

3.17.3.1. ejecutar()

```
Future< String > PrintUseCase::ejecutar (
    int idInventario,
    int cantidad) [inline], [private]
```

Inserta e imprime la cantidad indicada de unidades para el inventario dado.

Establece una única conexión BLE para todas las impresiones del lote. Por cada unidad, inserta el registro en el repositorio antes de imprimir. Si la impresión de una unidad falla, elimina el registro recién insertado para mantener la consistencia entre las unidades físicas y los registros del sistema, y detiene el proceso retornando un resumen parcial de la operación. La desconexión de la impresora se ejecuta siempre en el bloque finally. Rollback de la unidad insertada si la impresión falla, para evitar registros sin etiqueta física generada.

3.17.3.2. imprimirQrExistente()

```
Future< void > PrintUseCase::imprimirQrExistente (
    int idUnidad) [inline], [private]
```

Reimprime la etiqueta QR de una unidad existente sin crear un nuevo registro.

Útil para reponer etiquetas dañadas o perdidas de unidades ya registradas en el sistema. La conexión se establece con un timeout de 15 segundos. La desconexión se ejecuta siempre en el bloque finally.

3.17.4. Documentación de datos miembro

3.17.4.1. printer

```
final PrintService PrintUseCase::printer [private]
```

Servicio de impresión utilizado para conectar y enviar comandos a la impresora.

3.17.4.2. repo

```
final UnidadRepository PrintUseCase::repo [private]
```

Repositorio de unidades para insertar y eliminar registros durante el proceso.

La documentación de esta clase está generada del siguiente archivo:

- `lib/business/usecases/print_usecase.dart`

3.18. Referencia de la clase QrUseCase

Caso de uso que gestiona la interpretación y validación de códigos QR para la identificación de unidades físicas de inventario.

Métodos privados

- `QrUseCase` (`this.unidadRepository`)
- `Future< Unidad? > obtenerUnidad (String qr)` `async`
Extrae el identificador numérico del contenido del QR y retorna la unidad correspondiente si existe y está activa.
- `Future< bool > existeQr (String qr)` `async`
Verifica si el contenido del QR corresponde a una unidad registrada en el sistema, independientemente de si está activa o no.

Atributos privados

- final `UnidadRepository unidadRepository`

3.18.1. Descripción detallada

Caso de uso que gestiona la interpretación y validación de códigos QR para la identificación de unidades físicas de inventario.

3.18.2. Documentación de constructores y destructores

3.18.2.1. QrUseCase()

```
QrUseCase::QrUseCase (  
    this. unidadRepository) [private]
```

3.18.3. Documentación de funciones miembro

3.18.3.1. existeQr()

```
Future< bool > QrUseCase::existeQr (  
    String qr) [inline], [private]
```

Verifica si el contenido del QR corresponde a una unidad registrada en el sistema, independientemente de si está activa o no.

Retorna false si el contenido no es un número entero válido.

3.18.3.2. obtenerUnidad()

```
Future< Unidad?> QrUseCase::obtenerUnidad (  
    String qr) [inline], [private]
```

Extrae el identificador numérico del contenido del QR y retorna la unidad correspondiente si existe y está activa.

Utiliza una expresión regular para obtener la primera secuencia de dígitos del QR, lo que permite tolerar formatos con prefijos o sufijos de texto. Retorna null si el QR no contiene un número válido, si la unidad no existe o si está inactiva.

3.18.4. Documentación de datos miembro

3.18.4.1. unidadRepository

```
final UnidadRepository QrUseCase::unidadRepository [private]
```

La documentación de esta clase está generada del siguiente archivo:

- `lib/business/usecases/qr\_usecase.dart`

3.19. Referencia de la clase RegistrarInventarioUseCase

Caso de uso que gestiona el registro de items de inventario para una combinación de prenda, escuela y una o más tallas.

Métodos privados

- `RegistrarInventarioUseCase` ({ required `InventarioRepository` inventarioRepository, required `PrendaRepository` prendaRepository, required `TallaRepository` tallaRepository, required `EscuelaRepository` escuelaRepository, }) `Future< RegistrarInventarioResult > ejecutar` ({ required int idPrenda, required double `precio`, required List< int > idsTallas, required int idEscuela, }) `async`

Registra un item de inventario por cada talla seleccionada para la combinación de prenda y escuela indicadas.

Atributos privados

- final `InventarioRepository` `_inventarioRepository`
- final `PrendaRepository` `_prendaRepository`
- final `TallaRepository` `_tallaRepository`
- final `EscuelaRepository` `_escuelaRepository`

3.19.1. Descripción detallada

Caso de uso que gestiona el registro de items de inventario para una combinación de prenda, escuela y una o más tallas.

Valida la existencia de los catálogos involucrados y aplica la regla de unicidad por combinación antes de insertar cada registro.

3.19.2. Documentación de funciones miembro

3.19.2.1. ejecutar()

```
RegistrarInventarioUseCase({ required InventarioRepository inventarioRepository, required
PrendaRepository prendaRepository, required TallaRepository tallaRepository, required EscuelaRepository
escuelaRepository, }) Future< RegistrarInventarioResult > RegistrarInventarioUseCase::ejecutar
(
    { required int idPrenda, required double precio, required List< int > idsTallas,
required int idEscuela, } ) [inline], [private]
```

Registra un item de inventario por cada talla seleccionada para la combinación de prenda y escuela indicadas.

Valida que el precio sea mayor a cero, que se haya seleccionado al menos una talla, y que tanto la prenda como la escuela existan en el sistema. Por cada talla, verifica que no exista ya una combinación registrada antes de intentar la inserción. Los resultados individuales se acumulan y se retornan en un `[RegistrarInventarioResult]` con el resumen de la operación. Carga previa de todas las tallas requeridas para evitar consultas repetidas al repositorio dentro del ciclo de registro.

Regla de negocio: no se permite registrar más de un item de inventario para la misma combinación de escuela, prenda y talla.

3.19.3. Documentación de datos miembro

3.19.3.1. `_escuelaRepository`

```
final EscuelaRepository RegistrarInventarioUseCase::_escuelaRepository [private]
```

3.19.3.2. `_inventarioRepository`

```
final InventarioRepository RegistrarInventarioUseCase::_inventarioRepository [private]
```

3.19.3.3. `_prendaRepository`

```
final PrendaRepository RegistrarInventarioUseCase::_prendaRepository [private]
```

3.19.3.4. `_tallaRepository`

```
final TallaRepository RegistrarInventarioUseCase::_tallaRepository [private]
```

La documentación de esta clase está generada del siguiente archivo:

- `lib/business/usecases/regarstrar_inventario_usecase.dart`

3.20. Referencia de la clase `RegistroTallaResult`

Resultado del intento de registro de inventario para una talla específica.

Métodos privados

- `const RegistroTallaResult` ({ required this.idTalla, required this.nombreTalla, required this.exitoso, this.↵ mensajeError, })

Atributos privados

- final int `idTalla`
- final `String` `nombreTalla`
- final bool `exitoso`
- final `String` `mensajeError`

Mensaje descriptivo del error ocurrido. Null si el registro fue exitoso.

3.20.1. Descripción detallada

Resultado del intento de registro de inventario para una talla específica.

Indica si la operación fue exitosa e incluye el mensaje de error en caso de fallo.

3.20.2. Documentación de constructores y destructores

3.20.2.1. RegistroTallaResult()

```
const RegistroTallaResult::RegistroTallaResult (
    { required this.idTalla, required this.nombreTalla, required this.exitoso, this.↵
mensajeError, } ) [private]
```

3.20.3. Documentación de datos miembro

3.20.3.1. exitoso

```
final bool RegistroTallaResult::exitoso [private]
```

3.20.3.2. idTalla

```
final int RegistroTallaResult::idTalla [private]
```

3.20.3.3. mensajeError

```
final String RegistroTallaResult::mensajeError [private]
```

Mensaje descriptivo del error ocurrido. Null si el registro fue exitoso.

3.20.3.4. nombreTalla

```
final String RegistroTallaResult::nombreTalla [private]
```

La documentación de esta clase está generada del siguiente archivo:

- `lib/business/usecases/registro_inventario_usecase.dart`

3.21. Referencia de la clase ReporteRepository

Repositorio encargado de generar consultas estadísticas y métricas utilizadas en reportes.

Métodos privados

- `Future< double > obtenerTotalVendido` ({ required int year, int? month, int? idEscuela, }) `async`
Obtiene el total monetario vendido filtrando por año, mes y escuela de forma opcional.
- `Future< int > obtenerCantidadVentas` ({ required int year, int? month, int? idEscuela, }) `async`
Obtiene la cantidad de ventas realizadas filtrando por año, mes y escuela opcionalmente.
- `Future< List< Map< String, dynamic > > > obtenerPrendasMasVendidas` ({ required int year, int? month, int? idEscuela, }) `async`
Obtiene las prendas con mayor volumen de ventas.
- `Future< List< Map< String, dynamic > > > obtenerEscuelasMasVentas` ({ required int year, int? month, }) `async`
Obtiene las escuelas con mayores ingresos generados.

Atributos privados

- `Future< Database > get_db async => await DatabaseHelper.instance.database`

Obtiene una instancia activa de la base de datos.

3.21.1. Descripción detallada

Repositorio encargado de generar consultas estadísticas y métricas utilizadas en reportes.

Centraliza operaciones agregadas relacionadas con ventas, ingresos, prendas y escuelas.

3.21.2. Documentación de funciones miembro

3.21.2.1. obtenerCantidadVentas()

```
Future< int > ReporteRepository::obtenerCantidadVentas (
    { required int year, int? month, int? idEscuela, } ) [inline], [private]
```

Obtiene la cantidad de ventas realizadas filtrando por año, mes y escuela opcionalmente.

Las ventas canceladas son excluidas del conteo.

3.21.2.2. obtenerEscuelasMasVentas()

```
Future< List< Map< String, dynamic > > > ReporteRepository::obtenerEscuelasMasVentas (
    { required int year, int? month, } ) [inline], [private]
```

Obtiene las escuelas con mayores ingresos generados.

La consulta retorna:

- identificador de escuela,
- nombre,
- cantidad de registros vendidos,
- total monetario generado.

El resultado se limita a las 10 escuelas con mayor ingreso acumulado.

3.21.2.3. obtenerPrendasMasVendidas()

```
Future< List< Map< String, dynamic > > > ReporteRepository::obtenerPrendasMasVendidas (
    { required int year, int? month, int? idEscuela, } ) [inline], [private]
```

Obtiene las prendas con mayor volumen de ventas.

La consulta retorna:

- nombre de la prenda,
- cantidad de piezas vendidas,
- ingreso total generado.

El resultado se limita a las 10 prendas con mayor cantidad vendida.

3.21.2.4. obtenerTotalVendido()

```
Future< double > ReporteRepository::obtenerTotalVendido (
    { required int year, int? month, int? idEscuela, } ) [inline], [private]
```

Obtiene el total monetario vendido filtrando por año, mes y escuela de forma opcional.

Las ventas canceladas son excluidas del cálculo.

Cuando se especifica una escuela, el cálculo se realiza utilizando joins sobre detalle_venta, unidades e inventario para identificar únicamente productos asociados.

3.21.3. Documentación de datos miembro

3.21.3.1. async

```
Future<Database> get _db ReporteRepository::async => await DatabaseHelper.instance.database
[private]
```

Obtiene una instancia activa de la base de datos.

La documentación de esta clase está generada del siguiente archivo:

- [lib/data/repositories/reporte_repository.dart](#)

3.22. Referencia de la clase StatefulWidget

Pantalla para modificar la cantidad de unidades de un inventario específico.

Métodos privados

- const [AdministrarCantidadScreen](#) ({super.key, required this.idInventario})
- override State< [AdministrarCantidadScreen](#) > createState ()
- const [AdministrarPrendaScreen](#) ({ super.key, required this.idInventario, required this.nombreEscuela, required this.nombrePrenda, required this.talla, required this.cantidad, })
- override State< [AdministrarPrendaScreen](#) > createState ()
- const [AgregarPrendasPedidoScreen](#) ({super.key})
- override State< [AgregarPrendasPedidoScreen](#) > createState ()
- const [ExitoScreen](#) ({super.key})
- *Constructor principal de la clase [ExitoScreen].*
- override State< [ExitoScreen](#) > createState ()
- const [LoginScreen](#) ({super.key})
- override State< [LoginScreen](#) > createState ()
- const [AgregarPedidoScreen](#) ({ super.key, })
- override State< [AgregarPedidoScreen](#) > createState ()
- const [PedidosPendientesScreen](#) ({ super.key, })
- *Constructor por defecto de la pantalla de pedidos pendientes.*
- override State< [PedidosPendientesScreen](#) > createState ()
- final Future< [ScanFeedback](#) > Function (String qr) onScan

Callback que cada pantalla define para procesar el QR.

- `const QRScannerScreen ({ super.key, required this.onScan, this.mostrarMensaje=true, })`
- `override State< QRScannerScreen > createState ()`
- `const RegistrarPrendaScreen ({ super.key, required this.registrarInventarioUseCase, })`

Constructor que recibe el caso de uso responsable de registrar el inventario.

- `override State< RegistrarPrendaScreen > createState ()`
- `const RevisarPedidoScreen ({ super.key, required this.idPedido, })`

Constructor principal de la pantalla de revisión de pedidos.

- `override State< RevisarPedidoScreen > createState ()`
- `const RevisarVentaScreen ({ super.key, required this.idVenta, })`
- `override State< RevisarVentaScreen > createState ()`
- `const VentasScreen ({ super.key, })`
- `override State< VentasScreen > createState ()`

Atributos privados

- `final int idInventario`
Identificador único del inventario a administrar.
- `final String nombreEscuela`
Nombre de la escuela asociada al inventario.
- `final String nombrePrenda`
Nombre de la prenda asociada al inventario.
- `final String talla`
Talla de la prenda.
- `final int cantidad`
Cantidad inicial de unidades disponibles al abrir la pantalla.
- `final bool mostrarMensaje`
Indica si se debe mostrar el mensaje de feedback en pantalla.
- `final RegistrarInventarioUseCase registrarInventarioUseCase`
- `final int idPedido`
Identificador único del pedido que será consultado.
- `final int idVenta`
Identificador de la venta a revisar.

3.22.1. Descripción detallada

Pantalla para modificar la cantidad de unidades de un inventario específico.

Pantalla que lista las ventas registradas y permite filtrarlas por año y mes.

Pantalla para revisar los detalles de una venta.

Pantalla encargada de mostrar el detalle de un pedido pendiente, permitiendo registrar unidades mediante códigos QR, eliminar prendas del pedido y completar el pedido cuando todas las unidades han sido registradas.

Pantalla para registrar una nueva prenda en el inventario.

Pantalla que implementa un escáner de códigos QR usando `mobile_scanner`.

Pantalla que muestra la lista de pedidos pendientes.

Pantalla para crear una nueva orden o venta.

Pantalla de inicio de sesión con animaciones de entrada y validación.

Pantalla de confirmación visual que indica el éxito de una operación.

Pantalla para agregar prendas a un pedido existente.

Pantalla para visualizar y modificar los datos de una prenda de inventario.

Expone dos operaciones: agregar unidades mediante impresión de QRs, y restar unidades mediante escaneo de QRs físicos. Requiere `[idInventario]` para identificar el inventario sobre el que opera.

Permite editar el precio, consultar la cantidad disponible, navegar a `[AdministrarCantidadScreen]` para modificar unidades, y eliminar el registro del inventario.

Permite filtrar el inventario disponible por escuela y tipo de prenda, y agregar unidades al pedido activo. Nota: los datos de escuelas, prendas e inventario son estáticos en esta versión.

Componente de tipo `[StatefulWidget]` diseñado para notificar de forma efímera al usuario que una transacción o modificación de datos ha culminado exitosamente.

- Implementa un formulario simple con campos de usuario y contraseña.
- Usa `[AuthProvider]` para realizar la autenticación y mostrar estado de carga/errores.
- Incluye animaciones de entrada (fade + slide) controladas por un `[AnimationController]`.
- Permite seleccionar escuela, prenda y talla desde el inventario.
- Soporta agregar ítems manualmente o mediante escaneo de QR.
- Muestra las prendas agregadas agrupadas por inventario y permite confirmar la orden.
- Componente de tipo `[StatefulWidget]` que consulta al proveedor `[PedidosPendientesProvider]` para obtener los pedidos pendientes.
- Permite crear un nuevo pedido y revisar pedidos existentes.
- La recarga de datos se realiza en `initState` mediante el provider.
- Recibe un callback `onScan` que procesa el valor del QR y devuelve un `ScanFeedback`.
- `mostrarMensaje` controla si se muestra un mensaje visual tras el escaneo.
- Presenta un formulario con campos: tipo de prenda, precio, talla y escuela.
- Utiliza un caso de uso `[RegistrarInventarioUseCase]` para ejecutar la operación de registro y muestra retroalimentación al usuario mediante `SnackBars`.
- Muestra los ítems vendidos asociados a una venta específica.
- Permite devolver prendas (reactivar unidad, eliminar detalle de venta), reimprimir QRs y actualizar el inventario y listados relacionados.
- Usa `[VentasProvider]` para obtener y observar el listado de ventas.
- Permite navegar a la vista de revisión de una venta individual.

3.22.2. Documentación de funciones miembro

3.22.2.1. `AdministrarCantidadScreen()`

```
const StatefulWidget::AdministrarCantidadScreen (  
    {super.key, required this.idInventario} ) [private]
```

3.22.2.2. AdministrarPrendaScreen()

```
const StatefulWidget::AdministrarPrendaScreen (  
    { super.key, required this.idInventario, required this.nombreEscuela, required  
    this.nombrePrenda, required this.talla, required this.cantidad, } ) [private]
```

3.22.2.3. AgregarPedidoScreen()

```
const StatefulWidget::AgregarPedidoScreen (  
    { super.key, } ) [private]
```

3.22.2.4. AgregarPrendasPedidoScreen()

```
const StatefulWidget::AgregarPrendasPedidoScreen (  
    {super.key} ) [private]
```

3.22.2.5. createState() [1/12]

```
override State< AdministrarCantidadScreen > StatefulWidget::createState () [private]
```

3.22.2.6. createState() [2/12]

```
override State< AdministrarPrendaScreen > StatefulWidget::createState () [private]
```

3.22.2.7. createState() [3/12]

```
override State< AgregarPrendasPedidoScreen > StatefulWidget::createState () [private]
```

3.22.2.8. createState() [4/12]

```
override State< ExitoScreen > StatefulWidget::createState () [private]
```

3.22.2.9. createState() [5/12]

```
override State< LoginScreen > StatefulWidget::createState () [private]
```

3.22.2.10. createState() [6/12]

```
override State< AgregarPedidoScreen > StatefulWidget::createState () [private]
```

3.22.2.11. createState() [7/12]

```
override State< PedidosPendientesScreen > StatefulWidget::createState () [private]
```

3.22.2.12. createState() [8/12]

```
override State< QRScannerScreen > createState () [private]
```

3.22.2.13. createState() [9/12]

```
override State< RegistrarPrendaScreen > createState () [private]
```

3.22.2.14. createState() [10/12]

```
override State< RevisarPedidoScreen > createState () [private]
```

3.22.2.15. createState() [11/12]

```
override State< RevisarVentaScreen > createState () [private]
```

3.22.2.16. createState() [12/12]

```
override State< VentasScreen > createState () [private]
```

3.22.2.17. ExitScreen()

```
const StatefulWidget::ExitScreen (  
    {super.key} ) [private]
```

Constructor principal de la clase [ExitScreen].

3.22.2.18. Function()

```
final Future< ScanFeedback > StatefulWidget::Function (  
    String qr) [private]
```

Callback que cada pantalla define para procesar el QR.

3.22.2.19. LoginScreen()

```
const StatefulWidget::LoginScreen (  
    {super.key} ) [private]
```

3.22.2.20. PedidosPendientesScreen()

```
const StatefulWidget::PedidosPendientesScreen (  
    { super.key, } ) [private]
```

Constructor por defecto de la pantalla de pedidos pendientes.

3.22.2.21. QRScannerScreen()

```
const StatefulWidget::QRScannerScreen (
    { super.key, required this.onScan, this.mostrarMensaje=true, } ) [private]
```

3.22.2.22. RegistrarPrendaScreen()

```
const StatefulWidget::RegistrarPrendaScreen (
    { super.key, required this.registrarInventarioUseCase, } ) [private]
```

Constructor que recibe el caso de uso responsable de registrar el inventario.

3.22.2.23. RevisarPedidoScreen()

```
const StatefulWidget::RevisarPedidoScreen (
    { super.key, required this.idPedido, } ) [private]
```

Constructor principal de la pantalla de revisión de pedidos.

3.22.2.24. RevisarVentaScreen()

```
const StatefulWidget::RevisarVentaScreen (
    { super.key, required this.idVenta, } ) [private]
```

3.22.2.25. VentasScreen()

```
const StatefulWidget::VentasScreen (
    { super.key, } ) [private]
```

3.22.3. Documentación de datos miembro

3.22.3.1. cantidad

```
final int StatefulWidget::cantidad [private]
```

Cantidad inicial de unidades disponibles al abrir la pantalla.

3.22.3.2. idInventario

```
final int StatefulWidget::idInventario [private]
```

Identificador único del inventario a administrar.

Identificador único del registro de inventario.

3.22.3.3. idPedido

```
final int StatefulWidget::idPedido [private]
```

Identificador único del pedido que será consultado.

3.22.3.4. idVenta

```
final int StatefulWidget::idVenta [private]
```

Identificador de la venta a revisar.

3.22.3.5. mostrarMensaje

```
final bool StatefulWidget::mostrarMensaje [private]
```

Indica si se debe mostrar el mensaje de feedback en pantalla.

3.22.3.6. nombreEscuela

```
final String StatefulWidget::nombreEscuela [private]
```

Nombre de la escuela asociada al inventario.

3.22.3.7. nombrePrenda

```
final String StatefulWidget::nombrePrenda [private]
```

Nombre de la prenda asociada al inventario.

3.22.3.8. registrarInventarioUseCase

```
final RegistrarInventarioUseCase StatefulWidget::registrarInventarioUseCase [private]
```

3.22.3.9. talla

```
final String StatefulWidget::talla [private]
```

Talla de la prenda.

La documentación de esta clase está generada de los siguientes archivos:

- [lib/presentation/screens/administrar_cantidad_screen.dart](#)
- [lib/presentation/screens/administrar_prenda_screen.dart](#)
- [lib/presentation/screens/qr_screen.dart](#)
- [lib/presentation/screens/registrar_prenda_screen.dart](#)
- [lib/presentation/screens/revisar_pedido_screen.dart](#)
- [lib/presentation/screens/revisar_venta_screen.dart](#)
- [lib/presentation/screens/agregar_prendas_pedido_screen.dart](#)
- [lib/presentation/screens/exito_screen.dart](#)
- [lib/presentation/screens/login_screen.dart](#)
- [lib/presentation/screens/nueva_orden_screen.dart](#)
- [lib/presentation/screens/pedidos_pendientes_screen.dart](#)
- [lib/presentation/screens/ventas_screen.dart](#)

3.23. Referencia de la clase StatelessWidget

Widget raíz de la aplicación.

Métodos privados

- const `MyApp` ({ super.key, })
- override Widget `build` (BuildContext context,)
 - Construye la aplicación con sus rutas y configuración global.*
- const `BottomNavBar` ({super.key})
- override Widget `build` (BuildContext context)
 - Construye la barra de navegación inferior.*
- const `HomeScreen` ({ super.key, })
 - Constructor principal de la clase [HomeScreen].*
- override Widget `build` (BuildContext context,)
 - Construye la jerarquía visual delegando la representación a [HomeView].*
- const `_HomeButton` ({ required this.label, required this.icon, required this.onPressed, required this.backgroundColor, required this.textColor, required this.iconColor, })
 - Constructor que obliga a proporcionar todas las propiedades necesarias.*
- override Widget `build` (BuildContext context,)
 - Construye el botón con estilo consistente.*
- const `InventarioScreen` ({ super.key, })
 - Constructor por defecto de la pantalla de inventario.*
- override Widget `build` (BuildContext context,)
 - Construye la interfaz principal de inventario.*
- const `_Campo` ({ required this.controller, required this.hint, required this.icono, this.esPassword=false, this.verPassword=false, this.onTogglePassword, })
 - Constructor del campo reutilizable.*
- override Widget `build` (BuildContext context)
- const `QrPendientesScreen` ({ super.key, })
 - Constructor por defecto.*
- override Widget `build` (BuildContext context,)
 - Construye el árbol de widgets y envuelve la vista en un [ChangeNotifierProvider].*
- const `ReportesScreen` ({super.key})
- override Widget `build` (BuildContext context)
- const `_FiltrosSection` ({required this.provider})
- override Widget `build` (BuildContext context)
- const `_StatCard` ({ required this.label, required this.valor, required this.icono, required this.color, })
- override Widget `build` (BuildContext context)
- const `_RankingCard` ({ required this.items, this.esDinero=false, })
- override Widget `build` (BuildContext context)
- const `AlertasPopup` ({ super.key, })
- override Widget `build` (BuildContext context,)

Atributos privados

- final [String](#) `label`
Texto que se muestra en el botón.
- final [IconData](#) `icon`
Icono que se muestra a la izquierda del texto.
- final [VoidCallback](#) `onPressed`
Callback que se ejecuta al pulsar el botón.
- final [Color](#) `backgroundColor`
Color de fondo del botón.
- final [Color](#) `textColor`
Color del texto del botón.
- final [Color](#) `iconColor`
Color del icono del botón.
- final [TextEditingController](#) `controller`
- final [String](#) `hint`
- final [IconData](#) `icono`
- final [bool](#) `esPassword`
- final [bool](#) `verPassword`
- final [VoidCallback](#) `onTogglePassword`
- final [ReportesProvider](#) `provider`
- final [String](#) `valor`
- final [Color](#) `color`
- final [List](#)<[_RankingItem](#)> `items`
- final [bool](#) `esDinero`

Atributos estáticos privados

- static const [_inputDecoration](#)

3.23.1. Descripción detallada

Widget raíz de la aplicación.

Tarjeta que renderiza una lista de [[_RankingItem](#)] con barra de progreso y cifras.

Tarjeta que muestra una métrica resumida (label, valor e ícono).

Sección de filtros que permite seleccionar año, mes y escuela.

Pantalla principal de reportes.

Pantalla que muestra las unidades cuyo QR está pendiente de impresión.

Widget reutilizable para campos de texto.

Pantalla de gestión de inventario.

Widget privado que representa un botón del menú principal.

Pantalla principal de la aplicación.

Barra de navegación inferior persistente de la aplicación.

Configura [MaterialApp] con el título, las rutas nombradas y la pantalla inicial del sistema.

Muestra los accesos directos a la pantalla principal y al cierre de sesión. Utiliza [SafeArea] para evitar superposición con las barras del sistema operativo.

Componente de tipo [StatelessWidget] que actúa como punto de entrada a la interfaz principal después de la autenticación. Encamina la construcción de la vista a la clase [HomeView].

Este componente encapsula la apariencia y comportamiento de los botones del home (icono, texto, colores y acción). Se usa internamente en este archivo.

Componente de tipo [StatelessWidget] que muestra controles para:

- seleccionar una escuela,
- filtrar por prenda,
- navegar a pantallas de registro y administración de prendas,
- ver QR pendientes. La lógica de negocio relacionada con la carga y filtrado del inventario se delega al proveedor [InventarioProvider].
- Soporta campos normales y de contraseña con toggle de visibilidad.
- Recibe un [TextEditingController] para exponer el valor al estado padre.
- Aplica estilos y bordes consistentes con el diseño de la aplicación.
- Componente de tipo [StatelessWidget] que crea y provee un [QrPendientesProvider].
- El proveedor carga los pendientes al inicializarse mediante `cargarPendientes()`.
- Provee un [ReportesProvider] que encapsula la lógica de obtención y filtrado de datos para los reportes.
- Inicializa la carga de datos mediante `ReportesProvider.inicializar()`.
- Usa valores calculados localmente (últimos 5 años) y listas del provider.
- Al cambiar filtros se delega la acción al provider para recargar datos.
- `esDinero` controla el formato del valor mostrado (moneda o entero).

3.23.2. Documentación de funciones miembro

3.23.2.1. _Campo()

```
const StatelessWidget::_Campo (
    { required this.controller, required this.hint, required this.icono, this.es↵
    Password=false, this.verPassword=false, this.onTogglePassword, } ) [private]
```

Constructor del campo reutilizable.

3.23.2.2. _FiltrosSection()

```
const StatelessWidget::_FiltrosSection (
    {required this.provider} ) [private]
```

3.23.2.3. _HomeButton()

```
const StatelessWidget::_HomeButton (
    { required this.label, required this.icon, required this.onPressed, required
this.backgroundColor, required this.textColor, required this.iconColor, } ) [private]
```

Constructor que obliga a proporcionar todas las propiedades necesarias.

3.23.2.4. _RankingCard()

```
const StatelessWidget::_RankingCard (
    { required this.items, this.esDinero=false, } ) [private]
```

3.23.2.5. _StatCard()

```
const StatelessWidget::_StatCard (
    { required this.label, required this.valor, required this.icono, required this.↵
color, } ) [private]
```

3.23.2.6. AlertasPopup()

```
const StatelessWidget::AlertasPopup (
    { super.key, } ) [private]
```

3.23.2.7. BottomNavBar()

```
const StatelessWidget::BottomNavBar (
    {super.key} ) [private]
```

3.23.2.8. build() [1/12]

```
override Widget StatelessWidget::build (
    BuildContext context) [inline], [private]
```

Construye la barra de navegación inferior.

Compone un contenedor con color de fondo, protección de área segura y dos botones de navegación: inicio y cierre de sesión.

3.23.2.9. build() [2/12]

```
override Widget StatelessWidget::build (
    BuildContext context) [inline], [private]
```

3.23.2.10. build() [3/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

3.23.2.11. build() [4/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

3.23.2.12. build() [5/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

3.23.2.13. build() [6/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

3.23.2.14. build() [7/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

Construye la aplicación con sus rutas y configuración global.

3.23.2.15. build() [8/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

Construye la jerarquía visual delegando la representación a [HomeView].

Este widget actúa como contenedor simple que devuelve la vista real implementada en [HomeView]. No mantiene estado propio.

3.23.2.16. build() [9/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

Construye el botón con estilo consistente.

- Usa `ElevatedButton.icon` para combinar icono y texto.
- Aplica borde cuando el fondo es blanco para mantener contraste.

3.23.2.17. build() [10/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

Construye la interfaz principal de inventario.

- Define un [Scaffold] con [AppBar], cuerpo y barra de navegación inferior.
- El botón de añadir en la AppBar crea un [RegistrarInventarioUseCase] con los repositorios necesarios y navega a [RegistrarPantallaScreen].
- Tras regresar de la pantalla de registro se solicita recargar las escuelas mediante `InventarioProvider.recargarEscuelas()`.

3.23.2.18. build() [11/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

Construye el árbol de widgets y envuelve la vista en un [ChangeNotifierProvider].

- Se inyecta una instancia de [QrPendientesProvider] con su repositorio.
- Se llama a `cargarPendientes()` inmediatamente para iniciar la carga de datos.

3.23.2.19. build() [12/12]

```
override Widget StatelessWidget::build (  
    BuildContext context) [inline], [private]
```

3.23.2.20. HomeScreen()

```
const StatelessWidget::HomeScreen (  
    { super.key, } ) [private]
```

Constructor principal de la clase [HomeScreen].

3.23.2.21. InventarioScreen()

```
const StatelessWidget::InventarioScreen (  
    { super.key, } ) [private]
```

Constructor por defecto de la pantalla de inventario.

3.23.2.22. MyApp()

```
const StatelessWidget::MyApp (  
    { super.key, } ) [private]
```

3.23.2.23. QrPendientesScreen()

```
const StatelessWidget::QrPendientesScreen (  
    { super.key, } ) [private]
```

Constructor por defecto.

3.23.2.24. ReportesScreen()

```
const StatelessWidget::ReportesScreen (  
    {super.key} ) [private]
```

3.23.3. Documentación de datos miembro

3.23.3.1. _inputDecoration

```
const StatelessWidget::_inputDecoration [static], [private]
```

Valor inicial:

```
= InputDecoration(  
    filled: true,  
    fillColor: Colors.white,  
    contentPadding: EdgeInsets.symmetric(horizontal: 12, vertical: 10),  
    border: OutlineInputBorder(  
        borderRadius: BorderRadius.all(Radius.circular(12)),  
        borderSide: BorderSide(color: Color(0xFFE5E5E5)),  
    ),  
    enabledBorder: OutlineInputBorder(  
        borderRadius: BorderRadius.all(Radius.circular(12)),  
        borderSide: BorderSide(color: Color(0xFFE5E5E5)),  
    ),  
    focusedBorder: OutlineInputBorder(  
        borderRadius: BorderRadius.all(Radius.circular(12)),  
        borderSide: BorderSide(color: Color(0xFF1452BD)),  
    ),  
)
```

3.23.3.2. backgroundColor

```
final Color StatelessWidget::backgroundColor [private]
```

Color de fondo del botón.

3.23.3.3. color

```
final Color StatelessWidget::color [private]
```

3.23.3.4. controller

```
final TextEditingController StatelessWidget::controller [private]
```

3.23.3.5. esDinero

```
final bool StatelessWidget::esDinero [private]
```

3.23.3.6. esPassword

```
final bool StatelessWidget::esPassword [private]
```

3.23.3.7. hint

```
final String StatelessWidget::hint [private]
```

3.23.3.8. icon

```
final IconData StatelessWidget::icon [private]
```

Icono que se muestra a la izquierda del texto.

3.23.3.9. iconColor

```
final Color StatelessWidget::iconColor [private]
```

Color del icono del botón.

3.23.3.10. icono

```
final IconData StatelessWidget::icono [private]
```

3.23.3.11. items

```
final List<_RankingItem> StatelessWidget::items [private]
```

3.23.3.12. label

```
final String StatelessWidget::label [private]
```

Texto que se muestra en el botón.

3.23.3.13. onPressed

```
final VoidCallback StatelessWidget::onPressed [private]
```

Callback que se ejecuta al pulsar el botón.

3.23.3.14. onTogglePassword

```
final VoidCallback StatelessWidget::onTogglePassword [private]
```

3.23.3.15. provider

```
final ReportesProvider StatelessWidget::provider [private]
```

3.23.3.16. textColor

```
final Color StatelessWidget::textColor [private]
```

Color del texto del botón.

3.23.3.17. valor

```
final String StatelessWidget::valor [private]
```

3.23.3.18. verPassword

```
final bool StatelessWidget::verPassword [private]
```

La documentación de esta clase está generada de los siguientes archivos:

- lib/[main.dart](#)
- lib/presentation/screens/[home_screen.dart](#)
- lib/presentation/screens/[login_screen.dart](#)
- lib/presentation/screens/[reportes_screen.dart](#)
- lib/presentation/screens/[bottom_nav_bar.dart](#)
- lib/presentation/screens/[inventario_screen.dart](#)
- lib/presentation/screens/[qr_pendientes_screen.dart](#)
- lib/presentation/widgets/[alertas_popup.dart](#)

3.24. Referencia de la clase TallaRepository

Repositorio encargado de gestionar las operaciones CRUD relacionadas con la entidad Talla.

Métodos privados

- Future< int > **insertar** (Talla *talla*) **async**
Inserta una nueva talla en la base de datos.
- Future< Talla?> **obtenerPorId** (int *id*) **async**
Obtiene una talla mediante su identificador.
- Future< List< Talla > > **obtenerTodas** () **async**
Obtiene todas las tallas registradas ordenadas ascendentemente por nombre de talla.
- Future< int > **actualizar** (Talla *talla*) **async**
Actualiza la información de una talla existente.
- Future< int > **eliminar** (int *id*) **async**
Elimina una talla mediante su identificador.

Atributos privados

- Future< Database > **get_db** **async** => await DatabaseHelper.instance.database
Obtiene una instancia activa de la base de datos.

3.24.1. Descripción detallada

Repositorio encargado de gestionar las operaciones CRUD relacionadas con la entidad Talla.

3.24.2. Documentación de funciones miembro

3.24.2.1. actualizar()

```
Future< int > TallaRepository::actualizar (  
    Talla talla) [inline], [private]
```

Actualiza la información de una talla existente.

La operación utiliza el identificador primario como criterio de actualización.

3.24.2.2. eliminar()

```
Future< int > TallaRepository::eliminar (  
    int id) [inline], [private]
```

Elimina una talla mediante su identificador.

3.24.2.3. insertar()

```
Future< int > TallaRepository::insertar (  
    Talla talla) [inline], [private]
```

Inserta una nueva talla en la base de datos.

El identificador primario es removido del mapa antes de la inserción para permitir autoincremento.

La operación utiliza conflicto tipo abort para evitar registros duplicados o inválidos.

3.24.2.4. obtenerPorId()

```
Future< Talla?> TallaRepository::obtenerPorId (
    int id) [inline], [private]
```

Obtiene una talla mediante su identificador.

Retorna null cuando el registro no existe.

3.24.2.5. obtenerTodas()

```
Future< List< Talla > > TallaRepository::obtenerTodas () [inline], [private]
```

Obtiene todas las tallas registradas ordenadas ascendentemente por nombre de talla.

3.24.3. Documentación de datos miembro

3.24.3.1. async

```
Future<Database> get _db TallaRepository::async => await DatabaseHelper.instance.database
[private]
```

Obtiene una instancia activa de la base de datos.

La documentación de esta clase está generada del siguiente archivo:

- lib/data/repositories/[talla_repository.dart](#)

3.25. Referencia de la clase UnidadRepository

Repositorio encargado de gestionar las operaciones relacionadas con unidades de inventario.

Métodos privados

- Future< List< int > > [insertarUnidades](#) (int idInventario, int cantidad) [async](#)
Inserta múltiples unidades asociadas a un inventario.
- Future< Unidad?> [obtenerPorId](#) (int id) [async](#)
Obtiene una unidad mediante su identificador.
- Future< bool > [pertenece](#) (int idUnidad, int idInventario) [async](#)
Verifica si una unidad pertenece a un inventario específico.
- Future< List< Unidad > > [obtenerPorInventario](#) (int idInventario) [async](#)
Obtiene todas las unidades activas asociadas a un inventario.
- Future< int > [desactivar](#) (int id) [async](#)
Marca una unidad como inactiva.
- Future< int > [reactivar](#) (int id) [async](#)
Reactiva una unidad previamente desactivada.
- Future< int > [marcarPendienteImpresion](#) (int idUnidad) [async](#)
Marca una unidad como pendiente de impresión.
- Future< int > [quitarPendienteImpresion](#) (int idUnidad) [async](#)
Elimina el estado pendiente de impresión de una unidad.
- Future< List< Map< [String](#), dynamic > > > [obtenerPendientesImpresion](#) () [async](#)
Obtiene todas las unidades marcadas como pendientes de impresión.
- Future< int > [eliminar](#) (int id) [async](#)
Elimina una unidad mediante su identificador.

Atributos privados

- `Future< Database > get_db async => await DatabaseHelper.instance.database`
Obtiene una instancia activa de la base de datos.

3.25.1. Descripción detallada

Repositorio encargado de gestionar las operaciones relacionadas con unidades de inventario.

Permite administrar activación, impresión, validaciones y consultas asociadas a unidades físicas.

3.25.2. Documentación de funciones miembro

3.25.2.1. desactivar()

```
Future< int > UnidadRepository::desactivar (  
    int id) [inline], [private]
```

Marca una unidad como inactiva.

Esta operación representa lógicamente una unidad vendida o fuera de disponibilidad.

3.25.2.2. eliminar()

```
Future< int > UnidadRepository::eliminar (  
    int id) [inline], [private]
```

Elimina una unidad mediante su identificador.

3.25.2.3. insertarUnidades()

```
Future< List< int > > UnidadRepository::insertarUnidades (  
    int idInventario,  
    int cantidad) [inline], [private]
```

Inserta múltiples unidades asociadas a un inventario.

Cada unidad se crea inicialmente como activa y sin pendiente de impresión.

La operación se ejecuta dentro de una transacción para garantizar consistencia en inserciones masivas.

3.25.2.4. marcarPendienteImpresion()

```
Future< int > UnidadRepository::marcarPendienteImpresion (  
    int idUnidad) [inline], [private]
```

Marca una unidad como pendiente de impresión.

Esta bandera puede ser utilizada para procesos relacionados con etiquetas, códigos o tickets.

3.25.2.5. obtenerPendientesImpresion()

```
Future< List< Map< String, dynamic > > > UnidadRepository::obtenerPendientesImpresion ()  
[inline], [private]
```

Obtiene todas las unidades marcadas como pendientes de impresión.

La consulta incluye información descriptiva relacionada con escuela, prenda y talla.

3.25.2.6. obtenerPorId()

```
Future< Unidad?> UnidadRepository::obtenerPorId (  
    int id) [inline], [private]
```

Obtiene una unidad mediante su identificador.

Retorna null cuando no existe un registro asociado.

3.25.2.7. obtenerPorInventario()

```
Future< List< Unidad > > UnidadRepository::obtenerPorInventario (  
    int idInventario) [inline], [private]
```

Obtiene todas las unidades activas asociadas a un inventario.

Las unidades inactivas o vendidas son excluidas del resultado.

3.25.2.8. pertenece()

```
Future< bool > UnidadRepository::pertenece (  
    int idUnidad,  
    int idInventario) [inline], [private]
```

Verifica si una unidad pertenece a un inventario específico.

Retorna true cuando existe coincidencia entre la unidad y el inventario indicado.

3.25.2.9. quitarPendienteImpresion()

```
Future< int > UnidadRepository::quitarPendienteImpresion (  
    int idUnidad) [inline], [private]
```

Elimina el estado pendiente de impresión de una unidad.

3.25.2.10. reactivar()

```
Future< int > UnidadRepository::reactivar (  
    int id) [inline], [private]
```

Reactiva una unidad previamente desactivada.

La operación restablece la disponibilidad de la unidad dentro del inventario.

3.25.3. Documentación de datos miembro

3.25.3.1. async

```
Future<Database> get _db UnidadRepository::async => await DatabaseHelper.instance.database [private]
```

Obtiene una instancia activa de la base de datos.

La documentación de esta clase está generada del siguiente archivo:

- [lib/data/repositories/unidad_repository.dart](#)

3.26. Referencia de la clase UsuarioRepository

Repositorio encargado de gestionar las operaciones CRUD relacionadas con usuarios del sistema.

Métodos privados

- Future< int > [insertar](#) (Usuario usuario) [async](#)
Inserta un nuevo usuario en la base de datos.
- Future< Usuario?> [obtenerPorId](#) (int id) [async](#)
Obtiene un usuario mediante su identificador.
- Future< Usuario?> [obtenerPorUsuario](#) (String usuario) [async](#)
Obtiene un usuario utilizando el nombre de usuario.
- Future< List< Usuario > > [obtenerTodos](#) () [async](#)
Obtiene todos los usuarios registrados ordenados alfabéticamente por nombre.
- Future< List< Usuario > > [obtenerActivos](#) () [async](#)
Obtiene únicamente los usuarios activos.
- Future< int > [actualizar](#) (Usuario usuario) [async](#)
Actualiza la información de un usuario existente.
- Future< int > [desactivar](#) (int id) [async](#)
Desactiva un usuario de manera lógica.
- Future< int > [eliminar](#) (int id) [async](#)
Elimina un usuario mediante su identificador.

Atributos privados

- Future< Database > get _db [async](#) => await DatabaseHelper.instance.database
Obtiene una instancia activa de la base de datos.

3.26.1. Descripción detallada

Repositorio encargado de gestionar las operaciones CRUD relacionadas con usuarios del sistema.

Incluye consultas de autenticación lógica, administración de estados y mantenimiento de registros.

3.26.2. Documentación de funciones miembro

3.26.2.1. actualizar()

```
Future< int > UsuarioRepository::actualizar (
    Usuario usuario) [inline], [private]
```

Actualiza la información de un usuario existente.

La operación utiliza el identificador primario como criterio de actualización.

3.26.2.2. desactivar()

```
Future< int > UsuarioRepository::desactivar (
    int id) [inline], [private]
```

Desactiva un usuario de manera lógica.

La operación conserva el registro en la base de datos pero impide que sea considerado activo.

3.26.2.3. eliminar()

```
Future< int > UsuarioRepository::eliminar (
    int id) [inline], [private]
```

Elimina un usuario mediante su identificador.

3.26.2.4. insertar()

```
Future< int > UsuarioRepository::insertar (
    Usuario usuario) [inline], [private]
```

Inserta un nuevo usuario en la base de datos.

El identificador primario es removido del mapa antes de la inserción para permitir autoincremento.

La operación utiliza conflicto tipo abort para evitar registros inválidos o duplicados.

3.26.2.5. obtenerActivos()

```
Future< List< Usuario > > UsuarioRepository::obtenerActivos () [inline], [private]
```

Obtiene únicamente los usuarios activos.

Los registros desactivados son excluidos del resultado.

3.26.2.6. obtenerPorId()

```
Future< Usuario?> UsuarioRepository::obtenerPorId (
    int id) [inline], [private]
```

Obtiene un usuario mediante su identificador.

Retorna null cuando no existe un registro asociado.

3.26.2.7. obtenerPorUsuario()

```
Future< Usuario?> UsuarioRepository::obtenerPorUsuario (
    String usuario) [inline], [private]
```

Obtiene un usuario utilizando el nombre de usuario.

Esta operación puede ser utilizada para procesos de autenticación o validación de existencia.

Retorna null cuando no existe coincidencia.

3.26.2.8. obtenerTodos()

```
Future< List< Usuario > > UsuarioRepository::obtenerTodos () [inline], [private]
```

Obtiene todos los usuarios registrados ordenados alfabéticamente por nombre.

3.26.3. Documentación de datos miembro

3.26.3.1. async

```
Future<Database> get _db UsuarioRepository::async => await DatabaseHelper.instance.database
[private]
```

Obtiene una instancia activa de la base de datos.

La documentación de esta clase está generada del siguiente archivo:

- lib/data/repositories/[usuario_repository.dart](#)

3.27. Referencia de la clase ValidacionInventario

Clase utilitaria que centraliza las validaciones de negocio aplicables a los registros de inventario.

Métodos privados estáticos

- static void `validarCamposObligatorios` (Inventario `inventario`)
Verifica que los identificadores de escuela, prenda y talla sean valores positivos válidos.
- static void `validarPrecio` (Inventario `inventario`)
Verifica que el precio del inventario sea mayor a cero.
- static void `validarCantidadUnidades` (int `cantidad`)
Verifica que la cantidad de unidades a insertar esté dentro del rango permitido (1-9999).
- static Future< void > `validarEliminacion` (int `idInventario`, Future< bool > Function(int) `tieneVentasActivas`,) async
Verifica que el registro de inventario no tenga ventas activas asociadas antes de permitir su eliminación.

3.27.1. Descripción detallada

Clase utilitaria que centraliza las validaciones de negocio aplicables a los registros de inventario.

Todos sus métodos son estáticos y lanzan excepciones ante datos inválidos.

3.27.2. Documentación de funciones miembro

3.27.2.1. `validarCamposObligatorios()`

```
static void ValidacionInventario::validarCamposObligatorios (  
    Inventario inventario) [inline], [static], [private]
```

Verifica que los identificadores de escuela, prenda y talla sean valores positivos válidos.

3.27.2.2. `validarCantidadUnidades()`

```
static void ValidacionInventario::validarCantidadUnidades (  
    int cantidad) [inline], [static], [private]
```

Verifica que la cantidad de unidades a insertar esté dentro del rango permitido (1-9999).

3.27.2.3. `validarEliminacion()`

```
static Future< void > ValidacionInventario::validarEliminacion (  
    int idInventario,  
    Future< bool > Function(int) tieneVentasActivas) [inline], [static], [private]
```

Verifica que el registro de inventario no tenga ventas activas asociadas antes de permitir su eliminación.

Recibe una función delegada para consultar las ventas activas, manteniendo esta clase desacoplada de los repositorios.

3.27.2.4. validarPrecio()

```
static void ValidacionInventario::validarPrecio (
    Inventario inventario) [inline], [static], [private]
```

Verifica que el precio del inventario sea mayor a cero.

La documentación de esta clase está generada del siguiente archivo:

- lib/business/validators/[validacion_inventario.dart](#)

3.28. Referencia de la clase VentaRepository

Repositorio encargado de gestionar operaciones relacionadas con ventas y detalles de venta.

Métodos privados

- Future< int > [insertarVentaYDetalles](#) ({ required Venta [venta](#), required List< DetalleVenta > detalles, }) [async](#)
Inserta una venta junto con todos sus detalles asociados.
- Future< void > [insertarDetallesVenta](#) ({ required int [idVenta](#), required List< DetalleVenta > detalles, }) [async](#)
Inserta detalles adicionales a una venta ya existente.
- Future< void > [eliminarDetalleVenta](#) (int [idDetalleVenta](#)) [async](#)
Elimina un detalle específico de venta.
- Future< int > [contarDetallesVenta](#) (int [idVenta](#)) [async](#)
Obtiene la cantidad de detalles asociados a una venta.
- Future< void > [actualizarEstadoCancelado](#) (int [idVenta](#)) [async](#)
Cambia el estado de una venta a cancelada.
- Future< Venta?> [obtenerPorId](#) (int [id](#)) [async](#)
Obtiene una venta mediante su identificador.
- Future< List< Venta > > [obtenerTodas](#) () [async](#)
Obtiene todas las ventas registradas ordenadas por fecha descendente.
- Future< List< Venta > > [obtenerPorUsuario](#) (int [idUserario](#)) [async](#)
Obtiene todas las ventas realizadas por un usuario específico.
- Future< List< Map< [String](#), dynamic > > > [obtenerDetallesPorVenta](#) (int [idVenta](#),) [async](#)
Obtiene los detalles completos asociados a una venta.
- Future< int > [eliminar](#) (int [id](#)) [async](#)
Elimina una venta mediante su identificador.

Atributos privados

- Future< Database > [get_db](#) [async](#) => await DatabaseHelper.instance.database
Obtiene una instancia activa de la base de datos.

3.28.1. Descripción detallada

Repositorio encargado de gestionar operaciones relacionadas con ventas y detalles de venta.

Centraliza procesos de registro, consulta, cancelación y eliminación de ventas.

3.28.2. Documentación de funciones miembro

3.28.2.1. actualizarEstadoCancelado()

```
Future< void > VentaRepository::actualizarEstadoCancelado (
    int idVenta) [inline], [private]
```

Cambia el estado de una venta a cancelada.

La operación realiza una cancelación lógica sin eliminar el registro físicamente.

3.28.2.2. contarDetallesVenta()

```
Future< int > VentaRepository::contarDetallesVenta (
    int idVenta) [inline], [private]
```

Obtiene la cantidad de detalles asociados a una venta.

Esta operación puede utilizarse para validar si una venta conserva productos relacionados.

3.28.2.3. eliminar()

```
Future< int > VentaRepository::eliminar (
    int id) [inline], [private]
```

Elimina una venta mediante su identificador.

3.28.2.4. eliminarDetalleVenta()

```
Future< void > VentaRepository::eliminarDetalleVenta (
    int idDetalleVenta) [inline], [private]
```

Elimina un detalle específico de venta.

3.28.2.5. insertarDetallesVenta()

```
Future< void > VentaRepository::insertarDetallesVenta (
    { required int idVenta, required List< DetalleVenta > detalles, } ) [inline],
[private]
```

Inserta detalles adicionales a una venta ya existente.

La operación se ejecuta dentro de una transacción para mantener consistencia en inserciones múltiples.

3.28.2.6. insertarVentaYDetalles()

```
Future< int > VentaRepository::insertarVentaYDetalles (
    { required Venta venta, required List< DetalleVenta > detalles, } ) [inline],
[private]
```

Inserta una venta junto con todos sus detalles asociados.

La operación se ejecuta dentro de una transacción para garantizar integridad entre la tabla de ventas y detalle_↔venta.

Retorna el identificador de la venta creada.

3.28.2.7. obtenerDetallesPorVenta()

```
Future< List< Map< String, dynamic > > > VentaRepository::obtenerDetallesPorVenta (
    int idVenta) [inline], [private]
```

Obtiene los detalles completos asociados a una venta.

La consulta integra información relacionada con inventario, escuela, prenda y talla.

3.28.2.8. obtenerPorId()

```
Future< Venta?> VentaRepository::obtenerPorId (
    int id) [inline], [private]
```

Obtiene una venta mediante su identificador.

Retorna null cuando el registro no existe.

3.28.2.9. obtenerPorUsuario()

```
Future< List< Venta > > VentaRepository::obtenerPorUsuario (
    int idUsuario) [inline], [private]
```

Obtiene todas las ventas realizadas por un usuario específico.

El resultado se ordena por fecha descendente.

3.28.2.10. obtenerTodas()

```
Future< List< Venta > > VentaRepository::obtenerTodas () [inline], [private]
```

Obtiene todas las ventas registradas ordenadas por fecha descendente.

3.28.3. Documentación de datos miembro

3.28.3.1. async

```
Future<Database> get _db VentaRepository::async => await DatabaseHelper.instance.database
[private]
```

Obtiene una instancia activa de la base de datos.

La documentación de esta clase está generada del siguiente archivo:

- lib/data/repositories/venta_repository.dart

3.29. Referencia de la clase VentasUseCase

Caso de uso que expone las operaciones de consulta sobre el historial de ventas.

Métodos privados

- **VentasUseCase**({required [VentaRepository](#) ventaRepository}) Future< List< Venta > > [obtenerVentas](#) () async
Retorna todas las ventas registradas en el sistema.
- Future< Venta?> [obtenerVenta](#) (int idVenta) async
Retorna la venta correspondiente al identificador indicado.
- Future< List< Map< [String](#), dynamic > > > [obtenerDetallesVenta](#) (int idVenta) async
Retorna los detalles de una venta enriquecidos con información de unidad, prenda, talla y escuela para su presentación en pantalla.

Atributos privados

- final [VentaRepository](#) _ventaRepository

3.29.1. Descripción detallada

Caso de uso que expone las operaciones de consulta sobre el historial de ventas.

Actúa como intermediario entre la capa de presentación y el repositorio de ventas.

3.29.2. Documentación de funciones miembro

3.29.2.1. obtenerDetallesVenta()

```
Future< List< Map< String, dynamic > > > VentasUseCase::obtenerDetallesVenta (
    int idVenta) [inline], [private]
```

Retorna los detalles de una venta enriquecidos con información de unidad, prenda, talla y escuela para su presentación en pantalla.

3.29.2.2. obtenerVenta()

```
Future< Venta?> VentasUseCase::obtenerVenta (
    int idVenta) [inline], [private]
```

Retorna la venta correspondiente al identificador indicado.

Retorna null si no existe ninguna venta con ese identificador.

3.29.2.3. obtenerVentas()

```
VentasUseCase({required VentaRepository ventaRepository}) Future< List< Venta > > VentasUse↵
Case::obtenerVentas () [inline], [private]
```

Retorna todas las ventas registradas en el sistema.

3.29.3. Documentación de datos miembro

3.29.3.1. _ventaRepository

```
final VentaRepository VentasUseCase::_ventaRepository [private]
```

La documentación de esta clase está generada del siguiente archivo:

- lib/business/usecases/[ventas_usecase.dart](#)

Capítulo 4

Documentación de archivos

4.1. Referencia del archivo `lib/business/providers/auth_provider.dart`

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.2. Referencia del archivo `lib/business/providers/inventario_provider.dart`

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.3. Referencia del archivo `lib/business/services/backup_service.dart`

```
import dart;
```

Clases

- class [BackupInfo](#)
Modelo que representa la información de un archivo de backup generado.
- class [BackupService](#)
Servicio que gestiona las operaciones de backup de la base de datos.

Funciones

- `const BackupResult ({ required this.exitoso, required this.mensaje, this.backup, })`

Variables

- `class BackupInfo exitoso`
Modelo que representa el resultado de una operación de backup.
- `final String mensaje`
Mensaje descriptivo del resultado de la operación.
- `final BackupInfo backup`
Información del backup generado. Null si la operación falló.

4.3.1. Documentación de funciones

4.3.1.1. BackupResult()

```
const BackupResult (
  { required this.exitoso, required this.mensaje, this.backup, } )
```

4.3.2. Documentación de variables

4.3.2.1. backup

```
final BackupInfo backup
```

Información del backup generado. Null si la operación falló.

4.3.2.2. exitoso

```
class BackupInfo exitoso
```

Modelo que representa el resultado de una operación de backup.

Indica si la operación fue exitosa, un mensaje descriptivo y, opcionalmente, la información del backup generado. Indica si la operación de backup se completó sin errores.

4.3.2.3. mensaje

```
final String mensaje
```

Mensaje descriptivo del resultado de la operación.

4.4. Referencia del archivo lib/business/services/print_service.dart

```
import dart;
```

Clases

- class [PrintService](#)

Contrato base para servicios de impresión.

Funciones

- Future< void > [_conectar](#) () async
Establece la conexión BLE con la impresora.
- Future< void > [_desconectar](#) () async
Desconecta el dispositivo BLE actualmente conectado.
- override Future< void > [conectar](#) () async
- override Future< void > [desconectar](#) () async
- override Future< void > [imprimirQR](#) (String id) async
Genera y envía un comando TSPL a la impresora para imprimir una etiqueta con código QR e identificador en texto.

Variables

- abstract class [PrintService](#) [serviceUuid](#) = "49535343-fe7d-4ae5-8fa9-9fafd205e455"
Implementación de [\[PrintService\]](#) para impresoras térmicas que se comunican mediante Bluetooth Low Energy (BLE).
- static const [String](#) [charUuid](#) = "49535343-8841-43f4-a8d4-ecbe34729bb3"
UUID de la característica BLE utilizada para el envío de datos a la impresora.
- static const [String](#) [printerMac](#) = "66:32:DA:6B:86:A7"
Dirección MAC de la impresora térmica objetivo.
- BluetoothDevice [_device](#)
Referencia al dispositivo BLE conectado. Null si no hay conexión activa.
- BluetoothCharacteristic [_char](#)
Característica BLE utilizada para escribir los datos de impresión.

4.4.1. Documentación de funciones

4.4.1.1. [_conectar\(\)](#)

```
Future< void > _conectar ()
```

Establece la conexión BLE con la impresora.

Busca el dispositivo en la lista de dispositivos vinculados por su dirección MAC, se conecta a él y descubre los servicios y características disponibles. Lanza una excepción si la impresora no está vinculada o si no se encuentra el servicio o la característica requerida. Busca la impresora en los dispositivos vinculados comparando la dirección MAC de forma insensible a mayúsculas.

Espera breve posterior a la conexión para permitir que el stack BLE establezca el enlace antes de descubrir servicios.

Itera sobre los servicios descubiertos buscando el UUID de servicio y dentro de él la característica de escritura requerida.

4.4.1.2. `_desconectar()`

```
Future< void > _desconectar ()
```

Desconecta el dispositivo BLE actualmente conectado.

4.4.1.3. `conectar()`

```
override Future< void > conectar ()
```

4.4.1.4. `desconectar()`

```
override Future< void > desconectar ()
```

4.4.1.5. `imprimirQR()`

```
override Future< void > imprimirQR (  
    String id)
```

Genera y envía un comando TSPL a la impresora para imprimir una etiqueta con código QR e identificador en texto.

El comando se fragmenta en paquetes de 20 bytes con un retardo de 80ms entre cada uno para respetar los límites de transferencia BLE. Lanza una excepción si no hay una conexión activa al momento de imprimir. Comando TSPL que define el tamaño de la etiqueta (40x30mm), el espaciado entre etiquetas, el código QR y el texto identificador.

Envía los bytes del comando en fragmentos de 20 bytes usando escritura sin respuesta (`writeWithoutResponse`), con retardo entre fragmentos para evitar desbordamiento en el buffer BLE de la impresora.

4.4.2. Documentación de variables

4.4.2.1. `_char`

```
BluetoothCharacteristic _char
```

Característica BLE utilizada para escribir los datos de impresión.

Null si no se ha establecido la conexión o no se encontró la característica.

4.4.2.2. `_device`

```
BluetoothDevice _device
```

Referencia al dispositivo BLE conectado. Null si no hay conexión activa.

4.4.2.3. charUuid

```
const String charUuid = "49535343-8841-43f4-a8d4-ecbe34729bb3" [static]
```

UUID de la característica BLE utilizada para el envío de datos a la impresora.

4.4.2.4. printerMac

```
const String printerMac = "66:32:DA:6B:86:A7" [static]
```

Dirección MAC de la impresora térmica objetivo.

4.4.2.5. serviceUuid

```
abstract class PrintService serviceUuid = "49535343-fe7d-4ae5-8fa9-9fafd205e455"
```

Implementación de [PrintService] para impresoras térmicas que se comunican mediante Bluetooth Low Energy (BLE).

Utiliza el protocolo TSPL para el envío de comandos de impresión. UUID del servicio BLE expuesto por la impresora. Obtenido mediante herramienta nRF Connect u similar.

4.5. Referencia del archivo lib/business/usecases/alertas_stock_usecase.dart

```
import data;
```

Clases

- class [AlertasStockResult](#)

Agrupar el resultado de la consulta de alertas de stock, separando los items agotados de los que están en nivel crítico.

Enumeraciones

- enum [TipoAlerta](#) { [agotado](#) , [critico](#) }

Define los niveles de severidad para las alertas de stock.

Funciones

- const [AlertaStock](#) ({ required [this.idInventario](#), required [this.escuela](#), required [this.prenda](#), required [this.talla](#), required [this.precio](#), required [this.stock](#), required [this.tipo](#), })
- [AlertasStockUseCase](#) ({required [InventarioRepository](#) inventarioRepository}) Future< [AlertasStockResult](#) > [obtenerAlertas](#) () async

Retorna todas las alertas activas agrupadas en agotados y críticos.

- Future< List< [AlertaStock](#) > > [obtenerAgotados](#) () async

Retorna únicamente los items sin stock (0 unidades disponibles).

- Future< List< [AlertaStock](#) > > [obtenerCriticos](#) () async

Retorna únicamente los items con stock menor o igual a [umbralCritico] y mayor a 0.

- Future< [AlertaStock](#)?> [verificarItem](#) (int [idInventario](#)) async

Verifica si un item específico del inventario tiene una alerta activa.

- Future< String?> [obtenerResumen](#) () async

Genera un texto resumido con el conteo de alertas activas, apto para mostrar en badges o notificaciones.

Variables

- enum `TipoAlerta idInventario`
Modelo que representa una alerta de stock para un ítem de inventario específico.
- final `String escuela`
- final `String prenda`
- final `String talla`
- final double `precio`
- final int `stock`
- final `TipoAlerta tipo`
Tipo de transacción que representa este resultado.
- `String` get `mensaje`
Mensaje descriptivo de la alerta.
- class `AlertasStockResult _inventarioRepository`
Caso de uso encargado de detectar y clasificar alertas de stock en el inventario.
- static const int `umbralCritico` = 3
Umbral de stock crítico (inclusive).

4.5.1. Documentación de enumeraciones

4.5.1.1. TipoAlerta

```
enum TipoAlerta
```

Define los niveles de severidad para las alertas de stock.

Valores de enumeraciones

	agotado
	critico

4.5.2. Documentación de funciones

4.5.2.1. AlertaStock()

```
const AlertaStock (
    { required this.idInventario, required this.escuela, required this.prenda, required
    this.talla, required this.precio, required this.stock, required this.tipo, } )
```

4.5.2.2. obtenerAgotados()

```
Future< List< AlertaStock > > obtenerAgotados ()
```

Retorna únicamente los ítems sin stock (0 unidades disponibles).

4.5.2.3. obtenerAlertas()

```
AlertasStockUseCase({required InventarioRepository inventarioRepository}) Future< AlertasStockResult > obtenerAlertas ()
```

Retorna todas las alertas activas agrupadas en agotados y críticos.

4.5.2.4. obtenerCriticos()

```
Future< List< AlertaStock > > obtenerCriticos ()
```

Retorna únicamente los items con stock menor o igual a [umbralCritico] y mayor a 0.

4.5.2.5. obtenerResumen()

```
Future< String?> obtenerResumen ()
```

Genera un texto resumido con el conteo de alertas activas, apto para mostrar en badges o notificaciones.

Retorna null si no hay alertas activas. Ejemplo de salida: "2 agotados · 5 críticos". Obtiene del repositorio los items con stock agotado y los mapea a instancias de [AlertaStock] con tipo [TipoAlerta.agotado].

Obtiene del repositorio los items con stock crítico y los mapea a instancias de [AlertaStock] con tipo [TipoAlerta.↵ crítico].

4.5.2.6. verificarItem()

```
Future< AlertaStock?> verificarItem (
    int idInventario)
```

Verifica si un item específico del inventario tiene una alerta activa.

Consulta el stock actual del item y lo compara contra las reglas de negocio: stock igual a 0 genera alerta de agotado, stock menor o igual a [umbralCritico] genera alerta crítica. Retorna null si el item no tiene alerta activa.

4.5.3. Documentación de variables

4.5.3.1. _inventarioRepository

```
class AlertasStockResult _inventarioRepository
```

Caso de uso encargado de detectar y clasificar alertas de stock en el inventario.

Consulta el repositorio de inventario para identificar items agotados y críticos, y expone métodos para obtener el resultado completo, filtrado o resumido.

4.5.3.2. escuela

```
final String escuela
```

4.5.3.3. idInventario

```
enum TipoAlerta idInventario
```

Modelo que representa una alerta de stock para un ítem de inventario específico.

Contiene la información del ítem afectado, su stock actual y el tipo de alerta generada.

4.5.3.4. mensaje

```
String get mensaje
```

Valor inicial:

```
{
    switch (tipo) {
        case TipoAlerta.agotado:
            return ' Sin stock -- $escuela · $prenda · Talla $talla';
        case TipoAlerta.critico:
            return ' Stock crítico ($stock piezas) -- $escuela · $prenda · Talla $talla';
    }
}

int get prioridad => tipo == TipoAlerta.agotado ? 0 : 1
```

Mensaje descriptivo de la alerta.

4.5.3.5. precio

```
final double precio
```

4.5.3.6. prenda

```
final String prenda
```

4.5.3.7. stock

```
final int stock
```

4.5.3.8. talla

```
final String talla
```

4.5.3.9. tipo

```
final TipoConfirmacion tipo
```

Tipo de transacción que representa este resultado.

4.5.3.10. umbralCritico

```
const int umbralCritico = 3 [static]
```

Umbral de stock crítico (inclusive).

Items con stock igual o inferior a este valor y mayor a 0 se clasifican como críticos.

4.6. Referencia del archivo lib/business/usecases/login_usecase.dart

```
import dart;
```

Clases

- class [LoginUseCase](#)
Caso de uso para el inicio de sesión.

4.7. Referencia del archivo lib/business/usecases/nueva_order_usecase.dart

```
import data;
```

Clases

- class [NuevaOrdenUseCase](#)
Caso de uso que gestiona la creación de órdenes mixtas, combinando items de venta directa (con unidad física) y pedidos (sin unidad asignada).

Enumeraciones

- enum [TipoItemOrden](#) { [venta](#) , [pedido](#) }
Clasifica el origen de un ítem dentro de una orden.
- enum [TipoConfirmacion](#) { [venta](#) , [pedido](#) }
Indica el tipo de transacción que resultó de la confirmación de la orden.

Funciones

- const [ItemOrden](#) ({ required [this.idInventario](#), required [this.precioUnitario](#), required [this.tipo](#), [this.idUnidad](#), })
- const [NuevaOrdenResult](#) ({ [this.idVenta](#), [this.idPedido](#), required [this.tipo](#), required [this.total](#), required [this.totalPrendas](#), })

Variables

- enum `TipoItemOrden` `idInventario`
Representa un ítem individual dentro de una orden en construcción.
- final double `precioUnitario`
Precio unitario tomado del inventario al momento de agregar el ítem.
- final int `idUnidad`
Identificador de la unidad física asignada. Null si es un pedido sin unidad resuelta.
- final `TipoItemOrden` `tipo`
Indica si el ítem se procesará como venta directa o como pedido.
- bool get `tieneUnidadResuelta` => `idUnidad != null`
Retorna true si el ítem tiene una unidad física asignada.
- enum `TipoConfirmacion` `idVenta`
Resultado de la confirmación de una orden o sub-orden.
- final int `idPedido`
Identificador del pedido generado. Null si la confirmación fue de venta.
- final double `total`
Suma total de los precios unitarios de los ítems procesados.
- final int `totalPrendas`
Cantidad de prendas incluidas en la transacción.

4.7.1. Documentación de enumeraciones

4.7.1.1. TipoConfirmacion

enum `TipoConfirmacion`

Indica el tipo de transacción que resultó de la confirmación de la orden.

Valores de enumeraciones

	venta
	pedido

4.7.1.2. TipoItemOrden

enum `TipoItemOrden`

Clasifica el origen de un ítem dentro de una orden.

[venta] indica que el ítem tiene una unidad física escaneada o resuelta. [pedido] indica que el ítem es un encargo sin unidad física asignada aún.

Valores de enumeraciones

	venta
	pedido

4.7.2. Documentación de funciones

4.7.2.1. ItemOrden()

```
const ItemOrden (
    { required this.idInventario, required this.precioUnitario, required this.tipo,
    this.idUnidad, } )
```

4.7.2.2. NuevaOrdenResult()

```
const NuevaOrdenResult (
    { this.idVenta, this.idPedido, required this.tipo, required this.total, required
    this.totalPrendas, } )
```

4.7.3. Documentación de variables

4.7.3.1. idInventario

```
enum TipoItemOrden idInventario
```

Representa un ítem individual dentro de una orden en construcción.

Puede tener una unidad física asignada ([idUnidad]) si proviene de un escaneo QR, o carecer de ella si fue agregado manualmente como pedido. Identificador del registro de inventario al que pertenece este ítem.

4.7.3.2. idPedido

```
final int idPedido
```

Identificador del pedido generado. Null si la confirmación fue de venta.

4.7.3.3. idUnidad

```
final int idUnidad
```

Identificador de la unidad física asignada. Null si es un pedido sin unidad resuelta.

4.7.3.4. idVenta

```
enum TipoConfirmacion idVenta
```

Resultado de la confirmación de una orden o sub-orden.

Contiene los identificadores generados y el resumen de la transacción. Identificador de la venta generada. Null si la confirmación fue de pedido.

4.7.3.5. precioUnitario

```
final double precioUnitario
```

Precio unitario tomado del inventario al momento de agregar el ítem.

4.7.3.6. tieneUnidadResuelta

```
bool get tieneUnidadResuelta => idUnidad != null
```

Retorna true si el ítem tiene una unidad física asignada.

4.7.3.7. tipo

```
final TipoConfirmacion tipo
```

Indica si el ítem se procesará como venta directa o como pedido.

Tipo de transacción que representa este resultado.

4.7.3.8. total

```
final double total
```

Suma total de los precios unitarios de los ítems procesados.

4.7.3.9. totalPrendas

```
final int totalPrendas
```

Cantidad de prendas incluidas en la transacción.

4.8. Referencia del archivo

lib/business/usecases/pedidos_pendientes_usecase.dart

```
import models;
```

Clases

- class `PedidoPendienteUseCase`

Caso de uso que gestiona el ciclo de vida de los pedidos en estado pendiente.

4.9. Referencia del archivo lib/business/usecases/print_usecase.dart

```
import data;
```

Clases

- class [PrintUseCase](#)

Caso de uso que gestiona la impresión de etiquetas QR para unidades de inventario.

4.10. Referencia del archivo lib/business/usecases/qr_usecase.dart

```
import data;
```

Clases

- class [QrUseCase](#)

Caso de uso que gestiona la interpretación y validación de códigos QR para la identificación de unidades físicas de inventario.

4.11. Referencia del archivo lib/business/usecases/registrar_inventario_usecase.dart

```
import data;
```

Clases

- class [RegistroTallaResult](#)

Resultado del intento de registro de inventario para una talla específica.

- class [RegistrarInventarioUseCase](#)

Caso de uso que gestiona el registro de items de inventario para una combinación de prenda, escuela y una o más tallas.

Funciones

- const [RegistrarInventarioResult](#) ({ required [this.resultados](#), required [this.totalExitosos](#), required [this.totalFallidos](#), })

Variables

- class `RegistroTallaResult resultados`
Resultado global del caso de uso al finalizar el registro de todas las tallas.
- final int `totalExitosos`
- final int `totalFallidos`
- bool get `todosExitosos` => `totalFallidos` == 0
Retorna true si ningún registro falló.
- bool get `algunoExitoso` => `totalExitosos` > 0
Retorna true si al menos un registro fue exitoso.

4.11.1. Documentación de funciones

4.11.1.1. RegistrarInventarioResult()

```
const RegistrarInventarioResult (  
    { required this.resultados, required this.totalExitosos, required this.totalFallidos,  
} )
```

4.11.2. Documentación de variables

4.11.2.1. algunoExitoso

```
bool get algunoExitoso => totalExitosos > 0
```

Retorna true si al menos un registro fue exitoso.

4.11.2.2. resultados

```
class RegistroTallaResult resultados
```

Resultado global del caso de uso al finalizar el registro de todas las tallas.

Agrupar los resultados individuales y expone conteos de éxitos y fallos.

4.11.2.3. todosExitosos

```
bool get todosExitosos => totalFallidos == 0
```

Retorna true si ningún registro falló.

4.11.2.4. totalExitosos

```
final int totalExitosos
```

4.11.2.5. totalFallidos

```
final int totalFallidos
```

4.12. Referencia del archivo

lib/business/usecases/restar_unidades_usecase.dart

```
import data;
```

Enumeraciones

- enum `ResultadoRestarUnidad` {
`ok` , `qrInvalido` , `noExiste` , `noPertenece` ,
`yaDesactivada` }

Define los posibles resultados de intentar desactivar una unidad mediante QR.

Funciones

- `RestarUnidadesUseCase` (`this.unidadRepository`)
- `Future< ResultadoRestarUnidad > ejecutar` (`String qr`, `int idInventario`) `async`

Ejecuta el proceso de desactivación de una unidad a partir de su código QR.

Variables

- enum `ResultadoRestarUnidad` `unidadRepository`

Caso de uso que gestiona la baja de una unidad física de inventario mediante escaneo QR.

4.12.1. Documentación de enumeraciones

4.12.1.1. ResultadoRestarUnidad

```
enum ResultadoRestarUnidad
```

Define los posibles resultados de intentar desactivar una unidad mediante QR.

Valores de enumeraciones

	<code>ok</code>	La unidad fue desactivada correctamente.
	<code>qrInvalido</code>	El contenido del QR no contiene un identificador numérico válido.
	<code>noExiste</code>	No existe ninguna unidad con el identificador extraído del QR.
	<code>noPertenece</code>	La unidad existe pero no pertenece al inventario indicado.
	<code>yaDesactivada</code>	La unidad ya estaba desactivada previamente.

4.12.2. Documentación de funciones

4.12.2.1. ejecutar()

```
Future< ResultadoRestarUnidad > ejecutar (
    String qr,
    int idInventario)
```

Ejecuta el proceso de desactivación de una unidad a partir de su código QR.

Extrae la primera secuencia numérica del QR para obtener el identificador. Valida que la unidad exista, esté activa y pertenezca al inventario recibido antes de proceder con la desactivación. Retorna un [ResultadoRestarUnidad] que describe el resultado de la operación. Regla de negocio: una unidad ya desactivada no puede volver a restarse, ya que representa una prenda que fue vendida o dada de baja con anterioridad.

4.12.2.2. RestarUnidadesUseCase()

```
RestarUnidadesUseCase (
    this. unidadRepository)
```

4.12.3. Documentación de variables

4.12.3.1. unidadRepository

```
enum ResultadoRestarUnidad unidadRepository
```

Caso de uso que gestiona la baja de una unidad física de inventario mediante escaneo QR.

Extrae el identificador del QR, valida la existencia y estado de la unidad, verifica su pertenencia al inventario indicado y la desactiva si todas las validaciones son satisfactorias.

4.13. Referencia del archivo lib/business/usecases/ventas_usecase.dart

```
import models;
```

Clases

- class `VentasUseCase`

Caso de uso que expone las operaciones de consulta sobre el historial de ventas.

4.14. Referencia del archivo lib/business/validators/validacion_inventario.dart

```
import models;
```

Clases

- class [ValidacionInventario](#)

Clase utilitaria que centraliza las validaciones de negocio aplicables a los registros de inventario.

4.15. Referencia del archivo lib/data/database/database_helper.dart

```
import dart;
```

Clases

- class [DatabaseHelper](#)

Helper que gestiona el ciclo de vida de la base de datos SQLite local.

4.16. Referencia del archivo lib/data/repositories/backup_repository.dart

```
import dart;
```

Clases

- class [BackupRepository](#)

Repositorio encargado de gestionar la creación física de archivos de backup de la base de datos SQLite en el almacenamiento local del dispositivo.

4.17. Referencia del archivo lib/data/repositories/escuela_repository.dart

```
import package;
```

Clases

- class [EscuelaRepository](#)

Repositorio que gestiona las operaciones de acceso a datos para la tabla de escuelas.

**4.18. Referencia del archivo
lib/data/repositories/inventario_repository.dart**

```
import package;
```

Clases

- class [InventarioRepository](#)

Repositorio que gestiona las operaciones de acceso a datos para la tabla de inventario y sus consultas relacionadas con stock.

4.19. Referencia del archivo lib/data/repositories/orden_repository.dart

```
import package;
```

Clases

- class [OrdenRepository](#)

Repositorio que gestiona las operaciones de acceso a datos para la tabla de órdenes.

4.20. Referencia del archivo lib/data/repositories/pedido_repository.dart

```
import package;
```

Clases

- class [PedidoRepository](#)

Repositorio encargado de gestionar las operaciones relacionadas con pedidos y detalles de pedido en la base de datos.

4.21. Referencia del archivo lib/data/repositories/prenda_repository.dart

```
import package;
```

Clases

- class [PrendaRepository](#)

Repositorio encargado de gestionar las operaciones CRUD relacionadas con la entidad Prenda.

4.22. Referencia del archivo lib/data/repositories/reporte_repository.dart

```
import package;
```

Clases

- class [ReporteRepository](#)

Repositorio encargado de generar consultas estadísticas y métricas utilizadas en reportes.

4.23. Referencia del archivo lib/data/repositories/talla_repository.dart

```
import package;
```

Clases

- class [TallaRepository](#)

Repositorio encargado de gestionar las operaciones CRUD relacionadas con la entidad Talla.

4.24. Referencia del archivo lib/data/repositories/unidad_repository.dart

```
import package;
```

Clases

- class [UnidadRepository](#)

Repositorio encargado de gestionar las operaciones relacionadas con unidades de inventario.

4.25. Referencia del archivo lib/data/repositories/usuario_repository.dart

```
import package;
```

Clases

- class [UsuarioRepository](#)

Repositorio encargado de gestionar las operaciones CRUD relacionadas con usuarios del sistema.

4.26. Referencia del archivo lib/data/repositories/venta_repository.dart

```
import package;
```

Clases

- class [VentaRepository](#)

Repository encargado de gestionar operaciones relacionadas con ventas y detalles de venta.

4.27. Referencia del archivo lib/main.dart

```
import package;
```

Clases

- class [StatelessWidget](#)

Widget raíz de la aplicación.

Funciones

- void [main](#) () async

Punto de entrada de la aplicación.

4.27.1. Documentación de funciones

4.27.1.1. main()

```
void main ()
```

Punto de entrada de la aplicación.

Inicializa la base de datos local mediante [[DatabaseHelper](#)] y registra todos los providers globales mediante [[MultiProvider](#)] antes de lanzar [[MyApp](#)].

4.28. Referencia del archivo lib/models/models.dart

Enumeraciones

- enum [RolUsuario](#)

Funciones

- enum [RolUsuario toDb](#) ()

Variables

- enum RolUsuario completada
- enum RolUsuario cancelada
- enum RolUsuario pendiente
- enum RolUsuario completado
- enum RolUsuario cancelado
- admin
- vendedor

4.28.1. Documentación de enumeraciones

4.28.1.1. RolUsuario

```
enum RolUsuario
```

4.28.2. Documentación de funciones

4.28.2.1. toDb()

```
enum RolUsuario toDb ()
```

4.28.3. Documentación de variables

4.28.3.1. admin

```
admin
```

4.28.3.2. cancelada

```
enum RolUsuario cancelada
```

4.28.3.3. cancelado

```
enum RolUsuario cancelado
```

4.28.3.4. completada

```
enum RolUsuario completada
```

4.28.3.5. completado

```
enum RolUsuario completado
```

4.28.3.6. pendiente

```
enum RolUsuario pendiente
```

4.28.3.7. vendedor

```
vendedor
```

4.29. Referencia del archivo

lib/presentation/providers/alertas_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.30. Referencia del archivo

lib/presentation/providers/home_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.31. Referencia del archivo

lib/presentation/providers/nueva_orden_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.32. Referencia del archivo lib/presentation/providers/pedidos_pendientes_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.33. Referencia del archivo lib/presentation/providers/print_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.34. Referencia del archivo lib/presentation/providers/qr_pendientes_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.35. Referencia del archivo lib/presentation/providers/qr_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.36. Referencia del archivo

lib/presentation/providers/registrar_inventario_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.37. Referencia del archivo

lib/presentation/providers/reportes_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.38. Referencia del archivo

lib/presentation/providers/ventas_provider.dart

```
import package;
```

Clases

- class [ChangeNotifier](#)
Proveedor de estado para la autenticación de usuarios.

4.39. Referencia del archivo

lib/presentation/screens/administrar_cantidad_screen.dart

```
import package;
```

Clases

- class [StatefulWidget](#)
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- override void `initState ()`
- void `_confirmarAgregar ()`
Valida la cantidad ingresada y muestra un diálogo de confirmación antes de ejecutar la operación de agregar unidades.
- Future< void > `_confirmarRestar ()` async
Navega a [QRScannerScreen] y procesa el resultado del escaneo ejecutando [RestarUnidadesUseCase].
- override Widget `build (BuildContext context)`
Construye la interfaz de la pantalla.

Variables

- class `AdministrarCantidadScreen` extends `StatefulWidget _controller`
Estado interno de [AdministrarCantidadScreen].
- final `_unidadRepo = UnidadRepository()`
Repositorio de acceso a datos de unidades.
- late final `RestarUnidadesUseCase _restarUseCase`
Caso de uso encargado de ejecutar la lógica de negocio para restar unidades mediante validación de QR.

4.39.1. Documentación de funciones

4.39.1.1. `_confirmarAgregar()`

```
void _confirmarAgregar ()
```

Valida la cantidad ingresada y muestra un diálogo de confirmación antes de ejecutar la operación de agregar unidades.

Regla de negocio: no se permite agregar si la cantidad es menor o igual a cero. Al confirmar, delega la operación a [PrintProvider.agregarUnidades], el cual gestiona la impresión de QRs para las nuevas unidades.

4.39.1.2. `_confirmarRestar()`

```
Future< void > _confirmarRestar ()
```

Navega a [QRScannerScreen] y procesa el resultado del escaneo ejecutando [RestarUnidadesUseCase].

Mapea cada valor de [ResultadoRestarUnidad] a un [ScanFeedback] con el tipo de resultado y mensaje correspondiente.

Reglas de negocio aplicadas:

- Una unidad ya desactivada no puede restarse de nuevo.
- Solo se pueden restar unidades que pertenezcan al inventario indicado.
- El QR debe tener un formato válido reconocido por el sistema.

4.39.1.3. build()

```
override Widget build (  
    BuildContext context)
```

Construye la interfaz de la pantalla.

Construye la interfaz gráfica y los enlaces de navegación del menú.

Compone: campo de entrada de cantidad, botones de sumar y restar, y área de mensaje reactivo al estado de [PrintProvider].

- Obtiene instancias de proveedores mediante `Provider.of` para acceder a la lógica de navegación y al estado de alertas.
- Define la estructura `Scaffold` con `AppBar`, `body` y `bottomNavigationBar`.

4.39.1.4. initState()

```
override void initState ()
```

4.39.2. Documentación de variables

4.39.2.1. _controller

```
class AdministrarCantidadScreen extends StatefulWidget _controller
```

Estado interno de [AdministrarCantidadScreen].

Gestiona el ciclo de vida del controlador de texto, la instancia del caso de uso y la limpieza del estado del provider. Controlador vinculado al campo de entrada de cantidad. Se inicializa en "1" como valor mínimo operativo.

4.39.2.2. _restarUseCase

```
late final RestarUnidadesUseCase _restarUseCase
```

Caso de uso encargado de ejecutar la lógica de negocio para restar unidades mediante validación de QR.

4.39.2.3. _unidadRepo

```
final _unidadRepo = UnidadRepository()
```

Repositorio de acceso a datos de unidades.

Se inyecta directamente en [_restarUseCase].

4.40. Referencia del archivo lib/presentation/screens/administrar_prenda_screen.dart

```
import package;
```

Clases

- class [StatefulWidget](#)
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- override void [initState](#) ()
- Future< void > [_cargarDatos](#) () async
Carga los datos del inventario desde el repositorio usando [idInventario].
- void [_actualizarCantidadDesdeProvider](#) ()
Sincroniza [cantidadActual] con el stock registrado en [InventarioProvider].
- Future< void > [_guardarCambios](#) () async
Persiste el precio editado en la base de datos.
- Future< void > [_eliminar](#) () async
Elimina el registro de inventario identificado por [idInventario].
- void [_dialogoGuardar](#) ()
Muestra un diálogo de confirmación antes de ejecutar [_guardarCambios].
- void [_dialogoEliminar](#) ()
Muestra un diálogo de confirmación antes de ejecutar [_eliminar].
- override Widget [build](#) (BuildContext context)
Construye la interfaz de la pantalla.

Variables

- class AdministrarPrendaScreen extends [StatefulWidget](#) [_inventarioRepo](#) = [InventarioRepository](#)()
Estado interno de [AdministrarPrendaScreen].
- Inventario [inventario](#)
Instancia del inventario cargado desde la base de datos.
- bool [cargando](#) = true
Indica si los datos del inventario están siendo cargados.
- late int [cantidadActual](#)
Cantidad actual de unidades, sincronizada con [InventarioProvider].
- final TextEditingController [_precioController](#)
Controlador del campo de texto para editar el precio del inventario.

4.40.1. Documentación de funciones

4.40.1.1. [_actualizarCantidadDesdeProvider](#)()

```
void _actualizarCantidadDesdeProvider ()
```

Sincroniza [cantidadActual] con el stock registrado en [InventarioProvider].

Busca el ítem correspondiente a [idInventario] en la lista del provider y actualiza el estado local. Si el ítem no existe, no realiza cambios.

4.40.1.2. `_cargarDatos()`

```
Future< void > _cargarDatos ()
```

Carga los datos del inventario desde el repositorio usando [idInventario].

Actualiza [_precioController] con el precio actual y establece [cargando] en false al finalizar.

4.40.1.3. `_dialogoEliminar()`

```
void _dialogoEliminar ()
```

Muestra un diálogo de confirmación antes de ejecutar [_eliminar].

4.40.1.4. `_dialogoGuardar()`

```
void _dialogoGuardar ()
```

Muestra un diálogo de confirmación antes de ejecutar [_guardarCambios].

4.40.1.5. `_eliminar()`

```
Future< void > _eliminar ()
```

Elimina el registro de inventario identificado por [idInventario].

Al completarse, cierra la pantalla devolviendo true al caller.

4.40.1.6. `_guardarCambios()`

```
Future< void > _guardarCambios ()
```

Persiste el precio editado en la base de datos.

Regla de negocio: el precio debe ser un valor numérico válido. Si [inventario] es null o el precio no es parseable, la operación se cancela. Al completarse, cierra la pantalla devolviendo true al caller.

4.40.1.7. `build()`

```
override Widget build (  
    BuildContext context)
```

Construye la interfaz de la pantalla.

Muestra un indicador de carga mientras [cargando] es true. Si [inventario] es null tras la carga, muestra un mensaje de error. En caso exitoso, renderiza los datos de la prenda, el campo de precio, el acceso a [AdministrarCantidad↔Screen], y los botones de guardar y eliminar.

4.40.1.8. initState()

```
override void initState ()
```

4.40.2. Documentación de variables

4.40.2.1. _inventarioRepo

```
class AdministrarPrendaScreen extends StatefulWidget _inventarioRepo = InventarioRepository()
```

Estado interno de [AdministrarPrendaScreen].

Gestiona la carga del inventario, la edición del precio, la actualización de cantidad y las operaciones de guardado y eliminación. Repositorio de acceso a datos del inventario.

4.40.2.2. _precioController

```
final TextEditingController _precioController
```

Valor inicial:

```
=  
  TextEditingController()
```

Controlador del campo de texto para editar el precio del inventario.

4.40.2.3. cantidadActual

```
late int cantidadActual
```

Cantidad actual de unidades, sincronizada con [InventarioProvider].

4.40.2.4. cargando

```
bool cargando = true
```

Indica si los datos del inventario están siendo cargados.

4.40.2.5. inventario

```
Inventario inventario
```

Instancia del inventario cargado desde la base de datos.

Es null mientras se encuentra en estado de carga.

4.41. Referencia del archivo

lib/presentation/screens/agregar_prendas_pedido_screen.dart

```
import package;
```

Clases

- class [StatefulWidget](#)
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- Widget [_buildDropdown](#) ({required [String](#) hint, required [String?](#) value, required List< [String](#) > items, required void Function([String?](#)) onChanged})
Construye un dropdown estilizado con sombra y bordes redondeados.
- override Widget [build](#) (BuildContext context)
Construye la interfaz de la pantalla.

Variables

- class [AgregarPrendasPedidoScreen](#) extends [StatefulWidget](#) [_escuelaSeleccionada](#)
Estado interno de [AgregarPrendasPedidoScreen].
- [String](#) [_prendaSeleccionada](#)
Prenda actualmente seleccionada en el filtro. Null si no hay selección.
- final List< [String](#) > [_escuelas](#) = ['UACJ', 'ESC. PRIMARIA FLOR', 'CBTIS 114']
Lista estática de escuelas disponibles para el filtro.
- final List< [String](#) > [_prendas](#) = ['Playera', 'Falda', 'Pantalón']
Lista estática de tipos de prenda disponibles para el filtro.
- final List< Map< [String](#), dynamic > > [_prendasDisponibles](#)
Lista estática de prendas disponibles en inventario con sus atributos.

4.41.1. Documentación de funciones

4.41.1.1. [_buildDropdown\(\)](#)

```
Widget \_buildDropdown (  
    {required String hint, required String? value, required List< String > items,  
    required void Function(String?) onChanged} )
```

Construye un dropdown estilizado con sombra y bordes redondeados.

Recibe el texto de [hint], el [value] seleccionado, la lista de [items] y el callback [onChanged] a ejecutar al cambiar la selección.

4.41.1.2. build()

```
override Widget build (  
    BuildContext context)
```

Construye la interfaz de la pantalla.

Compone los filtros de escuela y prenda, la lista de inventario disponible y los botones de guardar y cancelar.

4.41.2. Documentación de variables

4.41.2.1. _escuelas

```
final List<String> _escuelas = ['UACJ', 'ESC. PRIMARIA FLOR', 'CBTIS 114']
```

Lista estática de escuelas disponibles para el filtro.

4.41.2.2. _escuelaSeleccionada

```
class AgregarPrendasPedidoScreen extends StatefulWidget _escuelaSeleccionada
```

Estado interno de [AgregarPrendasPedidoScreen].

Gestiona los filtros de escuela y prenda, y la lista de inventario disponible. Escuela actualmente seleccionada en el filtro. Null si no hay selección.

4.41.2.3. _prendas

```
final List<String> _prendas = ['Playera', 'Falda', 'Pantalón']
```

Lista estática de tipos de prenda disponibles para el filtro.

4.41.2.4. _prendasDisponibles

```
final List<Map<String, dynamic> > _prendasDisponibles
```

Valor inicial:

```
= [  
  {'titulo': 'ESC. PRIMARIA FLOR - Playera', 'talla': 'M', 'stock': 8, 'precio': 120.0},  
  {'titulo': 'ESC. PRIMARIA FLOR - Falda', 'talla': 'M', 'stock': 3, 'precio': 180.0},  
]
```

Lista estática de prendas disponibles en inventario con sus atributos.

4.41.2.5. _prendaSeleccionada

```
String _prendaSeleccionada
```

Prenda actualmente seleccionada en el filtro. Null si no hay selección.

4.42. Referencia del archivo

lib/presentation/screens/bottom_nav_bar.dart

```
import package;
```

Clases

- class [StatelessWidget](#)
Widget raíz de la aplicación.

4.43. Referencia del archivo lib/presentation/screens/exito_screen.dart

```
import package;
```

Clases

- class [StatefulWidget](#)
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- class [ExitoScreen](#) extends [StatefulWidget](#) `initState ()`
Estado asociado al componente [ExitoScreen].
- const [ExitoScreen](#) (`{super.key}`)
Constructor principal de la clase [ExitoScreen].
- override `State< ExitoScreen > createState ()`
- override `Widget build (BuildContext context)`
Construye la jerarquía visual de componentes de la interfaz de usuario.

4.43.1. Documentación de funciones

4.43.1.1. build()

```
override Widget build (  
    BuildContext context)
```

Construye la jerarquía visual de componentes de la interfaz de usuario.

Retorna una estructura basada en un lienzo [Scaffold] monocromático que aloja un indicador gráfico circular de confirmación y un bloque textual centralizado.

4.43.1.2. createState()

```
override State< ExitoScreen > initState::createState ()
```

4.43.1.3. ExitoScreen()

```
const initState::ExitoScreen (
    {super.key} )
```

Constructor principal de la clase [ExitoScreen].

4.43.1.4. initState()

```
class ExitoScreen extends StatefulWidget initState ()
```

Estado asociado al componente [ExitoScreen].

Gestiona el ciclo de vida del widget y controla la lógica de tiempo para la destrucción y desapilación de la pantalla de la pila de navegación. Subsistema de inicialización del estado del widget.

Invoca la ejecución asíncrona de la regla de negocio encargada del cierre automático de la interfaz mediante un retraso programado.

4.44. Referencia del archivo lib/presentation/screens/home_screen.dart

```
import package;
```

Clases

- class `StatelessWidget`
Widget raíz de la aplicación.

Funciones

- class `HomeScreen` extends `StatelessWidget` `HomeView` ({ super.key, })
Vista interna que define la estructura visual de la pantalla principal.
- const `HomeScreen` ({ super.key, })
Constructor principal de la clase [HomeScreen].
- override Widget `build` (BuildContext context,)
Construye la jerarquía visual delegando la representación a [HomeView].

4.44.1. Documentación de funciones

4.44.1.1. build()

```
override Widget HomeView::build (
    BuildContext context)
```

Construye la jerarquía visual delegando la representación a [HomeView].

Construye la interfaz gráfica y los enlaces de navegación del menú.

Este widget actúa como contenedor simple que devuelve la vista real implementada en [HomeView]. No mantiene estado propio.

- Obtiene instancias de proveedores mediante `Provider.of` para acceder a la lógica de navegación y al estado de alertas.
- Define la estructura `Scaffold` con `AppBar`, `body` y `bottomNavigationBar`.

4.44.1.2. HomeScreen()

```
const HomeView::HomeScreen (
    { super.key, } )
```

Constructor principal de la clase [HomeScreen].

4.44.1.3. HomeView()

```
class HomeScreen extends StatelessWidget HomeView (
    { super.key, } )
```

Vista interna que define la estructura visual de la pantalla principal.

Gestiona la disposición de los elementos del menú principal y establece la conexión con el manejador de estado [HomeProvider] para delegar la lógica de navegación cuando corresponde. Constructor principal de la clase [HomeView].

4.45. Referencia del archivo lib/presentation/screens/inventario_screen.dart

```
import package;
```

Clases

- class [StatelessWidget](#)
Widget raíz de la aplicación.

Funciones

- const [_ListaInventario](#) ({ required [this.items](#), })
Constructor que recibe la lista de items.
- override Widget [build](#) (BuildContext context,) *Construye el [ListView] agrupando por el campo 'prenda'.*

Variables

- class InventarioScreen extends [StatelessWidget](#) [String](#)
Lista que renderiza los elementos del inventario agrupados por prenda.
- class InventarioScreen extends [StatelessWidget](#) [items](#)

4.45.1. Documentación de funciones

4.45.1.1. _ListaInventario()

```
const _ListaInventario (
    { required this.items, } )
```

Constructor que recibe la lista de items.

4.45.1.2. build()

```
override Widget build (
    BuildContext context)
```

Construye el [ListView] agrupando por el campo 'prenda'.

- Mantiene una variable local `ultimaPrenda` para detectar cambios de grupo.
- Cada vez que `'item['prenda']difiere deultimaPrenda`` se renderiza un header.

4.45.2. Documentación de variables

4.45.2.1. items

```
class InventarioScreen extends StatelessWidget items
```

4.45.2.2. String

```
class InventarioScreen extends StatelessWidget String
```

Lista que renderiza los elementos del inventario agrupados por prenda.

- Recibe una lista de mapas con los campos esperados: 'prenda', 'talla', 'precio', 'stock', 'id', 'escuela'.
- Implementa lógica simple para mostrar un header cada vez que cambia la prenda. Lista de items de inventario representados como mapas dinámicos.

4.46. Referencia del archivo lib/presentation/screens/login_screen.dart

```
import package;
```

Clases

- class `StatefulWidget`
Pantalla para modificar la cantidad de unidades de un inventario específico.
- class `StatelessWidget`
Widget raíz de la aplicación.

Funciones

- override void `initState ()`
- override void `dispose ()`
- Future< void > `_handleLogin ()` async
Método que se ejecuta al presionar "Entrar".
- override Widget `build (BuildContext context)`

Variables

- class LoginScreen extends [StatefulWidget](#) [_usuarioController](#) = TextEditingController()
Estado asociado a [LoginScreen].
- final [_passwordController](#) = TextEditingController()
- bool [_verPassword](#) = false
- late AnimationController [_animController](#)
- late Animation< double > [_fadeAnim](#)
- late Animation< Offset > [_slideAnim](#)

4.46.1. Documentación de funciones

4.46.1.1. [_handleLogin\(\)](#)

```
Future< void > _handleLogin ()
```

Método que se ejecuta al presionar "Entrar".

- Obtiene el [AuthProvider] sin suscribirse a cambios.
- Llama a `login` del proveedor con los valores de los controladores.
- Si la autenticación es exitosa navega a la ruta '/home' pasando el usuario.
- Maneja el estado `mounted` antes de realizar navegación para evitar errores.

4.46.1.2. [build\(\)](#)

```
override Widget build (  
    BuildContext context)
```

4.46.1.3. [dispose\(\)](#)

```
override void dispose ()
```

4.46.1.4. [initState\(\)](#)

```
override void initState ()
```

4.46.2. Documentación de variables

4.46.2.1. [_animController](#)

```
late AnimationController _animController
```

4.46.2.2. `_fadeAnim`

```
late Animation<double> _fadeAnim
```

4.46.2.3. `_passwordController`

```
final _passwordController = TextEditingController()
```

4.46.2.4. `_slideAnim`

```
late Animation<Offset> _slideAnim
```

4.46.2.5. `_usuarioController`

```
class LoginScreen extends StatefulWidget _usuarioController = TextEditingController()
```

Estado asociado a [LoginScreen].

- Mantiene controladores de texto para los campos de entrada.
- Controla la visibilidad de la contraseña y las animaciones de entrada.
- Gestiona la interacción con [AuthProvider] para realizar el login.

4.46.2.6. `_verPassword`

```
bool _verPassword = false
```

4.47. Referencia del archivo lib/presentation/screens/nueva_orden_screen.dart

```
import package;
```

Clases

- class `StatefulWidget`
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- override void `initState ()`
- override void `dispose ()`
- Widget `_buildInputDecoration` ({ required Widget child, })
Contenedor estilizado que envuelve campos de entrada.
- Widget `_buildDropdown` ({ required `String?` value, required List< `String` > `items`, required void Function(`String?`) `onChanged`, })
Dropdown reutilizable con estilo consistente.
- Map< `String`, dynamic > `_agruparItem` ({ required `ItemOrden` item, required List< `ItemOrden` > `items`, required List< Map< `String`, dynamic > > `inventario`, })
Agrupar un `ItemOrden` con otros del mismo inventario y tipo.

Variables

- class `AgregarPedidoScreen` extends `StatefulWidget_clienteController`
Estado de la pantalla de agregar pedido.
- `String` `_escuelaSeleccionada`
- `String` `_prendaSeleccionada`
- `String` `_tallaSeleccionada`

4.47.1. Documentación de funciones

4.47.1.1. `_agruparItem()`

```
Map< String, dynamic > _agruparItem (
    { required ItemOrden item, required List< ItemOrden > items, required List< Map<
String, dynamic > > inventario, } )
```

Agrupar un `ItemOrden` con otros del mismo inventario y tipo.

- Busca los metadatos del inventario (escuela, prenda, talla).
- Calcula la cantidad de ítems iguales en la orden y el precio total.

4.47.1.2. `_buildDropdown()`

```
Widget _buildDropdown (
    { required String? value, required List< String > items, required void Function(String?)
onChanged, } )
```

Dropdown reutilizable con estilo consistente.

- value: valor seleccionado.
- items: lista de opciones.
- onChanged: callback al cambiar selección.

4.47.1.3. `_buildInputDecoration()`

```
Widget _buildInputDecoration (
    { required Widget child, } )
```

Contenedor estilizado que envuelve campos de entrada.

- Aplica fondo blanco, radio de borde y sombra ligera.

4.47.1.4. `dispose()`

```
override void dispose ()
```

4.47.1.5. `initState()`

```
override void initState ()
```

4.47.2. Documentación de variables

4.47.2.1. `_clienteController`

```
class AgregarPedidoScreen extends StatefulWidget _clienteController
```

Valor inicial:

```
=
    TextEditingController()
```

Estado de la pantalla de agregar pedido.

- Mantiene controladores y selección local para filtros (escuela, prenda, talla).
- Inicializa la carga de inventario a través de `NuevaOrdenProvider`.

4.47.2.2. `_escuelaSeleccionada`

```
String _escuelaSeleccionada
```

4.47.2.3. `_prendaSeleccionada`

```
String _prendaSeleccionada
```

4.47.2.4. `_tallaSeleccionada`

```
String _tallaSeleccionada
```

4.48. Referencia del archivo

lib/presentation/screens/pedidos_pendientes_screen.dart

```
import package;
```

Clases

- class [StatefulWidget](#)
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- class [PedidosPendientesScreen](#) extends [StatefulWidget](#) `initState ()`
Estado asociado a [PedidosPendientesScreen].
- const [PedidosPendientesScreen](#) (`{ super.key, }`)
Constructor por defecto de la pantalla de pedidos pendientes.
- override `State< PedidosPendientesScreen > createState ()`
- override `Widget build (BuildContext context,)`
- `Widget _buildPedidoCard (BuildContext context, Pedido pedido,)`
Construye la tarjeta visual que representa un pedido pendiente.

4.48.1. Documentación de funciones

4.48.1.1. `_buildPedidoCard()`

```
Widget _buildPedidoCard (  
    BuildContext context,  
    Pedido pedido)
```

Construye la tarjeta visual que representa un pedido pendiente.

- Muestra información básica: id, cliente, fecha y estado.
- Incluye acción "Ver" que navega a [RevisarPedidoScreen] y espera un resultado booleano que indica si el pedido fue eliminado o modificado.
- Si el resultado es `true`, se recargan los pedidos desde el provider.

4.48.1.2. `build()`

```
override Widget build (  
    BuildContext context)
```

4.48.1.3. `createState()`

```
override State< PedidosPendientesScreen > initState::createState ()
```

4.48.1.4. initState()

```
class PedidosPendientesScreen extends StatefulWidget initState ()
```

Estado asociado a [PedidosPendientesScreen].

- Inicializa la carga de pedidos pendientes una vez que el primer frame ha sido renderizado.
- Observa cambios en el provider para actualizar la UI.

4.48.1.5. PedidosPendientesScreen()

```
const initState::PedidosPendientesScreen (  
    { super.key, } )
```

Constructor por defecto de la pantalla de pedidos pendientes.

4.49. Referencia del archivo lib/presentation/screens/qr_pendientes_screen.dart

```
import package;
```

Clases

- class `StatelessWidget`
Widget raíz de la aplicación.

Funciones

- class `QrPendientesScreen` extends `StatelessWidget _QrPendientesView` ()
Vista interna que consume [QrPendientesProvider] y renderiza la UI.
- const `QrPendientesScreen` ({ super.key, })
Constructor por defecto.
- override Widget `build` (BuildContext context,)
Construye el árbol de widgets y envuelve la vista en un [ChangeNotifierProvider].

4.49.1. Documentación de funciones

4.49.1.1. _QrPendientesView()

```
class QrPendientesScreen extends StatelessWidget _QrPendientesView ()
```

Vista interna que consume [QrPendientesProvider] y renderiza la UI.

- Muestra estados: cargando, vacío o lista de pendientes.
- Permite reimprimir QRs y actualizar el estado en el repositorio.

4.49.1.2. build()

```
override Widget _QrPendientesView::build (  
    BuildContext context)
```

Construye el árbol de widgets y envuelve la vista en un [ChangeNotifierProvider].

- Se inyecta una instancia de [QrPendientesProvider] con su repositorio.
- Se llama a `cargarPendientes()` inmediatamente para iniciar la carga de datos.

4.49.1.3. QrPendientesScreen()

```
const _QrPendientesView:QrPendientesScreen (  
    { super.key, } )
```

Constructor por defecto.

4.50. Referencia del archivo lib/presentation/screens/qr_screen.dart

```
import package;
```

Clases

- class [StatefulWidget](#)
Pantalla para modificar la cantidad de unidades de un inventario específico.
- class [CustomPainter](#)
Overlay visual del scanner que dibuja:

Enumeraciones

- enum [ResultadoScan](#) { [ok](#) , [error](#) , [duplicado](#) }
Resultado genérico para cualquier pantalla de escaneo.

Funciones

- [ScanFeedback](#) ({ required [this.resultado](#), required [this.mensaje](#), })
Constructor que obliga a proporcionar resultado y mensaje.
- override void [initState](#) ()
- Future< void > [_handleScan](#) (String qr,) async
Maneja el resultado del escaneo.
- override void [dispose](#) ()
- override Widget [build](#) (BuildContext context,)

Variables

- enum `ResultadoScan` `resultado`
Modelo de feedback configurable que encapsula el resultado del escaneo y un mensaje legible para mostrar al usuario.
- final `String` `mensaje`
Mensaje descriptivo asociado al resultado.
- class `QRScannerScreen` extends `StatefulWidget` `controller`
Estado asociado a `[QRScannerScreen]`.
- bool `_procesando`
Indicador para evitar procesar múltiples códigos simultáneamente.
- `String` `_mensaje` = ""
Mensaje temporal que se muestra en pantalla tras el escaneo.
- Color `_colorMensaje`
Color del contenedor del mensaje (verde/rojo/transparente).

4.50.1. Documentación de enumeraciones

4.50.1.1. ResultadoScan

enum `ResultadoScan`

Resultado genérico para cualquier pantalla de escaneo.

- `ok`: escaneo exitoso.
- `error`: ocurrió un error al procesar el escaneo.
- `duplicado`: el código ya fue procesado previamente.

Valores de enumeraciones

	ok
	error
	duplicado

4.50.2. Documentación de funciones

4.50.2.1. _handleScan()

```
Future< void > _handleScan (
    String qr)
```

Maneja el resultado del escaneo.

- Evita reentradas usando `_procesando`.
- Llama al callback `widget.onScan` y actualiza el mensaje y color según el feedback.
- Espera un breve lapso para mostrar el mensaje y luego lo oculta.

4.50.2.2. build()

```
override Widget build (  
    BuildContext context)
```

4.50.2.3. dispose()

```
override void dispose ()
```

4.50.2.4. initState()

```
override void initState ()
```

4.50.2.5. ScanFeedback()

```
ScanFeedback (  
    { required this.resultado, required this.mensaje, } )
```

Constructor que obliga a proporcionar resultado y mensaje.

4.50.3. Documentación de variables

4.50.3.1. _colorMensaje

```
Color _colorMensaje
```

Valor inicial:

```
=  
    Colors.transparent
```

Color del contenedor del mensaje (verde/rojo/transparente).

4.50.3.2. _mensaje

```
String _mensaje = ""
```

Mensaje temporal que se muestra en pantalla tras el escaneo.

4.50.3.3. _procesando

```
bool _procesando
```

Valor inicial:

```
=  
    false
```

Indicador para evitar procesar múltiples códigos simultáneamente.

4.50.3.4. controller

```
class QRScannerScreen extends StatefulWidget controller
```

Estado asociado a [QRScannerScreen].

- Controla el `MobileScannerController`.
- Gestiona la lógica de procesamiento para evitar reentradas.
- Muestra mensajes temporales de resultado con color según el tipo. Controlador del paquete `mobile_scanner` para manejar la cámara.

4.50.3.5. mensaje

```
final String mensaje
```

Mensaje descriptivo asociado al resultado.

4.50.3.6. resultado

```
enum ResultadoScan resultado
```

Modelo de feedback configurable que encapsula el resultado del escaneo y un mensaje legible para mostrar al usuario.

Resultado del escaneo (ok, error, duplicado).

4.51. Referencia del archivo lib/presentation/screens/registrar_prenda_screen.dart

```
import package;
```

Clases

- class `StatefulWidget`
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- override void `initState` ()
- override void `dispose` ()
- void `_guardarCambios` () async
Valida el formulario y ejecuta el caso de uso para registrar la prenda.
- Future< void > `_mostrarDialogoEscuela` () async
Muestra un diálogo para crear una nueva escuela.
- InputDecoration `_inputDecoration` ()
Estilo reutilizable para los campos de entrada del formulario.
- override Widget `build` (BuildContext context)

Variables

- class RegistrarPrendaScreen extends `StatefulWidget` `_formKey` = `GlobalKey<FormState>()`
Estado de la pantalla de registro de prenda.
- final `_precioController` = `TextEditingController()`
- int `_idPrendaSeleccionado`
- int `_idTallaSeleccionada`
- int `_idEscuela`
- bool `_cargando` = false

4.51.1. Documentación de funciones

4.51.1.1. `_guardarCambios()`

```
void _guardarCambios ()
```

Valida el formulario y ejecuta el caso de uso para registrar la prenda.

Reglas de negocio y comportamiento:

- Valida que el formulario sea válido y que los dropdowns tengan selección.
- Muestra un `SnackBar` si faltan campos obligatorios.
- Mientras se ejecuta el caso de uso, `_cargando` es true para deshabilitar el botón.
- Al finalizar, muestra un `SnackBar` con el resultado: éxito o errores detallados.

4.51.1.2. `_inputDecoration()`

```
InputDecoration _inputDecoration ()
```

Estilo reutilizable para los campos de entrada del formulario.

- Aplica fondo blanco, bordes y color de borde consistente con el diseño.

4.51.1.3. `_mostrarDialogoEscuela()`

```
Future< void > _mostrarDialogoEscuela ()
```

Muestra un diálogo para crear una nueva escuela.

- Si el usuario ingresa un nombre válido, delega en el provider para crearla.
- Si la creación es exitosa, selecciona la escuela recién creada en el formulario.

4.51.1.4. build()

```
override Widget build (  
    BuildContext context)
```

4.51.1.5. dispose()

```
override void dispose ()
```

4.51.1.6. initState()

```
override void initState ()
```

4.51.2. Documentación de variables

4.51.2.1. _cargando

```
bool _cargando = false
```

4.51.2.2. _formKey

```
class RegistrarPrendaScreen extends StatefulWidget _formKey = GlobalKey<FormState>()
```

Estado de la pantalla de registro de prenda.

- Mantiene el estado del formulario, controladores y selecciones locales.
- Carga catálogos iniciales desde [RegistrarInventarioProvider] en initState.

4.51.2.3. _idEscuela

```
int _idEscuela
```

4.51.2.4. _idPrendaSeleccionado

```
int _idPrendaSeleccionado
```

4.51.2.5. _idTallaSeleccionada

```
int _idTallaSeleccionada
```

4.51.2.6. `_precioController`

```
final _precioController = TextEditingController()
```

4.52. Referencia del archivo `lib/presentation/screens/reportes_screen.dart`

```
import package;
```

Clases

- class `StatelessWidget`
Widget raíz de la aplicación.

Funciones

- class `ReportesScreen` extends `StatelessWidget _ReportesView ()`
Vista que consume [ReportesProvider] y renderiza la UI de reportes.
- const `ReportesScreen` ({super.key})
- override Widget `build` (BuildContext context)
- const `_SectionTitle` ({required `this.titulo`})
- const `_RankingItem` ({ required `this.nombre`, required `this.monto`, required `this.subtitulo`, required `this.proporcion`, })
- const `_EmptyState` ({required `this.mensaje`})

Variables

- class `_FiltrosSection` extends `StatelessWidget titulo`
Widget simple para mostrar títulos de sección en mayúsculas y estilo pequeño.
- class `_StatCard` extends `StatelessWidget nombre`
Modelo simple que representa un elemento del ranking.
- final double `monto`
- final `String` `subtitulo`
- final double `proporcion`
- class `_RankingCard` extends `StatelessWidget mensaje`
Widget que muestra un estado vacío con icono y mensaje.

4.52.1. Documentación de funciones

4.52.1.1. `_EmptyState()`

```
const _EmptyState (
  {required this.mensaje} )
```

4.52.1.2. `_RankingItem()`

```
const _RankingItem (
    { required this.nombre, required this.monto, required this.subtitulo, required
    this.proporcion, } )
```

4.52.1.3. `_ReportesView()`

```
class ReportesScreen extends StatelessWidget _ReportesView ()
```

Vista que consume [ReportesProvider] y renderiza la UI de reportes.

- Muestra filtros (año, mes, escuela), resumen de métricas y rankings.
- Gestiona estados: cargando, vacío o con datos.

4.52.1.4. `_SectionTitle()`

```
const _SectionTitle (
    {required this.titulo} )
```

4.52.1.5. `build()`

```
override Widget _ReportesView::build (
    BuildContext context)
```

4.52.1.6. `ReportesScreen()`

```
const _ReportesView::ReportesScreen (
    {super.key} )
```

4.52.2. Documentación de variables

4.52.2.1. `mensaje`

```
class _RankingCard extends StatelessWidget mensaje
```

Widget que muestra un estado vacío con icono y mensaje.

4.52.2.2. `monto`

```
final double monto
```

4.52.2.3. nombre

```
class _StatCard extends StatelessWidget nombre
```

Modelo simple que representa un elemento del ranking.

4.52.2.4. proporcion

```
final double proporcion
```

4.52.2.5. subtítulo

```
final String subtítulo
```

4.52.2.6. título

```
class _FiltrosSection extends StatelessWidget título
```

Widget simple para mostrar títulos de sección en mayúsculas y estilo pequeño.

4.53. Referencia del archivo

lib/presentation/screens/revisar_pedido_screen.dart

```
import package;
```

Clases

- class `StatefulWidget`
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- class `RevisarPedidoScreen` extends `StatefulWidget` `initState ()`
Estado asociado a la pantalla RevisarPedidoScreen.
- const `RevisarPedidoScreen` (`{ super.key, required this.idPedido, }`)
Constructor principal de la pantalla de revisión de pedidos.
- override `State< RevisarPedidoScreen > createState ()`
- override `Widget build` (`BuildContext context,`)

Variables

- final int `idPedido`
Identificador único del pedido que será consultado.

4.53.1. Documentación de funciones

4.53.1.1. build()

```
override Widget build (  
    BuildContext context)
```

4.53.1.2. createState()

```
override State< RevisarPedidoScreen > initState::createState ()
```

4.53.1.3. initState()

```
class RevisarPedidoScreen extends StatefulWidget initState ()
```

Estado asociado a la pantalla RevisarPedidoScreen.

Gestiona la carga de detalles del pedido, el registro de códigos QR y las acciones relacionadas con el flujo operativo del pedido.

4.53.1.4. RevisarPedidoScreen()

```
const initState::RevisarPedidoScreen (  
    { super.key, required this.idPedido, } )
```

Constructor principal de la pantalla de revisión de pedidos.

4.53.2. Documentación de variables

4.53.2.1. idPedido

```
final int idPedido
```

Identificador único del pedido que será consultado.

4.54. Referencia del archivo lib/presentation/screens/revisar_venta_screen.dart

```
import package;
```

Clases

- class `StatefulWidget`

Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- override void `initState` ()
- Future< void > `_devolverPrenda` ({ required int idDetalleVenta, required int `idUnidad`, required int `idInventario`, }) async
Devuelve una prenda asociada a la venta.
- override Widget `build` (BuildContext context,)

Variables

- class RevisarVentaScreen extends `StatefulWidget` `_ventaRepository`
Estado asociado a [RevisarVentaScreen].
- final `_unidadRepository`

4.54.1. Documentación de funciones

4.54.1.1. `_devolverPrenda()`

```
Future< void > _devolverPrenda (
    { required int idDetalleVenta, required int idUnidad, required int idInventario,
  } )
```

Devuelve una prenda asociada a la venta.

Flujo:

1. Reactiva la unidad en el repositorio de unidades.
2. Elimina el detalle de venta correspondiente.
3. Intenta reimprimir el QR; si falla, marca la unidad como pendiente de impresión.
4. Recalcula si la venta quedó vacía y, en ese caso, elimina la venta completa.
5. Recarga listados y notifica al usuario mediante `SnackBars`.

4.54.1.2. `build()`

```
override Widget build (
    BuildContext context)
```

4.54.1.3. `initState()`

```
override void initState ()
```

4.54.2. Documentación de variables

4.54.2.1. _unidadRepository

```
final _unidadRepository
```

Valor inicial:

```
=  
    UnidadRepository()
```

4.54.2.2. _ventaRepository

```
class RevisarVentaScreen extends StatefulWidget _ventaRepository
```

Valor inicial:

```
=  
    VentaRepository()
```

Estado asociado a [RevisarVentaScreen].

- Carga los detalles de la venta en `initState`.
- Proporciona la lógica para devolver prendas y actualizar vistas relacionadas.

4.55. Referencia del archivo lib/presentation/screens/ventas_screen.dart

```
import package;
```

Clases

- class `StatefulWidget`
Pantalla para modificar la cantidad de unidades de un inventario específico.

Funciones

- override void `initState` ()
- override Widget `build` (BuildContext context,)
- `String _nombreMes` (int mes,)
Devuelve el nombre del mes en español para un número de mes (1-12).

Variables

- class `VentasScreen` extends `StatefulWidget _anioSeleccionado`
Estado de la pantalla de ventas.
- int `_mesSeleccionado`

4.55.1. Documentación de funciones

4.55.1.1. `_nombreMes()`

```
String _nombreMes (  
    int mes)
```

Devuelve el nombre del mes en español para un número de mes (1-12).

4.55.1.2. `build()`

```
override Widget build (  
    BuildContext context)
```

4.55.1.3. `initState()`

```
override void initState ()
```

4.55.2. Documentación de variables

4.55.2.1. `_anioSeleccionado`

```
class VentasScreen extends StatefulWidget _anioSeleccionado
```

Estado de la pantalla de ventas.

- Mantiene filtros locales (`_anioSeleccionado`, `_mesSeleccionado`).
- Solicita la carga inicial de ventas en `initState`.

4.55.2.2. `_mesSeleccionado`

```
int _mesSeleccionado
```

4.56. Referencia del archivo `lib/presentation/widgets/alertas_popup.dart`

```
import package;
```

Clases

- class `StatelessWidget`
Widget raíz de la aplicación.

Funciones

- `const _ItemAlerta` ({ required `this.color`, required `this.titulo`, required `this.subtitulo`, required `this.stock`, })
- `override Widget build` (`BuildContext context`,)

Variables

- `class AlertasPopup` extends `StatelessWidget` `color`
- `final String titulo`
- `final String subtitulo`
- `final String stock`

4.56.1. Documentación de funciones

4.56.1.1. `_ItemAlerta()`

```
const _ItemAlerta (  
    { required this.color, required this.titulo, required this.subtitulo, required  
this.stock, } )
```

4.56.1.2. `build()`

```
override Widget build (  
    BuildContext context)
```

4.56.2. Documentación de variables

4.56.2.1. `color`

```
class AlertasPopup extends StatelessWidget color
```

4.56.2.2. `stock`

```
final String stock
```

4.56.2.3. `subtitulo`

```
final String subtitulo
```

4.56.2.4. `titulo`

```
final String titulo
```

